SIMATS ENGINEERING

CSA3703 – SOFTWARE TESTING

ASSIGNMENT 1 – CO1

Apply, Analyze and Justify Software Testing Principles and Defect Management Strategies for a Hospital Outpatient Management System

Quality Assurance of the Lakeside Hospital "MediQueue" Outpatient System

Submission detail	Information
Student Name	Srikesh K
Register Number	192321040
Department / Section	Information Technology
Course Code and Name	CSA3703 – Software Testing
Assignment Nature	Individual
Duration	1 week
Total Marks	50
Date of Submission	4-11-26
Faculty Name	Dr Gnanajeyaraman Rajaram

EXECUTIVE SUMMARY

This report develops a measurable testing and defect-management approach for Lakeside Hospital's MediQueue outpatient system. The system supports registration, appointment booking, consultation billing, pharmacy dispensing, laboratory-report delivery and SMS reminders. The scenario is safety- and revenue-sensitive: a missed age boundary can deny a concession, a repeated payment can create duplicate billing, a concurrency defect can double-book a doctor, and slow report delivery can interrupt clinical workflows.

The supplied operating baseline is 47 KLOC across six modules, a three-week release cycle, 100 person-hours of test effort, 800 concurrent peak users and a response-time target of no more than two seconds. In the previous release, 135 of 164 defects were detected before release and 29 were reported by users after release. The baseline defect-removal efficiency (DRE) is therefore 135/164 = 82.32%. Nine of the 29 escaped defects were described as critical, so the priority is not merely to increase the number of tests but to improve risk coverage, test-oracle reliability, traceability and release gates.

Across the ten tasks, the recommended direction is continuous, risk-based testing with automated regression for deterministic business rules; explicit requirement and code reviews; independent test design; pharmacist participation in prescription acceptance criteria; a controlled defect repository; and a release gate that combines functional, concurrency, performance and patient-safety evidence. Automation does not remove the need for domain review or exploratory testing.

Key measure	Baseline / recommendation	Interpretation
Total modules / size	6 modules / 47 KLOC	Defect counts should be normalized by module size.
Test capacity	100 person-hours per 3-week release	Effort must be risk-allocated rather than evenly divided.
Previous release DRE	82.32% (135 ÷ 164)	17.68% of recorded defects escaped pre-release testing.
Escaped defects	29; 9 critical	Safety-focused triage and root-cause prevention are required.
Peak load / response target	800 concurrent / ≤ 2 seconds	Performance testing must reproduce the morning peak.
Primary recommendation	Continuous engineering + risk-based automated gates	Best balance of early feedback, repeatability and controlled release risk.

Important evidence note: the brief provides ten named defects but does not provide the full 164-record defect log or measured tool timings. For this report, additional counts, effort values, detection outcomes, test runs and repository timings are labelled as modelled or illustrative examples. They demonstrate calculations and decision logic and must not be represented as real execution results unless independently reproduced.

CONTENTS

1. 1. Case background, system rules and method
2. 2. Task 1 – Testing activity models
3. 3. Task 2 – Process models for test activities
4. 4. Task 3 – Test oracle strategies
5. 5. Task 4 – Test prioritisation strategies
6. 6. Task 5 – Models of test responsibility
7. 7. Task 6 – Defect origin analysis
8. 8. Task 7 – Defect classification schemes
9. 9. Task 8 – Defect tracking repositories
10. 10. Task 9 – Integrated test and defect-management practice
11. 11. Task 10 – Re-apply and adapt the practice
12. 12. Annexures: test cases, illustrative defect log, calculations, release checklist and references

REPORT CONVENTIONS AND ASSUMPTIONS

All currency values are expressed in Indian rupees. The fee calculation follows the supplied business rule exactly: under age 5 is free; ages 5–17 receive 20% off; ages 18–59 pay the full ₹500; age 60 and above receive 40% off. The nearest-rupee rounding rule is half-up, so a value ending in exactly 0.5 rounds upward. For the stated ₹500 base fee, expected fees are ₹0, ₹400, ₹500 and ₹300 respectively.

The cancellation rule is a ₹100 fee only when cancellation occurs strictly less than two hours before the appointment. At exactly two hours, the wording indicates no late fee. The maximum is two active appointments per patient. Prescriptions can be dispensed within seven days of their date; the exact treatment of the seventh-day boundary should be confirmed by the product owner and pharmacist.

Where a task requests experiments, seeded faults, independent raters, tool screenshots or timings, the report provides reproducible plans and clearly marked illustrative outcomes. A real submission should replace these with actual Python test output, real tracker screenshots and the student's recorded observations.

Term	Meaning in this report
Fault / defect	A flaw in a requirement, design, code, test, configuration or data artefact.
Failure	Observable incorrect behaviour when a fault is activated.
Error	A human action or misunderstanding that introduces a fault.
DRE	Defects detected before release divided by all known defects in the measurement set.
KLOC	One thousand lines of code; used here as a size-normalization denominator.
Modelled data	An explicit example or assumption, not a measured production result.

1. CASE BACKGROUND AND TEST BASIS

1.1 System overview

MediQueue is a web-based outpatient management system with an Angular front end, Node.js back end and PostgreSQL database. Its modules form a connected patient journey: registration establishes identity and demographic data; appointments reserve clinician time; billing calculates and collects consultation charges; pharmacy validates and dispenses prescriptions; lab reports expose clinical results; reminders communicate appointment timing.

Testing must consider both module-level correctness and cross-module transactions. For example, changing an appointment can affect slot availability, late-cancellation fees and SMS reminders. A successful payment response must not create a second bill if the browser refreshes or the network retries. A valid report must display the correct issue date and remain available within the response-time target.

Module	Size (KLOC)	Key risk examples	Primary test focus
Registration	4	D5: apostrophe causes database error	Input validation, encoding, database parameterization, privacy
Appointment	10	D3/D4/D7: cancellation, race, stale reminder	Boundary rules, concurrency, rescheduling and notification integration
Billing	12	D1/D2/D8: concession and duplicate bill	Age boundaries, arithmetic oracle, idempotency, payment reconciliation
Pharmacy	9	D9: prescription older than 7 days dispensed	Validity boundaries, quantity, expiry and duplicate-dispense controls
Lab Reports	7	D6/D10: slow page and wrong date field	Performance, data mapping, authorization and PDF verification
Reminders	5	D7: old appointment time sent	Event-driven rescheduling, timing, message content and delivery status

1.2 Business rules translated into testable statements

Rule ID	Testable requirement	High-value boundaries
BR-01	Consultation base fee is ₹500.	Base fee, zero/negative age validation if applicable
BR-02	Age < 5: fee ₹0.	Age 4 and age 5
BR-03	Age 5–17: 20% concession; fee ₹400.	Age 5, 17 and 18
BR-04	Age 18–59: no concession; fee ₹500.	Age 18, 59 and 60
BR-05	Age ≥ 60: 40% concession; fee ₹300.	Age 60 and 61
BR-06	Cancellation less than 2 hours before slot incurs ₹100.	1 h 59 min; exactly 2 h; 2 h 1 min
BR-07	Maximum 2 active appointments per patient.	0, 1, 2 and attempted third booking
BR-08	Prescription dispensed only within 7 days of its date.	Day 6, day 7, day 8 and day 9

BR-09	Response time ≤ 2 seconds under specified peak conditions.	800 concurrent users; p95/p99 and error rate

TASK 1 – TESTING AS AN ENGINEERING ACTIVITY: COMPARE TESTING ACTIVITY MODELS

Assignment weight and evidence expectations are taken from the supplied Software Testing Assignment 1 brief. The analysis below preserves the MediQueue scenario and its stated business rules. Any added experimental counts are identified as illustrative.

1.1 Task interpretation and evaluation criteria

Three models are compared for the Billing and Appointment modules: test-last, review-plus-test, and continuous engineering activity. The key distinction is when testing begins and whether planning, design, execution and measurement are integrated throughout development. The case is not adequately served by a final screen check because the observed failures include boundary logic, transaction idempotency, concurrency and ambiguous requirements.

Evaluation criteria are defect detection, earliest detection stage, repeatability, ambiguity discovery, traceability, effort within 100 person-hours and residual risk. The illustrative seeded-fault set contains five faults: age > 60 instead of ≥ 60; 10% instead of 20% concession; four-hour instead of two-hour cancellation limit; age 4 treated as age 5; and omission of the ₹100 cancellation fee. Two planted ambiguities are whether exactly two hours is chargeable and whether rescheduling cancels/replaces a pending reminder.

Evidence table

Model	Planning/design	Illustrative faults found	Effort (h)	Detection	Faults/hour	Escaped seeded faults
A. Test-last	Mostly after coding	3 of 5	12	60%	0.25	2
B. Review + test	Review then system test	4 of 5	18	80%	0.22	1
C. Continuous	From requirements through builds	5 of 5	24	100%	0.21	0

Interpretation and control implication

The model must be judged by where defects are found, not by a high raw test count. Early detection reduces rework and helps expose ambiguous requirements before they become code.

1.2 Model A – test-last

Under test-last, testers receive a nearly complete build and inspect screens after coding. The model may identify visible symptoms such as a wrong displayed fee, but it has weak leverage over requirement ambiguity, code-level design choices and concurrency races. If the test list is informal, the team may never test age 60, age 4, the exact two-hour cancellation boundary or repeated payment confirmation.

Because late findings require rework across code, integration and regression, the apparent reduction in early planning effort can be offset by expensive repair and retesting. The model is useful only as a limited final system check, not as the primary quality strategy for a hospital workflow.

Evidence table

Model	Planning/design	Illustrative faults found	Effort (h)	Detection	Faults/hour	Escaped seeded faults
A. Test-last	Mostly after coding	3 of 5	12	60%	0.25	2
B. Review + test	Review then system test	4 of 5	18	80%	0.22	1
C. Continuous	From requirements through builds	5 of 5	24	100%	0.21	0

1.3 Model B – review-plus-test

Review-plus-test adds requirement and code review before system testing. Reviewers can spot the age comparator, incorrect percentage, inconsistent cancellation threshold and missing fee when the relevant rules and code are visible. A requirement review can also expose the phrase “less than two hours” as a boundary that must distinguish 1 hour 59 minutes from exactly two hours.

However, if execution occurs only at system-test stage, integration defects and race conditions can remain hidden until late. Reviews are also vulnerable to shared assumptions: reviewers may all interpret rescheduling the same way unless the ambiguity is explicitly raised with the product owner.

Evidence table

Model	Planning/design	Illustrative faults found	Effort (h)	Detection	Faults/hour	Escaped seeded faults
A. Test-last	Mostly after coding	3 of 5	12	60%	0.25	2
B. Review + test	Review then system test	4 of 5	18	80%	0.22	1
C. Continuous	From requirements through builds	5 of 5	24	100%	0.21	0

1.4 Model C – continuous engineering activity

Continuous engineering starts requirement analysis and test design before coding, reviews acceptance criteria, creates unit tests, executes integration and system tests on each build, and measures results. A decision table captures the age bands and fee outputs; boundary-value tests exercise ages 4, 5, 17, 18, 59 and 60. Cancellation tests cover just below, exactly at and just above two hours.

Automation makes the same tests repeatable after changes, while exploratory and concurrency tests target behaviours that are not fully represented by deterministic unit tests. The model still needs human judgement: the LLM or test framework cannot resolve the business meaning of rescheduling without an approved requirement.

Evidence table

Model	Planning/design	Illustrative faults found	Effort (h)	Detection	Faults/hour	Escaped seeded faults
A. Test-last	Mostly after coding	3 of 5	12	60%	0.25	2
B. Review + test	Review then system test	4 of 5	18	80%	0.22	1
C. Continuous	From requirements through builds	5 of 5	24	100%	0.21	0

Interpretation and control implication

The model must be judged by where defects are found, not by a high raw test count. Early detection reduces rework and helps expose ambiguous requirements before they become code.

1.5 Illustrative seeded-fault results

To make the comparison transparent, the following outcomes are a modelled example rather than measured execution. Assume the five faults are assessed independently and each model has different activities. A single fault may be detected in a review or test run; the count is not a claim about an actual hospital release.

Model A detects three of five faults through selected system scenarios. Model B detects four of five through reviews plus system tests. Model C detects all five in the planned test set and also identifies both requirement ambiguities before implementation. These outcomes illustrate why early test design improves coverage; they must be replaced by actual test logs for formal evidence.

Evidence table

Activity	A: test-last	B: review + test	C: continuous
Requirement ambiguity review	Absent/late	Before system test	At definition and each change
Test design	Screen checklist	Derived from reviewed rules	Traceable, risk-based and maintained
Execution	Late system pass	System pass after review	Unit, integration, system, regression

Measurement	Weak	Some defect counts	Coverage, escape, trend and risk metrics
Prevention	Low	Moderate	High when feedback is acted upon

1.6 Effort, detection and escaped-cost comparison

Detection percentage = faults detected ÷ five seeded faults × 100. Faults detected per person-hour = faults detected ÷ effort. For illustration, allocate 12 hours to A, 18 hours to B and 24 hours to C for the focused Billing/Appointment exercise; the rest of the 100-hour budget remains for cross-module regression, performance, pharmacy and release checks.

The cost model in the assignment gives ₹800 at requirements/design, ₹2,500 during coding, ₹9,000 at system test and ₹45,000 after release. If a fault escapes to users, its assumed repair cost is much higher. The cost calculation below therefore treats escaped-fault cost as a risk estimate, not a guaranteed invoice.

Evidence table

Activity	A: test-last	B: review + test	C: continuous
Requirement ambiguity review	Absent/late	Before system test	At definition and each change
Test design	Screen checklist	Derived from reviewed rules	Traceable, risk-based and maintained
Execution	Late system pass	System pass after review	Unit, integration, system, regression
Measurement	Weak	Some defect counts	Coverage, escape, trend and risk metrics
Prevention	Low	Moderate	High when feedback is acted upon

1.7 Engineering-activity comparison and decision

Test-last is strongest in late execution but weakest in planning, early design, prevention and measurement. Review-plus-test adds prevention and requirement scrutiny. Continuous engineering includes planning, test design, execution, measurement and feedback at each build. Continuous work makes the test suite a maintained engineering artefact rather than a one-time checklist.

Recommended choice: Model C within the 100-hour budget, implemented proportionately. Automate age/cancellation/payment invariants; review ambiguous requirements; reserve dedicated effort for race conditions and load tests; and use manual exploratory testing for workflow gaps. Residual risk remains for unmodelled behaviours, environmental differences and rare timing interleavings.

Evidence table

Activity	A: test-last	B: review + test	C: continuous
Requirement ambiguity review	Absent/late	Before system test	At definition and each change

Test design	Screen checklist	Derived from reviewed rules	Traceable, risk-based and maintained
Execution	Late system pass	System pass after review	Unit, integration, system, regression
Measurement	Weak	Some defect counts	Coverage, escape, trend and risk metrics
Prevention	Low	Moderate	High when feedback is acted upon

Interpretation and control implication

The model must be judged by where defects are found, not by a high raw test count. Early detection reduces rework and helps expose ambiguous requirements before they become code.

1.8 LLM-generated plan critique

An LLM can propose useful test ideas, but its plan must be checked against the exact rules. A plausible generated plan may miss the age-60 boundary, incorrectly use 10% concession, treat exactly two hours as chargeable, omit a duplicate-payment idempotency test, or ignore the 800-user/two-second performance target. It may also write generic statements such as “test all inputs” without concrete values and expected outcomes.

The safe workflow is: provide the approved requirements; ask for traceable test cases; independently recompute expected values; review boundary and negative cases; run tests; and log omissions. No LLM-generated expected result should become an oracle without human verification.

Evidence table

Activity	A: test-last	B: review + test	C: continuous
Requirement ambiguity review	Absent/late	Before system test	At definition and each change
Test design	Screen checklist	Derived from reviewed rules	Traceable, risk-based and maintained
Execution	Late system pass	System pass after review	Unit, integration, system, regression
Measurement	Weak	Some defect counts	Coverage, escape, trend and risk metrics
Prevention	Low	Moderate	High when feedback is acted upon

Task conclusion

Adopt continuous engineering activity, with review-plus-test discipline and a risk-based allocation. Test-last should remain a final verification layer only.

1.9 Operational meaning of early testing

Early testing means that testability and correctness are considered when requirements are written, not merely when a build is ready. For MediQueue, the consultation-fee rule should be converted into a decision table before

implementation. The table exposes the exact boundaries at ages 4, 5, 17, 18, 59 and 60, and forces the analyst to state whether the age is calculated on the appointment date or the registration date. Without that decision, two implementations can each appear reasonable while producing different bills.

Review activities are tests of work products. A reviewer can detect a missing requirement, an ambiguous comparator, a duplicated acceptance criterion or an unhandled error path without executing the application. These reviews are not substitutes for dynamic tests: a code review may identify a missing transaction boundary, while a synchronized integration test demonstrates whether two requests can actually reserve the same slot. The strongest model combines prevention with executable evidence.

Feedback latency is a practical measure of process quality. If a defect is introduced in a requirement on Monday but found after release, the team has lost the opportunity to correct it cheaply. A continuous model aims to shorten the time between introduction and discovery through small changes, targeted automated checks and clear failure reports. Short feedback loops are especially valuable in a three-week release cycle because they reduce the number of changes that must be diagnosed together.

Evidence table

Activity	When performed	MediQueue example	Evidence retained
Requirement review	Before design	Clarify age boundaries and cancellation threshold	Approved rule table
Design review	Before implementation	Review payment and appointment state transitions	Architecture review notes
Unit test	During coding	Test fee function for every age band	Automated test result
Integration test	On connected build	Verify booking and payment commit together	Request and database logs
System test	Release candidate	Complete patient booking journey	Run report
Exploratory test	Throughout	Interrupt payment, refresh and retry	Session notes and defect IDs
Regression test	After each change	Re-run fee, slot and prescription rules	Pipeline report
Acceptance test	Before release	Clinical/business representative validates workflow	Signed acceptance record

Interpretation and control implication: The table is a planning aid. Each expected result must be checked against the approved requirement and verified in the intended build. Where a requirement remains ambiguous, record the decision and owner before treating the case as a release gate.

1.10 Testability as a design property

Testability is the degree to which the system makes it practical to observe and control its behaviour. A system is easier to test when business rules are isolated in small functions, state transitions are explicit, logs carry correlation identifiers, and test environments can create deterministic patients, appointments and payments. Testability should be discussed during design because retrofitting it after defects appear may require changes to architecture, logging and data setup.

For example, a payment retry test needs a way to simulate a gateway timeout after the gateway has accepted the payment but before the browser receives confirmation. If the integration cannot simulate this condition, the team may test only the easy success path and falsely assume retries are safe. A sandbox, controllable stub or fault-injection seam allows the case to be reproduced without charging real patients.

Testability does not mean exposing confidential patient data or weakening production security. Test fixtures should use synthetic identities, restricted access and resettable records. Logging should record transaction identifiers and status transitions without unnecessarily recording medical details or payment secrets. The goal is sufficient observability for diagnosis with privacy and security preserved.

Evidence table

Design choice	Testing benefit	Possible drawback	Control
Pure fee-calculation function	Fast, deterministic boundary tests	May not prove UI displays the same amount	Add UI-to-API integration case
Correlation ID per workflow	Connects logs across services	Must propagate consistently	Validate ID propagation
Payment sandbox	Safe retry and failure simulation	Sandbox may differ from production	Contract tests and controlled pilot
Synthetic patient factory	Repeatable test data	Fixtures can become unrealistic	Review representative scenarios
Database constraints	Protect invariants under concurrency	Migration and error mapping required	Test constraint violations
Feature flags	Controlled rollout and rollback	Flag combinations multiply	Maintain approved flag matrix

Interpretation and control implication: The table is a planning aid. Each expected result must be checked against the approved requirement and verified in the intended build. Where a requirement remains ambiguous, record the decision and owner before treating the case as a release gate.

1.11 Metrics that explain quality rather than decorate it

Metrics should guide decisions, not be used as a substitute for understanding. Test count alone can rise while critical behaviours remain untested. For this case, useful measures include requirement coverage, boundary coverage, critical workflow pass rate, defect-removal efficiency, escaped-defect severity, mean time to diagnose and the proportion of failures that recur after a fix. Every metric needs a defined numerator, denominator, time window and data source.

Defect-removal efficiency can be calculated as pre-release defects divided by all known defects, multiplied by 100. Using the supplied scenario counts, 135 pre-release defects and 29 post-release defects produce 135 ÷ (135 + 29) × 100, approximately 82.3%. This is a scenario-level calculation, not proof that all defects had equal severity or that the defect log is complete. It should be interpreted alongside the severity of escaped defects.

Metrics can be gamed if they are used as individual performance targets. A team pressured to maximize closure count may split defects artificially, close unresolved issues or avoid reporting difficult problems. A better review asks what changed, what risk remains and what evidence supports release. The purpose is learning and risk control rather than blame.

Evidence table

Metric	Definition	Useful interpretation	Caution
Requirement coverage	Requirements with linked tests ÷ total approved requirements	Highlights untested scope	A linked test may still be weak
Boundary coverage	Required boundary points executed ÷ identified points	Useful for fee and date rules	Keep expected results independently reviewed
Pass rate	Passed executed tests ÷ all executed tests	Shows current build health	Do not count Not Run as Pass
DRE	Pre-release defects ÷ all known defects × 100	Estimates containment before release	Depends on complete, consistent defect records
Escaped critical defects	Critical defects first found after release	Direct signal of residual risk	Review exposure and detection controls
Reopen rate	Reopened defects ÷ closed defects	May indicate weak fixes or unclear acceptance	Interpret with defect complexity
Feedback time	Time from change to actionable result	Measures pipeline responsiveness	Separate queue time from execution time

Interpretation and control implication: The table is a planning aid. Each expected result must be checked against the approved requirement and verified in the intended build. Where a requirement remains ambiguous, record the decision and owner before treating the case as a release gate.

TASK 2 – ROLE OF PROCESS IN SOFTWARE QUALITY: COMPARE PROCESS MODELS

Assignment weight and evidence expectations are taken from the supplied Software Testing Assignment 1 brief. The analysis below preserves the MediQueue scenario and its stated business rules. Any added experimental counts are identified as illustrative.

2.1 Process alternatives and feature scope

The feature “book an appointment with concession and online payment” crosses requirement interpretation, slot reservation, fee calculation, payment processing, database persistence and notification scheduling. The three processes are incremental delivery with end testing, risk-driven spiral with test gates, and a DevOps pipeline with automated gates. They differ in feedback timing, automation and release control.

Process quality matters because 29 defects escaped in the previous release. A process should not merely increase the number of tests; it should make failures visible early, prevent known regressions and block releases when critical controls fail.

Evidence table

Repair stage	Unit cost	15 defects	Total
Requirements/design	₹800	15	₹12,000
Coding	₹2,500	15	₹37,500
System testing	₹9,000	15	₹1,35,000
After release	₹45,000	15	₹6,75,000

Interpretation and control implication

The process should make quality evidence repeatable and visible. A passing build is a necessary gate, but high-risk workflows still need integrated and domain acceptance testing.

2.2 A – incremental delivery with end testing

With this approach, registration, appointment booking, concession calculation and payment may be built in increments, but meaningful integrated testing waits until the final increment. Unit tests may occur informally, yet the defining feature is that formal testing is concentrated near the end.

The advantage is low initial process overhead and a simple plan. The disadvantages are late feedback, difficult fault isolation and a risk that a mid-cycle concession-rule change invalidates several completed components. A duplicate bill can be difficult to diagnose if payment retry, browser refresh and persistence were integrated only at the end.

Evidence table

Repair stage	Unit cost	15 defects	Total
Requirements/design	₹800	15	₹12,000
Coding	₹2,500	15	₹37,500
System testing	₹9,000	15	₹1,35,000
After release	₹45,000	15	₹6,75,000

2.3 B – risk-driven spiral with test gates

Each iteration begins with risk analysis and ends with a gate. For MediQueue, a high-risk iteration would cover payment idempotency, slot locking and fee-boundary correctness before expanding into lower-risk visual details. The gate should require evidence that the highest-severity risks have been tested and that unresolved critical defects are not accepted.

This process is effective when risks are uncertain and expert review is available. It can incur more planning and documentation overhead than a lightweight pipeline. If requirements change every cycle, risk analysis and gate artefacts must be updated each time; otherwise the process becomes ceremony rather than control.

Evidence table

Repair stage	Unit cost	15 defects	Total
Requirements/design	₹800	15	₹12,000
Coding	₹2,500	15	₹37,500
System testing	₹9,000	15	₹1,35,000
After release	₹45,000	15	₹6,75,000

2.4 C – DevOps pipeline with automated gates

Each commit triggers build verification, unit tests, static analysis and regression tests. Integration tests verify database constraints, payment idempotency and appointment locking. A deployment gate blocks the build if critical tests fail or agreed coverage thresholds are not met. Performance and acceptance tests may run on scheduled or release-candidate builds where their cost is higher.

The main benefit is fast, repeatable feedback. It does not mean every test can run on every commit: concurrency and load tests may be staged to protect pipeline speed. A controlled test pyramid combines many unit tests, a smaller number of integration tests and focused end-to-end tests.

Evidence table

Repair stage	Unit cost	15 defects	Total
Requirements/design	₹800	15	₹12,000
Coding	₹2,500	15	₹37,500
System testing	₹9,000	15	₹1,35,000
After release	₹45,000	15	₹6,75,000

Interpretation and control implication

The process should make quality evidence repeatable and visible. A passing build is a necessary gate, but high-risk workflows still need integrated and domain acceptance testing.

2.5 Defect removal efficiency and repair-cost calculation

Baseline DRE = defects found before release ÷ total known defects = 135 ÷ 164 = 0.82317, or 82.32%. The complementary escape fraction is 29 ÷ 164 = 17.68%. The ratio is a historical measure, not a guarantee of future quality; it is affected by reporting completeness and defect severity.

For 15 similar defects at one phase, the assignment's per-defect costs yield: requirements/design 15 × ₹800 = ₹12,000; coding 15 × ₹2,500 = ₹37,500; system testing 15 × ₹9,000 = ₹1,35,000; after release 15 × ₹45,000 = ₹6,75,000. The large cost gradient supports earlier detection.

Evidence table

Process	Feedback speed	Change handling	Cycle fit	Main weakness
A. Incremental + end test	Slow	Rework late	Weak for 3 weeks	Integration surprises
B. Risk-driven gates	At each iteration	Risk review repeated	Good with disciplined gates	Review overhead
C. DevOps gates	Every commit	Automated regression	Strong after setup	Pipeline/test maintenance

2.6 Timelines and responsibilities

Illustrative timeline A: requirements review → coding increments → final integration → unit/integration/system/acceptance testing concentrated late → release. Developers own most unit checks; testers discover many integration issues at the end; clinicians review acceptance late.

Timeline B: risk review → iteration design/build → unit and integration tests → risk-based system tests → gate review → next iteration. Developers, independent testers, QA lead and pharmacist participate at different gates. Timeline C: every commit → build/unit/static checks → integration regression → staged system/performance tests → release-candidate acceptance → deployment gate.

Evidence table

Process	Feedback speed	Change handling	Cycle fit	Main weakness
A. Incremental + end test	Slow	Rework late	Weak for 3 weeks	Integration surprises
B. Risk-driven gates	At each iteration	Risk review repeated	Good with disciplined gates	Review overhead
C. DevOps gates	Every commit	Automated regression	Strong after setup	Pipeline/test maintenance

2.7 Maturity, quality gates and recommendation

The observed late defects indicate weaknesses in requirement traceability, regression automation and release evidence. A maturity assessment should be evidence-based rather than assigning a formal CMMI/TMMi level without an audit. Two clear gaps are inadequate handling of boundary/idempotency cases and performance validation under the specified peak load.

Recommend C, with risk-driven gate principles from B for payment, pharmacy and clinical data. A reasonable starting gate is 100% pass of critical safety and billing tests, zero open critical defects, and at least 85% branch coverage of changed business-rule code, with exclusions documented. Coverage is a signal, not proof of correctness. Testing should not be shortened merely because a deadline approaches; scope can be prioritized, but mandatory safety gates remain.

Evidence table

Process	Feedback speed	Change handling	Cycle fit	Main weakness
A. Incremental + end test	Slow	Rework late	Weak for 3 weeks	Integration surprises
B. Risk-driven gates	At each iteration	Risk review repeated	Good with disciplined gates	Review overhead
C. DevOps gates	Every commit	Automated regression	Strong after setup	Pipeline/test maintenance

Interpretation and control implication

The process should make quality evidence repeatable and visible. A passing build is a necessary gate, but high-risk workflows still need integrated and domain acceptance testing.

2.7 Process entry and exit criteria

Entry and exit criteria prevent a process from depending on vague statements such as ‘testing is ready’ or ‘the build looks stable’. Entry criteria specify the minimum evidence required to begin an activity. For integration testing, this may include a versioned build, migrated test database, approved interface contracts, known test accounts and a list of included changes. If these are absent, test results may be invalid or impossible to reproduce.

Exit criteria specify what must be true before work is considered complete. For a release candidate, criteria could include all P0 tests passed, no unresolved critical patient-safety defect, all known payment and booking defects triaged, agreed performance evidence available and business acceptance recorded. A percentage threshold should never override a mandatory safety gate. For example, a 98% overall pass rate does not compensate for a failed duplicate-payment test.

Criteria should be proportionate. Requiring a full load test for every tiny documentation change would waste effort, while skipping performance testing for a database query change would be risky. The pipeline can classify changes by affected component and risk, execute fast checks on every commit and reserve longer-running concurrency, load and end-to-end suites for appropriate integration or release stages.

Evidence table

Stage	Entry criteria	Exit criteria	If criteria fail
Requirement analysis	Business owner and rules available	Ambiguities recorded and decisions approved	Block design for unresolved critical rule
Unit testing	Code and unit-test environment available	Critical branch and boundary tests pass	Return to developer
Integration	Compatible component versions deployed	Interface and transaction checks pass	Isolate failing component pair
System testing	Release candidate and stable data set	Critical journeys and negative cases pass	Raise defects and retest
Performance	Workload model and environment agreed	Results interpreted against target	Tune or document approved exception
Acceptance	Representative workflow and approver available	Business/clinical acceptance recorded	Do not infer approval from silence
Release	Approved candidate and rollback plan	Release gates and ownership complete	Hold release or formally accept bounded risk

Interpretation and control implication: The table is a planning aid. Each expected result must be checked against the approved requirement and verified in the intended build. Where a requirement remains ambiguous, record the decision and owner before treating the case as a release gate.

2.8 Change control and regression selection

Changes in MediQueue may have effects beyond the edited screen. A concession-rule change affects the fee service, invoice display, payment amount, refunds, reports and possibly audit records. A database index change may affect response time without changing functional output. A test process therefore needs impact analysis:

identify changed requirements, code dependencies, data contracts and previously observed defect patterns before selecting regression tests.

Regression selection should combine traceability and risk. A test linked directly to the changed fee function is mandatory, but tests for payment retries and invoice persistence may also be needed because they consume the computed amount. A small, targeted suite provides fast feedback; a broader nightly or release suite checks for unintended effects elsewhere. The selection decision and any deferred cases should be recorded.

Automated dependency analysis can help but cannot replace engineering judgement. Static references may not reveal business coupling, asynchronous events or shared database effects. The team should ask what could be affected if the change is wrong, what evidence exists about prior failures and what is the cost of missing the regression.

Evidence table

Change	Direct tests	Indirect regression	Reason
Age concession formula	Age decision table	Invoice, payment amount, refund	Calculated amount propagates downstream
Cancellation policy	2-hour boundary cases	Appointment state, fee ledger, notification	Cancellation has multiple side effects
Payment callback	Callback handler tests	Duplicate callback and refresh retry	At-least-once delivery is common
Database schema	Migration and constraint tests	Booking concurrency, report queries	Persistence semantics may change
Authentication role	Permission matrix	Pharmacy, patient records, admin screens	Shared access controls affect modules
Report query	Correctness and pagination	800-user load scenario	Query plan can change performance

Interpretation and control implication: The table is a planning aid. Each expected result must be checked against the approved requirement and verified in the intended build. Where a requirement remains ambiguous, record the decision and owner before treating the case as a release gate.

2.9 Process failure modes and corrective controls

A process can be documented and still fail in practice. Common failure modes include test data that does not represent real states, unstable environments, flaky automation, unreviewed requirement changes, defects without owners and pressure to waive exit criteria. The response should address the cause rather than simply add more test cases. If test failures are unreliable because data is shared, data isolation and deterministic setup are the corrective actions.

Process retrospectives should examine both escapes and near misses. A defect found before release can reveal a control gap even if no patient was affected. For instance, a test that detects duplicate billing only after a manual refresh demonstrates that the retry invariant was not considered early enough. The retrospective should record the missing control, assign an owner and verify implementation in the next cycle.

Continuous improvement needs a small number of measurable actions. Examples include reducing flaky-test rate, increasing automated coverage of approved boundary rules, shortening mean time to triage and ensuring

every critical defect has a root-cause category. Actions should be revisited at the next release review to establish whether the change had the intended effect.

Evidence table

Observed symptom	Likely process cause	Corrective action	Effectiveness check
Same defect recurs	Root cause not addressed	Require cause and prevention action at closure	Track repeat category
Many tests marked blocked	Environment or data instability	Stabilize test setup and ownership	Blocked-test trend
Critical defect found late	Missing risk review or coverage	Add requirement-linked test earlier	Review next-cycle detection stage
Flaky automated suite	Shared state or timing assumptions	Isolate data and replace fixed sleeps	Flake rate over repeated runs
Unclear release decision	No objective exit criteria	Adopt documented go/no-go checklist	Audit release record
Late scope additions	Change control bypassed	Run impact and regression assessment	Trace changed scope to test evidence

Interpretation and control implication: The table is a planning aid. Each expected result must be checked against the approved requirement and verified in the intended build. Where a requirement remains ambiguous, record the decision and owner before treating the case as a release gate.

TASK 3 – TESTING AS A PROCESS AND BASIC DEFINITIONS: TEST ORACLE STRATEGIES

Assignment weight and evidence expectations are taken from the supplied Software Testing Assignment 1 brief. The analysis below preserves the MediQueue scenario and its stated business rules. Any added experimental counts are identified as illustrative.

3.1 Why the oracle matters

A test oracle determines the expected result against which actual behaviour is compared. Without a reliable oracle, a test can execute successfully yet provide no trustworthy verdict. MediQueue's D8 demonstrates this directly: TC-52 expects ₹450 for a 12-year-old, but the rule says 20% off ₹500, which is ₹400. Faulty code applying 10% concession passes the incorrect test.

The three strategies are mental calculation, a specification-table oracle and an independent reference implementation. They should be applied to the same function and the same inputs so that differences reflect oracle quality rather than test selection.

Evidence table

Age	Rule band	Discount	Expected fee	Fault exposed
0	Below 5	100%	₹0	Free-band logic
4	Below 5	100%	₹0	Boundary before age 5
5	5–17	20%	₹400	Lower child-band boundary
12	5–17	20%	₹400	D8 wrong oracle would accept ₹450

17	5–17	20%	₹400	Upper child-band boundary

Interpretation and control implication

The expected result is part of the test artefact. Incorrect expected values can make a defective implementation appear correct, so oracle review is a control in its own right.

3.2 Strategy A – tester's mental calculation

The tester computes expected fees while executing tests. This can be quick for a small number of examples and can help exploratory testing, but it is hard to audit and vulnerable to arithmetic slips, inconsistent interpretations and memory errors. Expected values are not always recorded, making regression comparison difficult.

For example, if a tester remembers that school-age patients receive "some concession" and assumes 10%, a faulty output of ₹450 may be incorrectly accepted. Mental calculation should therefore not be the sole oracle for a billing rule.

Evidence table

Age	Rule band	Discount	Expected fee	Fault exposed
0	Below 5	100%	₹0	Free-band logic
4	Below 5	100%	₹0	Boundary before age 5
5	5–17	20%	₹400	Lower child-band boundary
12	5–17	20%	₹400	D8 wrong oracle would accept ₹450
17	5–17	20%	₹400	Upper child-band boundary
18	18–59	0%	₹500	Adult boundary
59	18–59	0%	₹500	Upper no-discount boundary
60	60+	40%	₹300	D1: ≥ 60 comparator
61	60+	40%	₹300	Senior band
80	60+	40%	₹300	Older patient regression

3.3 Strategy B – specification decision table

A decision table makes the fee rule explicit and auditable. Age bands are mutually exclusive and collectively cover valid ages: below 5 → free; 5–17 → 20% off; 18–59 → full fee; 60 and above → 40% off. For a ₹500 base, expected results are ₹0, ₹400, ₹500 and ₹300. The half-up rounding rule is also stated even though these four outputs are whole numbers.

The table is reviewed with the business owner and reused in unit, integration and regression tests. It is the recommended primary oracle because it is traceable to the requirement and can be independently inspected. The requirement owner must resolve invalid age values and non-integer ages if the system permits them.

Evidence table

Age	Rule band	Discount	Expected fee	Fault exposed
0	Below 5	100%	₹0	Free-band logic
4	Below 5	100%	₹0	Boundary before age 5
5	5–17	20%	₹400	Lower child-band boundary
12	5–17	20%	₹400	D8 wrong oracle would accept ₹450
17	5–17	20%	₹400	Upper child-band boundary

3.4 Strategy C – independent reference implementation

A second implementation can calculate expected outputs and compare them with the production implementation (back-to-back testing). It is useful when the calculation is complex, but two implementations can share the same misunderstanding. A reference implementation must be simple, independently reviewed and derived from the approved specification rather than copied from the production code.

For this fee function, a short decision-table-based reference function is adequate. It should not call the production function internally or reuse the same faulty condition, because that would make the comparison circular.

Evidence table

Age	Rule band	Discount	Expected fee	Fault exposed
0	Below 5	100%	₹0	Free-band logic
4	Below 5	100%	₹0	Boundary before age 5
5	5–17	20%	₹400	Lower child-band boundary
12	5–17	20%	₹400	D8 wrong oracle would accept ₹450
17	5–17	20%	₹400	Upper child-band boundary
18	18–59	0%	₹500	Adult boundary
59	18–59	0%	₹500	Upper no-discount boundary
60	60+	40%	₹300	D1: ≥ 60 comparator
61	60+	40%	₹300	Senior band
80	60+	40%	₹300	Older patient regression

Interpretation and control implication

The expected result is part of the test artefact. Incorrect expected values can make a defective implementation appear correct, so oracle review is a control in its own right.

3.5 Ten-case test set and seeded faults

The illustrative test set includes ages 0, 4, 5, 12, 17, 18, 59, 60, 61 and 80 with a ₹500 base. Expected fees are respectively ₹0, ₹0, ₹400, ₹400, ₹400, ₹500, ₹500, ₹300, ₹300 and ₹300. Two seeded faults are: F1, use age > 60 instead of age ≥ 60; F2, use truncation instead of half-up rounding in a generalized fee calculation.

F1 is exposed by age 60. F2 requires a test value whose discounted result has a fractional component ending in .5; the four supplied age bands at a ₹500 base do not expose it because their expected fees are whole numbers. A good report states that a seeded fault is not detected by a test set when the input cannot activate it.

Evidence table

Oracle strategy	Strength	Weakness	Decision
Mental calculation	Fast for few cases	Unrecorded, error-prone	Supplement only
Specification table	Traceable and auditable	Requires clear approved rules	Primary oracle
Independent reference	Cross-checks implementation	Can share wrong assumptions	Secondary oracle

3.6 Error, fault and failure; test artefacts

An error is a human action, such as misunderstanding the 20% concession. A fault is the resulting incorrect rule or code, such as applying 10%. A failure is the observable result, such as displaying ₹450 for a 12-year-old. The terms describe different stages in a causal chain and should not be used interchangeably.

A test is one execution with input and expected result; a test set is a selected collection of tests; a test suite is the maintained organized collection run for a purpose; a test harness is the code and environment that executes and records tests; and a test oracle supplies expected results. For audit, the specification-table oracle is the primary choice, with independent implementation as a cross-check.

Evidence table

Oracle strategy	Strength	Weakness	Decision
Mental calculation	Fast for few cases	Unrecorded, error-prone	Supplement only
Specification table	Traceable and auditable	Requires clear approved rules	Primary oracle
Independent reference	Cross-checks implementation	Can share wrong assumptions	Secondary oracle

3.7 Test oracle design for the fee rule

A test oracle is the mechanism used to decide whether observed behaviour is correct. For the consultation fee, the oracle should be independent of the production implementation. If the test simply calls the same function

that the application uses to calculate the expected amount, a defect can be repeated in both actual and expected values and the test may pass incorrectly.

The business rule can be expressed as a piecewise function: age below five yields ₹0; ages five through seventeen yield 80% of the ₹500 base fee, or ₹400; ages eighteen through fifty-nine yield ₹500; and age sixty or above yields 60% of the base fee, or ₹300. The boundary ages must be tested explicitly. If age is based on date of birth, the system must also define which date is used to calculate age.

Independent oracles may include a manually reviewed decision table, a second implementation written by a different reviewer, a set of approved examples, or a property-based invariant. Multiple oracle methods improve confidence when they are genuinely independent. They should not be treated as independent if they were all copied from the same ambiguous statement without review.

Evidence table

Age	Expected fee	Why it matters	Test type
4	₹0	Last age below free threshold	Boundary
5	₹400	First age in 20% concession band	Boundary
17	₹400	Last age in concession band	Boundary
18	₹500	First full-fee age	Boundary
59	₹500	Last full-fee age	Boundary
60	₹300	First senior concession age	Boundary
61	₹300	Confirms senior band persists	Equivalence class
Negative age	Validation error	Invalid input must not create a bill	Negative

Interpretation and control implication: The table is a planning aid. Each expected result must be checked against the approved requirement and verified in the intended build. Where a requirement remains ambiguous, record the decision and owner before treating the case as a release gate.

3.8 Metamorphic and invariant-based oracles

Some behaviours are difficult to validate with a single expected output. Metamorphic testing checks relationships between outputs when inputs are transformed. For example, within the 5–17 age band, changing age from 8 to 12 should not change the fee if age is the only relevant input. Similarly, changing the display name should not alter the amount charged. These relationships can reveal hidden dependencies that example-based tests miss.

Invariant-based testing checks a property that must always hold. A payment invariant is that retrying a completed payment with the same idempotency key must not create a second charge. An appointment invariant is that one doctor cannot have two confirmed bookings for the same slot. A dispensing invariant is that a prescription that fails the approved validity rule cannot be dispensed.

These invariants are particularly useful for stateful workflows because they describe what must remain true across many sequences. A model-based test can generate actions such as book, cancel, refresh, retry and

reschedule, then check the invariant after every step. This can find interactions that a short fixed script does not cover.

Evidence table

Invariant	Generated or transformed action	Oracle	Failure evidence
Fee consistency	Change patient name only	Fee remains unchanged	Actual amount differs
Idempotent payment	Repeat same payment request	One successful transaction	Duplicate transaction IDs
Unique slot	Concurrent booking attempts	At most one confirmation	Two active bookings for slot
Prescription validity	Move date beyond allowed window	Dispensing blocked	Dispense record created
Report identity	Request report for another patient	Only authorized patient's data returned	Patient mismatch or data leak
Audit completeness	Change fee through authorized override	Actor, reason and time recorded	Missing audit event

Interpretation and control implication: The table is a planning aid. Each expected result must be checked against the approved requirement and verified in the intended build. Where a requirement remains ambiguous, record the decision and owner before treating the case as a release gate.

3.9 Oracle independence and review

Oracle independence is a governance issue as much as a coding issue. The requirement author should confirm business meaning, the tester should derive expected results, and a second reviewer should check high-risk calculations. For clinical workflows, the appropriate clinical or pharmacy owner must approve domain-specific interpretations. The team should document assumptions rather than silently choosing a rule where the brief is ambiguous.

An oracle review should ask whether expected values are calculated independently, whether all boundary points are present, whether invalid values have defined behaviour, and whether the assertion checks persisted state as well as displayed text. For a payment test, seeing ‘Success’ in the browser is insufficient if the database contains two transactions.

When an oracle cannot be defined because a requirement is unclear, the correct outcome is a clarification request. A test should not encode a guess and then use the resulting pass as evidence of correctness. Unresolved requirements should be visible as risks and should block affected release gates when the ambiguity can cause financial or patient harm.

Evidence table

Oracle source	Strength	Limitation	Recommended use
Approved rule table	Readable and reviewable	Can still omit scenarios	Primary reference for boundaries
Independent calculation	Reduces copied implementation error	May share misunderstood assumptions	Second-person review
Database assertion	Verifies persisted outcome	May miss user-visible defect	Pair with UI/API assertion
Metamorphic relation	Finds inconsistent behaviour	Does not define every exact value	Complement examples
State invariant	Strong for concurrency and retries	Requires reliable state observation	Critical workflow testing

Clinical/business sign-off	Confirms domain interpretation	Can be delayed or subjective	Formal approval for policy ambiguity

Interpretation and control implication: The table is a planning aid. Each expected result must be checked against the approved requirement and verified in the intended build. Where a requirement remains ambiguous, record the decision and owner before treating the case as a release gate.

TASK 4 – SOFTWARE TESTING PRINCIPLES: TEST PRIORITISATION STRATEGIES

Assignment weight and evidence expectations are taken from the supplied Software Testing Assignment 1 brief. The analysis below preserves the MediQueue scenario and its stated business rules. Any added experimental counts are identified as illustrative.

4.1 Input-space size and why exhaustive testing is impractical

The Appointment module input-space factors are 250 doctors × 20 slots × 3 appointment types × 3 patient categories × 2 payment modes × 2 active-appointment states. The product is 250 × 20 = 5,000; ×3 = 15,000; ×3 = 45,000; ×2 = 90,000; ×2 = 180,000 combinations.

At one test per second, 180,000 seconds are 50 hours, or 2 days 2 hours of uninterrupted execution. This ignores setup, data creation, cleanup, concurrent users, retries and multiple states. The figure only counts the stated dimensions; realistic systems contain further variables such as time zones, network retries and rescheduling.

Evidence table

Factor	Values	Product so far
Doctors	250	250
Slots per session	20	250 × 20 = 5,000
Appointment types	3	5,000 × 3 = 15,000
Patient categories	3	15,000 × 3 = 45,000
Payment modes	2	45,000 × 2 = 90,000

Interpretation and control implication

Prioritization should be justified by risk and historical evidence. The full 164-record log is needed for an actual Pareto chart; the ten named defects alone do not establish a module-level distribution.

4.2 Strategy A – exhaustive testing

Exhaustive testing attempts every combination and offers strong combinatorial coverage only if the dimensions and values are complete and the tests are valid. It is impractical within a 100-hour total testing budget because the raw run time alone consumes 50 hours for a single pass, before test design and diagnosis.

Exhaustive testing also cannot guarantee that unmodelled timing races or ambiguous requirements are covered. It remains useful for small finite state spaces or as exhaustive unit testing of a tightly bounded function.

Evidence table

Factor	Values	Product so far
Doctors	250	250
Slots per session	20	250 × 20 = 5,000
Appointment types	3	5,000 × 3 = 15,000
Patient categories	3	15,000 × 3 = 45,000
Payment modes	2	45,000 × 2 = 90,000
Active-appointment states	2	90,000 × 2 = 180,000

4.3 Strategy B – experience-driven selection

An experienced tester can use prior defects and clinical workflow knowledge to select cases rapidly. However, memory is not a documented coverage strategy and can overfocus on familiar defects. Experience-driven testing may overlook a new failure mode, such as two simultaneous confirmations for the same slot or a stale SMS after rescheduling.

Experience is valuable as an input to selection, but test rationale and expected outcomes should be recorded. A second tester should review the selection to reduce individual blind spots.

Evidence table

Factor	Values	Product so far
Doctors	250	250
Slots per session	20	250 × 20 = 5,000
Appointment types	3	5,000 × 3 = 15,000
Patient categories	3	15,000 × 3 = 45,000
Payment modes	2	45,000 × 2 = 90,000

4.4 Strategy C – risk-based and principle-guided selection

Risk-based selection prioritizes patient safety, financial integrity, high-use workflows, defect history and business-rule boundaries. For MediQueue, high priority includes age 60, age 5, cancellation at two hours, duplicate payment after refresh, concurrent booking, prescription day 7/day 8 and the 800-user performance target.

The pesticide paradox means repeatedly running the same tests eventually finds fewer new defects; the suite must be refreshed as code, data and failure history change. The absence-of-errors fallacy means passing all selected tests does not prove the system meets every user need.

Evidence table

Factor	Values	Product so far
Doctors	250	250
Slots per session	20	250 × 20 = 5,000
Appointment types	3	5,000 × 3 = 15,000
Patient categories	3	15,000 × 3 = 45,000
Payment modes	2	45,000 × 2 = 90,000
Active-appointment states	2	90,000 × 2 = 180,000

Interpretation and control implication

Prioritization should be justified by risk and historical evidence. The full 164-record log is needed for an actual Pareto chart; the ten named defects alone do not establish a module-level distribution.

4.5 Proposed 25-test allocation

Within the assignment's 25-case constraint for Appointment, allocate tests to critical boundaries and failure modes rather than random screens. A balanced set includes six age/fee boundary cases, four cancellation boundaries, four active-appointment limits, five slot concurrency/idempotency cases, three payment retry cases and three reminder/rescheduling cases. Some tests cover more than one risk, but traceability should be explicit.

An example allocation is shown below. The exact list should be adjusted after risk review and recent defect clustering; it is not intended to replace broader integration and performance testing elsewhere in the release.

Evidence table

Test ID	Risk	Input/condition	Expected result	Priority
T01	Age boundary	Age 60	₹300 fee	P0
T02	Age boundary	Age 5	₹400 fee	P0
T03	Cancellation	2 h 1 min before slot	No late fee	P1
T04	Cancellation	Exactly 2 h before slot	No late fee	P0
T05	Cancellation	1 h 59 min before slot	₹100 fee	P0

4.6 Defect clustering and normalization

The brief provides module sizes but not the complete 164-record defect log by module. Therefore a genuine defect percentage or 80/20 concentration cannot be calculated from the supplied evidence alone. A defensible report should not invent a measured Pareto result. The correct method is to count defects by module, calculate percentage = module defects ÷ 164 × 100, then normalize density = module defects ÷ module KLOC.

An illustrative example can show the calculation method, but it must be labelled synthetic. For instance, if Billing had 40 of 164 defects, its share would be 24.39% and density would be 40/12 = 3.33 defects/KLOC. This is not a claim that Billing actually had 40 defects.

Evidence table

Test ID	Risk	Input/condition	Expected result	Priority
T01	Age boundary	Age 60	₹300 fee	P0
T02	Age boundary	Age 5	₹400 fee	P0
T03	Cancellation	2 h 1 min before slot	No late fee	P1
T04	Cancellation	Exactly 2 h before slot	No late fee	P0
T05	Cancellation	1 h 59 min before slot	₹100 fee	P0
T06	Capacity	Already 2 active bookings; attempt third	Reject booking	P0
T07	Concurrency	Two users confirm same doctor/slot simultaneously	Only one succeeds	P0
T08	Payment retry	Refresh confirmation after payment	One bill only	P0
T09	Reminder	Reschedule appointment	Old reminder cancelled/replaced	P1
T10	Payment	Network timeout then retry	Idempotent transaction	P0

4.7 Decision and refresh rules

Choose C because the risk is uneven and a small number of carefully selected cases can target the most damaging failures. Preserve a few experience-based exploratory tests, but require each selected test to identify its risk, rule, input, expected output and evidence.

Refresh the set after every escaped defect, material requirement change and significant production incident. A module with zero open defects is not automatically safe to release; safety depends on test adequacy, unresolved risk, environment, coverage and the severity of remaining uncertainty.

Evidence table

Test ID	Risk	Input/condition	Expected result	Priority
T01	Age boundary	Age 60	₹300 fee	P0
T02	Age boundary	Age 5	₹400 fee	P0
T03	Cancellation	2 h 1 min before slot	No late fee	P1
T04	Cancellation	Exactly 2 h before slot	No late fee	P0
T05	Cancellation	1 h 59 min before slot	₹100 fee	P0

Interpretation and control implication

Prioritization should be justified by risk and historical evidence. The full 164-record log is needed for an actual Pareto chart; the ten named defects alone do not establish a module-level distribution.

4.7 Risk-based prioritisation in practice

Risk-based prioritisation assigns testing effort according to the probability and consequence of failure. In a hospital workflow, financial errors and patient-safety risks deserve more attention than minor cosmetic inconsistencies. A simple risk score can multiply likelihood by impact, but the score should support discussion rather than create false mathematical certainty. A low-frequency duplicate dispense event may still require a mandatory test because its impact is high.

Prioritisation should be revisited when evidence changes. If a defect has appeared repeatedly in appointment concurrency, the likelihood estimate should rise. If a module is untouched and has strong stable regression evidence, the team may allocate less exploratory effort there, while still retaining essential regression checks. Risk-based testing is dynamic, not a one-time ranking made at project start.

Dependencies matter. A low-level fee calculation function may be small, but it feeds billing, payment, refund and reporting. Its priority should reflect downstream exposure. Similarly, a login change may affect every protected module. The test plan should consider blast radius, not just the size of the code change.

Evidence table

Risk item	Likelihood (1–5)	Impact (1–5)	Illustrative score	Priority response
Duplicate payment on retry	3	5	15	Automated idempotency and callback tests
Double-booked slot	3	5	15	Synchronized concurrency test
Wrong age concession	4	4	16	Decision table and boundary suite
Expired prescription dispensed	2	5	10	Mandatory negative dispensing test
Report response slower than target	3	3	9	Peak-load test and query monitoring
Button alignment issue	2	1	2	Usability review after critical gates
Incorrect access to patient record	2	5	10	Authorization matrix and security tests

Interpretation and control implication: The table is a planning aid. Each expected result must be checked against the approved requirement and verified in the intended build. Where a requirement remains ambiguous, record the decision and owner before treating the case as a release gate.

4.8 Test selection under a 100-hour budget

The supplied budget is 100 person-hours, so the plan must balance breadth, depth and execution time. A useful allocation reserves time for planning and requirement clarification, automated critical-rule tests, integration and system testing, targeted concurrency and performance work, defect retesting, regression and release reporting. The allocation should be treated as a planning baseline and adjusted when actual defect levels or environment delays become known.

Cutting test effort should not mean cutting every activity by the same percentage. If time is reduced, preserve tests that protect critical invariants and remove low-value repetition first. For example, a single carefully designed test that proves payment idempotency under a timeout may be more valuable than dozens of similar happy-path checks. The trade-off and residual risk must be documented.

Reserve contingency for diagnosis. A test budget that spends all 100 hours on planned execution leaves no time to investigate failures, repair test data or confirm fixes. A failed critical test often creates additional work, but that work is the reason testing exists; it should not be treated as an unexpected nuisance.

Evidence table

Activity	Illustrative hours	Deliverable	Priority if budget is cut
Requirement and risk review	10	Approved test basis and risk list	Protect
Test design and data setup	12	Traceable test cases and fixtures	Protect for critical rules
Unit and API automation	18	Repeatable business-rule suite	Protect
Integration/system tests	22	End-to-end workflow evidence	Protect
Concurrency and payment fault tests	10	Invariant evidence	Protect
Performance tests	8	Workload and response report	Targeted scope; retain release gate
Defect retest and regression	12	Fix verification and regression results	Protect
Reporting and release review	8	Evidence pack and residual-risk decision	Protect

Interpretation and control implication: The table is a planning aid. Each expected result must be checked against the approved requirement and verified in the intended build. Where a requirement remains ambiguous, record the decision and owner before treating the case as a release gate.

4.9 Prioritisation limits and fairness

Priority labels can become misleading if every team marks its own issue as critical. Define consistent severity and priority separately: severity describes the consequence of the defect, while priority describes the urgency of action. A severe defect may have a narrow workaround, but the workaround must be validated and approved rather than assumed safe. A cosmetic defect can be urgent if it prevents a critical control from being used.

Test priority should also consider test value and cost. Expensive tests may be scheduled later if cheaper tests can quickly detect the same class of problem, but expensive high-risk tests must not be omitted simply because they take longer. A layered suite is effective: quick deterministic checks on every change, broader integration checks on merge, and performance/concurrency suites at scheduled gates.

Reviewers should challenge risk scores that are based on intuition alone. Use defect history, change size, architectural coupling, exposure, transaction volume and consequences. Where data is unavailable, state that the score is a qualitative judgement and identify what evidence would improve it.

Evidence table

Priority level	Typical scope	Example	Gate behaviour
P0 – release blocker	Safety, privacy, duplicate financial transaction	Two confirmed bookings for one slot	Must pass or release held
P1 – high	Core workflow correctness	Wrong concession at age 60	Fix or formal high-level decision
P2 – medium	Degraded but recoverable operation	Report filter fails for one valid option	Plan fix based on impact
P3 – low	Cosmetic or minor convenience	Spacing inconsistent on non-critical page	May be scheduled after release
Test suite fast	Seconds/minutes	Fee function and validation	Every relevant commit
Test suite extended	Minutes/hours	Load and concurrency	Integration/release gate

Interpretation and control implication: The table is a planning aid. Each expected result must be checked against the approved requirement and verified in the intended build. Where a requirement remains ambiguous, record the decision and owner before treating the case as a release gate.

TASK 5 – TESTER'S ROLE: COMPARE MODELS OF TEST RESPONSIBILITY

Assignment weight and evidence expectations are taken from the supplied Software Testing Assignment 1 brief. The analysis below preserves the MediQueue scenario and its stated business rules. Any added experimental counts are identified as illustrative.

5.1 Responsibility options

The Pharmacy module handles a prescription validity rule, quantity constraints, expiry checks and duplicate-dispense prevention. These controls have clinical implications, so the testing arrangement must combine technical competence with domain knowledge. The three options are peer-developer testing, an independent test team and a tester working with a pharmacist.

No single role is sufficient for every activity. Developers are best placed to test local logic and diagnose code; independent testers challenge assumptions and test across boundaries; a pharmacist validates clinical meaning and acceptable workflow outcomes.

Evidence table

Arrangement	Faults found (illustrative)	False reports	Effort	Interpretation
Peer developer	3/4	1	5 h	Technical strength, domain gap
Independent testers	3/4	0	7 h	Good independence, domain assumptions remain
Tester + pharmacist	4/4	0	8 h	Best combined technical and clinical coverage

Interpretation and control implication

Technical independence and clinical expertise should be combined. Clear defect reports reduce clarification time and make retest decisions reproducible.

5.2 A – peer-developer testing

A developer from another module may understand APIs, data structures, SQL and concurrency. This arrangement is relatively fast and can expose implementation faults, but the developer may not understand dispensing practice or may accept behaviour that is technically consistent with code but clinically inappropriate.

Peer testing is suitable as an early check, especially for code review, unit tests and integration contracts. It should not be the sole sign-off for prescription validity, expired batches or duplicate dispensing.

Evidence table

Arrangement	Faults found (illustrative)	False reports	Effort	Interpretation
Peer developer	3/4	1	5 h	Technical strength, domain gap
Independent testers	3/4	0	7 h	Good independence, domain assumptions remain
Tester + pharmacist	4/4	0	8 h	Best combined technical and clinical coverage

5.3 B – independent test team

Independent testers design tests from requirements and are less attached to the implementation. They can challenge happy-path assumptions, build reproducible test evidence and report defects with expected versus actual behaviour. Independence, however, does not automatically supply pharmacy knowledge; ambiguous rules may still be interpreted incorrectly.

The team should maintain a traceable test matrix and ask the pharmacist to review clinically consequential expected results. Independence should mean a different perspective, not isolation from developers or clinical staff.

Evidence table

Arrangement	Faults found (illustrative)	False reports	Effort	Interpretation
Peer developer	3/4	1	5 h	Technical strength, domain gap
Independent testers	3/4	0	7 h	Good independence, domain assumptions remain
Tester + pharmacist	4/4	0	8 h	Best combined technical and clinical coverage

5.4 C – tester plus pharmacist

The tester and pharmacist jointly review prescription rules and agree acceptance examples before implementation. This is strongest for translating clinical intent into testable conditions, such as the exact seven-day boundary, expired-batch behaviour and whether a duplicate dispense is blocked transactionally.

The main trade-off is scheduling and communication effort. A structured session can reduce rework: prepare examples before the meeting, record decisions, assign requirement IDs, and obtain explicit approval for unresolved policy questions.

Evidence table

Arrangement	Faults found (illustrative)	False reports	Effort	Interpretation
Peer developer	3/4	1	5 h	Technical strength, domain gap
Independent testers	3/4	0	7 h	Good independence, domain assumptions remain
Tester + pharmacist	4/4	0	8 h	Best combined technical and clinical coverage

Interpretation and control implication

Technical independence and clinical expertise should be combined. Clear defect reports reduce clarification time and make retest decisions reproducible.

5.5 Four seeded faults and illustrative outcomes

The task requests four faults: prescription validity beyond seven days, maximum quantity violation, expired-batch dispensing and duplicate dispensing. For comparison, the report assumes a structured pass where each arrangement receives the same four injected faults and records findings, false reports and effort. These figures are illustrative, not observed test-run data.

An illustrative result is: peer developer finds 3/4 faults in 5 hours with one false report; independent testers find 3/4 in 7 hours with zero false reports; tester plus pharmacist finds 4/4 in 8 hours with zero false reports. The result demonstrates the intended comparison but must be replaced with actual recorded outcomes if submitted as experimental evidence.

Evidence table

Report field	Weak report	Strong report
Summary	Pharmacy broken	Prescription older than 7 days can be dispensed
Steps	Not provided	Open prescription → select Dispense → confirm
Expected	Not stated	System rejects expired validity window

Actual	Does not work	Dispense transaction succeeds
Environment	Not stated	Staging build, pharmacist role, synthetic patient

5.6 Defect reports and communication quality

A weak report such as “pharmacy not working” does not identify the prescription, date, steps, expected result or environment. The developer must ask follow-up questions and may fail to reproduce the issue. A strong report identifies a synthetic patient/prescription ID, the date and time, steps, expected and actual result, severity, build, browser and supporting logs.

Example strong report: “Given a prescription dated nine days before the test date, when an authorized pharmacist selects Dispense, the system completes dispensing. Expected: reject the prescription because it is outside the seven-day validity window. Actual: dispense succeeds. Environment: staging build X, test database, pharmacist role. Severity: high; priority: urgent.”

Evidence table

Report field	Weak report	Strong report
Summary	Pharmacy broken	Prescription older than 7 days can be dispensed
Steps	Not provided	Open prescription → select Dispense → confirm
Expected	Not stated	System rejects expired validity window
Actual	Does not work	Dispense transaction succeeds
Environment	Not stated	Staging build, pharmacist role, synthetic patient
Severity	Not stated	High; patient-safety control
Evidence	None	Timestamp, request ID, screenshot/log with no real patient data

5.7 Staffing, sign-off and recommendation

With three testers and five developers, a reasonable allocation is one tester focused on Billing/Appointment transactions, one on Pharmacy/Registration, and one on Lab Reports/Reminders plus cross-module regression and performance coordination. The QA lead manages risk, coverage and release evidence; developers own unit tests and fixes; the pharmacist approves clinical acceptance criteria.

Recommend C for patient-safety rules, supported by independent testing and developer checks. Release authority should be a controlled decision: QA lead confirms test evidence and defect status, product owner

confirms business acceptance, and the medical officer/pharmacist signs off clinical safety rules. No single tester should unilaterally waive a critical patient-safety defect.

Evidence table

Report field	Weak report	Strong report
Summary	Pharmacy broken	Prescription older than 7 days can be dispensed
Steps	Not provided	Open prescription → select Dispense → confirm
Expected	Not stated	System rejects expired validity window
Actual	Does not work	Dispense transaction succeeds
Environment	Not stated	Staging build, pharmacist role, synthetic patient

Interpretation and control implication

Technical independence and clinical expertise should be combined. Clear defect reports reduce clarification time and make retest decisions reproducible.

5.7 Independence and communication in the test team

Independence is not an all-or-nothing property. Developers can write unit tests, a separate QA engineer can review system behaviour, and a business representative can validate acceptance. The value comes from different perspectives and explicit responsibilities, not from moving all testing to a separate department. Developers understand implementation details; testers often challenge assumptions; domain owners confirm whether the behaviour is acceptable in practice.

Independent testing can fail if communication is poor. A defect report should include reproducible steps, actual and expected results, build version, test data, logs and business impact. The developer should be able to reproduce the issue without guessing. A short triage meeting can classify severity, identify ownership and agree what evidence will prove the fix.

Testers should not be rewarded for finding the largest number of defects or developers for having the fewest reported defects. Such incentives discourage collaboration. A healthier measure is whether the team identifies important problems early, resolves root causes, prevents recurrence and provides honest release evidence.

Evidence table

Role	Primary responsibility	Must collaborate with	Evidence
Developer	Unit tests and code-level diagnosis	QA, reviewer, operations	Commit, test result, fix note
QA engineer	Independent scenario design and execution	Developer, product owner	Test report and defect record
Business owner	Approve business rules and priorities	QA, clinical owner	Signed requirement decision

Clinical/pharmacy reviewer	Confirm domain safety rules	Business owner, QA	Approved validity/dispensing rule
DevOps/operations	Environment, monitoring and rollback	Developer, QA	Deployment and health evidence
Release manager	Go/no-go record and risk ownership	All accountable roles	Release decision record

Interpretation and control implication: The table is a planning aid. Each expected result must be checked against the approved requirement and verified in the intended build. Where a requirement remains ambiguous, record the decision and owner before treating the case as a release gate.

5.8 Test responsibility matrix

A responsibility matrix reduces the chance that a critical activity is assumed to belong to someone else. For example, QA may identify that a prescription-age rule is ambiguous, but QA should not invent the clinical policy. The domain owner approves the rule; development implements it; QA verifies it; and release management ensures the evidence is complete.

The matrix should be used during planning and revisited when team composition changes. It should name roles rather than relying on vague group ownership. For a small student or project team, one person may hold multiple roles, but the approval step should still be explicit and documented.

Accountability also applies to test data, environments and defect closure. A test that cannot be executed because the environment is down needs an owner and a recovery estimate. A defect cannot be closed merely because code was merged; the agreed verification evidence must exist.

Evidence table

Activity	Responsible	Accountable	Consulted / informed
Approve fee rules	Business analyst	Business owner	QA and developer
Approve prescription validity	Pharmacy/clinical owner	Clinical lead	QA and business owner
Design automated unit tests	Developer	Development lead	QA
Design risk-based system tests	QA	QA lead	Business and development
Provision test environment	Operations/DevOps	Environment owner	QA and development
Triage critical defect	QA + developer	Release/engineering lead	Business/clinical owner
Accept residual business risk	Authorized business owner	Release authority	QA, operations, clinical owner
Verify release candidate	QA	QA lead	All workstream owners

Interpretation and control implication: The table is a planning aid. Each expected result must be checked against the approved requirement and verified in the intended build. Where a requirement remains ambiguous, record the decision and owner before treating the case as a release gate.

5.9 Conflict resolution and escalation

Disagreements are expected when test evidence conflicts with delivery pressure. The process should make the decision transparent: describe the observed behaviour, connect it to an approved requirement, quantify the affected workflow, identify workarounds and state what remains unknown. The decision should be made by the role authorized to accept that type of risk, not by whichever person is most senior in the room.

A critical patient-safety or privacy risk should be escalated immediately. A low-severity cosmetic issue can follow normal triage. The same escalation rules should apply consistently across teams so that a deadline does not silently downgrade a defect. If a waiver is granted, record the scope, expiry or review date, containment and monitoring.

After the release, the team should review whether the decision was correct and whether the process needs improvement. A waiver that repeatedly appears for the same class of issue indicates a systemic weakness, not a series of unrelated exceptions.

Evidence table

Conflict	Required evidence	Decision owner	Record
QA reports duplicate charge; developer cannot reproduce	Request IDs, gateway logs, database rows	Engineering lead with payment owner	Defect and reproduction notes
Business asks to release with wrong fee	Approved rule, affected age groups, workaround	Business/release authority	Risk acceptance and expiry
Prescription rule unclear	Source requirement and clinical interpretation	Clinical/pharmacy owner	Approved policy decision
Load target missed narrowly	Workload profile, percentiles, error rate	Service owner and release authority	Performance exception if accepted
Environment instability blocks tests	Incident duration and blocked coverage	Environment owner	Blocker and recovery action

Interpretation and control implication: The table is a planning aid. Each expected result must be checked against the approved requirement and verified in the intended build. Where a requirement remains ambiguous, record the decision and owner before treating the case as a release gate.

TASK 6 – ORIGINS OF DEFECTS: COMPARE DEFECT ORIGIN ANALYSIS METHODS

Assignment weight and evidence expectations are taken from the supplied Software Testing Assignment 1 brief. The analysis below preserves the MediQueue scenario and its stated business rules. Any added experimental counts are identified as illustrative.

6.1 Purpose and methods

Defect origin analysis asks where a defect was introduced, how it escaped, and what prevention could stop recurrence. The three methods are origin-by-artefact tagging, fault-tree analysis and Pareto analysis by cause category. They answer different questions: tagging supports trend analysis, fault trees explain a specific failure path, and Pareto highlights high-frequency categories.

The ten supplied defects D1–D10 are used as the primary grounded sample. The full release log is not provided, so the report cannot truthfully claim a measured matrix for all 164 records. A synthetic log can demonstrate the method, but its results must be clearly labelled as modelled.

Evidence table

Defect	Primary origin	Detection gap	Suggested prevention
D1	Code logic	Age 60 boundary absent	Decision table + boundary tests
D2	Design/code	Retry/idempotency invariant missing	Idempotency key + unique transaction
D3	Code logic	2-hour boundary wrong	Boundary-value suite
D4	Design/code	Race/atomic reservation	Database invariant + concurrency test
D5	Input/data handling	Apostrophe encoding path	Parameterized SQL + input test

Interpretation and control implication

Origin tagging, causal analysis and frequency analysis are complementary. The hospital should not label every problem a coding mistake without evidence.

6.2 A – origin-by-artefact tagging

Each defect receives an origin artefact such as requirement, design, code, test case, configuration or data, plus a detection phase. D1 is primarily a code logic defect (wrong comparator) with a likely test-design gap because age 60 was not covered. D7 is primarily a requirement ambiguity with a notification-state handling gap. D8 is a test-oracle defect because TC-52 encodes the wrong expected fee and allows faulty code to pass.

The matrix helps distinguish injection from detection. A code fault discovered by system testing is not necessarily caused by system testing; it may have been introduced during coding and missed by earlier controls. This distinction is essential when choosing preventive actions.

Evidence table

Defect	Primary origin	Detection gap	Suggested prevention
D1	Code logic	Age 60 boundary absent	Decision table + boundary tests
D2	Design/code	Retry/idempotency invariant missing	Idempotency key + unique transaction
D3	Code logic	2-hour boundary wrong	Boundary-value suite
D4	Design/code	Race/atomic reservation	Database invariant + concurrency test
D5	Input/data handling	Apostrophe encoding path	Parameterized SQL + input test
D6	Performance/configuration	Peak workload not validated	Load test at 800 concurrent
D7	Requirement	Reschedule semantics unspecified	Approved state-transition requirement
D8	Test case/oracle	Expected fee incorrect	Independent oracle review
D9	Code logic	Days compared as months	Day 7/day 8 tests
D10	Data mapping	Wrong date field	Field mapping and PDF assertion

6.3 B – fault-tree analysis for duplicate billing (D2)

Top event: the same consultation is billed twice after the confirmation page is refreshed. Plausible contributing events include the browser resubmitting the POST request, the server accepting repeated requests, no idempotency key or unique transaction constraint, and a payment status that is not reconciled before a second bill is written.

The preventive chain includes server-side idempotency, unique transaction identifiers, database uniqueness constraints, safe redirect-after-POST behaviour, and tests that refresh, retry after timeout and submit concurrently. A user-interface-only button disable is insufficient because network retries can occur without a second deliberate click.

Evidence table

Defect	Primary origin	Detection gap	Suggested prevention
D1	Code logic	Age 60 boundary absent	Decision table + boundary tests
D2	Design/code	Retry/idempotency invariant missing	Idempotency key + unique transaction
D3	Code logic	2-hour boundary wrong	Boundary-value suite
D4	Design/code	Race/atomic reservation	Database invariant + concurrency test
D5	Input/data handling	Apostrophe encoding path	Parameterized SQL + input test

6.4 B – fault-tree analysis for double booking (D4)

Top event: two patients receive the same doctor slot. An OR combination of missing atomic reservation, a read-then-write race, absent database uniqueness, and lack of concurrency tests can produce the failure. Two requests can both observe the slot as free before either commits a reservation.

Controls include a database constraint on doctor/slot, transaction isolation or atomic reservation, conflict handling, and concurrency tests that submit simultaneous confirmations. The server and database must enforce the invariant; front-end checks alone cannot prevent the race.

Evidence table

Defect	Primary origin	Detection gap	Suggested prevention
D1	Code logic	Age 60 boundary absent	Decision table + boundary tests
D2	Design/code	Retry/idempotency invariant missing	Idempotency key + unique transaction
D3	Code logic	2-hour boundary wrong	Boundary-value suite
D4	Design/code	Race/atomic reservation	Database invariant + concurrency test
D5	Input/data handling	Apostrophe encoding path	Parameterized SQL + input test
D6	Performance/configuration	Peak workload not validated	Load test at 800 concurrent
D7	Requirement	Reschedule semantics unspecified	Approved state-transition requirement
D8	Test case/oracle	Expected fee incorrect	Independent oracle review

D9	Code logic	Days compared as months	Day 7/day 8 tests
D10	Data mapping	Wrong date field	Field mapping and PDF assertion

Interpretation and control implication

Origin tagging, causal analysis and frequency analysis are complementary. The hospital should not label every problem a coding mistake without evidence.

6.5 C – Pareto analysis and cause categories

Group causes into categories such as requirement ambiguity, boundary/logic errors, transaction/concurrency errors, input/data handling, performance, mapping/integration and test-oracle defects. Count the categories, sort descending and calculate cumulative percentage. The supplied D1–D10 sample illustrates categories but is too small and selected to establish a reliable 80/20 distribution for the full release.

Pareto is useful for deciding where to invest prevention effort after a complete defect log is categorized. It can hide causal chains and context; therefore it should be paired with detailed origin tagging and fault-tree analysis for critical escapes.

Evidence table

D2 event	Failure tree / causal contributor	Preventive control
Top event	Duplicate bill after refresh	Idempotent payment API
Branch 1	Browser repeats confirmation request	Post/redirect/get; safe retry
Branch 2	Server treats retry as new payment	Idempotency key and stored result
Branch 3	Database accepts duplicate invoice	Unique constraint on consultation/payment
Branch 4	No regression case covers refresh	Automated retry/refresh test

6.6 Illustrative classification matrix

The table below assigns a primary origin and a detection phase to the ten named defects. These are analytical classifications based on the symptom descriptions, not verified root-cause findings from source code inspection. The team should revise tags after a blameless root-cause review.

Requirement-origin defects require clarification and acceptance examples; code-origin defects require boundary or static checks; test-case-origin defects require oracle review; concurrency defects require atomicity design and race testing; performance defects require workload modelling and monitoring.

Evidence table

D2 event	Failure tree / causal contributor	Preventive control

Top event	Duplicate bill after refresh	Idempotent payment API
Branch 1	Browser repeats confirmation request	Post/redirect/get; safe retry
Branch 2	Server treats retry as new payment	Idempotency key and stored result
Branch 3	Database accepts duplicate invoice	Unique constraint on consultation/payment
Branch 4	No regression case covers refresh	Automated retry/refresh test

6.7 Containment and prevention

Containment effectiveness can be calculated as defects caught by a review activity divided by all defects eligible for that activity, but the denominator and detection-stage data must be available. The brief does not provide phase tags for all 164 records, so a measured requirement-review containment percentage cannot be asserted.

Recommended prevention actions are (1) a reviewed decision table and boundary-test template for all business rules; and (2) transaction invariants and concurrency regression tests for payment and slot reservation. A target reduction such as 30% fewer repeated boundary and transaction defects can be adopted as a goal, but it should be measured over subsequent releases rather than reported as an achieved result.

Evidence table

D2 event	Failure tree / causal contributor	Preventive control
Top event	Duplicate bill after refresh	Idempotent payment API
Branch 1	Browser repeats confirmation request	Post/redirect/get; safe retry
Branch 2	Server treats retry as new payment	Idempotency key and stored result
Branch 3	Database accepts duplicate invoice	Unique constraint on consultation/payment
Branch 4	No regression case covers refresh	Automated retry/refresh test

Interpretation and control implication

Origin tagging, causal analysis and frequency analysis are complementary. The hospital should not label every problem a coding mistake without evidence.

6.7 Root-cause analysis from symptom to prevention

Root-cause analysis should distinguish the visible failure from the condition that allowed it. A duplicate payment is a symptom. The underlying cause may be a non-idempotent API, an ambiguous transaction state, a missing database uniqueness constraint, a callback handler that processes repeated notifications, or a test plan that never exercised retries. More than one contributing cause may exist.

A useful investigation reconstructs the event sequence using timestamps, correlation IDs, gateway responses and database records. It then asks why each safeguard failed. ‘The developer made a mistake’ is not a sufficient

root cause because it does not lead to a durable control. A stronger finding might be that payment state transitions were not defined and the design allowed a second transaction after an uncertain response.

Corrective actions should be verifiable. If the root cause is a missing idempotency guarantee, actions may include an idempotency key, a unique transaction constraint, safe replay behaviour and a test that simulates timeout after gateway acceptance. The team should prove the regression test fails on the old build and passes on the corrected build whenever practical.

Evidence table

Analysis step	Question	Evidence	Output
Define failure	What exactly happened?	Observed UI and persisted records	Precise defect statement
Reconstruct timeline	What happened before and after?	Logs and correlation IDs	Event sequence
Identify contributing conditions	What made the failure possible?	Code, schema, config and test plan	Cause candidates
Validate root cause	Would removing it prevent recurrence?	Reproduction and fault isolation	Confirmed cause
Contain	How is immediate harm limited?	Monitoring and workaround checks	Temporary control
Correct	What must change?	Code/design/process proposal	Approved corrective action
Prevent recurrence	What test or review is added?	Regression evidence	Updated suite and checklist
Verify effectiveness	Did the same class of issue decline?	Subsequent releases	Follow-up review

Interpretation and control implication: The table is a planning aid. Each expected result must be checked against the approved requirement and verified in the intended build. Where a requirement remains ambiguous, record the decision and owner before treating the case as a release gate.

6.8 Fault-tree analysis for duplicate payment

Fault-tree analysis begins with a top event and decomposes the conditions that could produce it. For duplicate billing, the top event is ‘one patient consultation results in two successful charges’. One branch could be repeated client submission; another could be a duplicate gateway callback; a third could be concurrent requests. Each branch is then linked to safeguards such as idempotency keys, transaction uniqueness, callback deduplication and reconciliation.

The tree should not be interpreted as a complete probability model unless reliable event probabilities are available. Its primary value here is completeness: it prompts the team to examine multiple paths to the same harmful outcome. It also helps derive tests from each leaf condition rather than focusing only on the most obvious refresh scenario.

After a safeguard is introduced, the tree should be reviewed again. A client-side disabled button may reduce accidental double-clicks but does not protect against retries from another device, network middleware or repeated callbacks. Server-side idempotency and persistence constraints are therefore stronger controls than a visual prevention alone.

Evidence table

Fault path	Possible trigger	Preventive/detective control	Test derived
Client retry	Refresh after timeout	Idempotency key and stored outcome	Repeat same request
Concurrent requests	Two submissions arrive together	Atomic transaction and unique constraint	Synchronized double submit
Duplicate callback	Gateway retries notification	Callback event deduplication	Replay callback twice
Partial commit	Gateway succeeds but local write fails	Reconciliation and retry-safe state machine	Simulate local persistence failure
Wrong key usage	New key created for each retry	Stable key per business transaction	Retry after connection reset
Amount mismatch	Same key reused with different amount	Reject conflicting payload	Replay key with changed amount

Interpretation and control implication: The table is a planning aid. Each expected result must be checked against the approved requirement and verified in the intended build. Where a requirement remains ambiguous, record the decision and owner before treating the case as a release gate.

6.9 Pareto analysis and defect recurrence

Pareto analysis groups defects by cause, module or failure mode and ranks categories by count or impact. It helps determine where a limited prevention effort may yield the greatest benefit. If a small number of categories account for most high-severity defects, the team can prioritize targeted design reviews, automated tests or architectural changes in those areas.

Counts should not be interpreted without severity and exposure. Ten cosmetic defects may be less important than one privacy leak. A useful Pareto view can be built separately for total count, critical severity and escaped defects. The categories should be mutually understandable and consistently assigned, otherwise apparent trends may reflect classification changes rather than actual quality improvement.

After an intervention, compare later releases using the same definitions. If duplicate-payment defects decline but appointment race conditions rise, the team should not claim universal improvement. Record the intervention date, release size, changed scope and any known data limitations.

Evidence table

Category	Count field	Impact field	Possible action
Boundary/decision logic	Number of fee/date defects	Incorrect charge or eligibility	Decision-table review and boundary automation
Concurrency/state	Race conditions and duplicate transitions	Double booking or double charge	Atomic persistence and synchronized tests
Integration/contracts	API/schema mismatches	Broken patient journey	Contract tests and version compatibility
Validation/data quality	Invalid or inconsistent data	Wrong records and workflow blocks	Input validation and constraints
Performance	Latency/timeouts under load	Delayed care and retry pressure	Profiling and realistic workload tests
Access control	Authorization/privacy defects	Unauthorized disclosure or action	Role matrix and negative security tests

Interpretation and control implication: The table is a planning aid. Each expected result must be checked against the approved requirement and verified in the intended build. Where a requirement remains ambiguous, record the decision and owner before treating the case as a release gate.

TASK 7 – DEFECT CLASSES: COMPARE CLASSIFICATION SCHEMES

Assignment weight and evidence expectations are taken from the supplied Software Testing Assignment 1 brief. The analysis below preserves the MediQueue scenario and its stated business rules. Any added experimental counts are identified as illustrative.

7.1 Purpose and schemes

A common defect taxonomy allows the hospital to compare releases, route defects to appropriate owners and prioritize risk. The three schemes are hierarchical class, anomaly-attribute and patient-safety/risk. They are complementary rather than mutually exclusive: one identifies the defect's technical class, one stores several analysis dimensions, and one communicates impact.

Standard classification should be sufficiently simple that reporters use it consistently. If the scheme is too complex, fields will be left blank or assigned arbitrarily; if it is too broad, it will not support prevention or release decisions.

Evidence table

ID	Hierarchical class	Attribute/risk tags	Severity	Priority
D1	Coding – control logic	Financial; code; boundary	Medium–High	High
D2	Design/coding – transaction	Financial; duplicate transaction	High	Urgent
D3	Coding – boundary logic	Financial; cancellation	Medium	High
D4	Design/coding – concurrency	Availability/operations; race	High	Urgent
D5	Coding – input/interface	Usability; data handling	Medium	Medium–High

Interpretation and control implication

Severity and priority are distinct. Controlled attributes support trend analysis, while risk tags support patient-safety release decisions.

7.2 A – hierarchical class scheme

Classes include requirements/specification, design, coding (control/logic, data, algorithmic or interface), documentation and testing. D1 is coding/control logic because the age comparator is wrong. D7 is requirements/specification because rescheduling behaviour is not defined. D8 is a testing defect because the expected result in TC-52 is incorrect.

This scheme is easy to explain and supports origin analysis, but one defect may cross categories. D4 can be design or code depending on whether the missing atomic reservation is an architecture issue or an implementation error. The team should define primary class rules and permit secondary tags.

Evidence table

ID	Hierarchical class	Attribute/risk tags	Severity	Priority
D1	Coding – control logic	Financial; code; boundary	Medium–High	High
D2	Design/coding – transaction	Financial; duplicate transaction	High	Urgent
D3	Coding – boundary logic	Financial; cancellation	Medium	High
D4	Design/coding – concurrency	Availability/operations; race	High	Urgent
D5	Coding – input/interface	Usability; data handling	Medium	Medium–High
D6	Performance/configuration	Availability/performance	High at peak	Urgent
D7	Requirement/specification	Reminder; rescheduling ambiguity	Medium–High	High
D8	Testing – oracle defect	Financial; expected-value error	High risk of masking	High
D9	Coding – date logic	Patient safety; prescription validity	High	Urgent
D10	Coding/data mapping	Report correctness; date field	Medium–High	High

7.3 B – anomaly-attribute scheme

Each record stores type, cause, injection phase, detection phase, severity and resolution, with optional module, environment, requirement ID and recurrence flag. This scheme captures more information and is suitable for audit, trend analysis and defect prevention.

The cost is data-entry effort and the need for controlled definitions. Mandatory fields should be limited to those required for triage and traceability; additional root-cause fields can be completed during analysis rather than blocking initial reporting.

Evidence table

ID	Hierarchical class	Attribute/risk tags	Severity	Priority
D1	Coding – control logic	Financial; code; boundary	Medium–High	High
D2	Design/coding – transaction	Financial; duplicate transaction	High	Urgent
D3	Coding – boundary logic	Financial; cancellation	Medium	High
D4	Design/coding – concurrency	Availability/operations; race	High	Urgent
D5	Coding – input/interface	Usability; data handling	Medium	Medium–High

7.4 C – patient-safety and risk scheme

Risk categories include safety-critical, data privacy, financial, availability/performance, usability and cosmetic. D9 is safety-critical because an invalid prescription can be dispensed; D2 is financial and transaction integrity; D6 is availability/performance; D5 affects registration usability and data handling.

This scheme helps allocate test effort and decide release blockers, but it does not explain where a defect originated. It should complement the technical taxonomy rather than replace it.

Evidence table

ID	Hierarchical class	Attribute/risk tags	Severity	Priority
D1	Coding – control logic	Financial; code; boundary	Medium–High	High
D2	Design/coding – transaction	Financial; duplicate transaction	High	Urgent
D3	Coding – boundary logic	Financial; cancellation	Medium	High
D4	Design/coding – concurrency	Availability/operations; race	High	Urgent
D5	Coding – input/interface	Usability; data handling	Medium	Medium–High
D6	Performance/configuration	Availability/performance	High at peak	Urgent
D7	Requirement/specification	Reminder; rescheduling ambiguity	Medium–High	High
D8	Testing – oracle defect	Financial; expected-value error	High risk of masking	High
D9	Coding – date logic	Patient safety; prescription validity	High	Urgent
D10	Coding/data mapping	Report correctness; date field	Medium–High	High

Interpretation and control implication

Severity and priority are distinct. Controlled attributes support trend analysis, while risk tags support patient-safety release decisions.

7.5 D1–D10 comparison and severity

Severity describes impact if the defect occurs; priority describes urgency and order of work. A high-severity but low-priority case can occur when a catastrophic failure is extremely unlikely and a safe workaround exists, but patient-safety issues generally remain high priority until risk is understood. A low-severity high-priority defect can be a frequently encountered operational blocker with a simple urgent fix.

Illustrative triage: D2 high severity/high priority; D4 high severity/high priority; D9 high severity/high priority; D6 high severity/high priority during peak operations; D1 medium-to-high severity/high priority because it affects financial fairness; D10 medium severity depending on clinical reliance and whether the wrong date can mislead interpretation. Final ratings require hospital risk governance.

Evidence table

Agreement calculation	Formula	Example only
Exact-match agreement	matching labels ÷ 10 × 100	If 7 labels match, agreement = 70%
Disagreement review	List mismatched defects and reasons	D4 may be design or code; D7 may be requirement or integration
LLM role	Suggest labels; human confirms	Never allow unreviewed safety severity

7.6 Inter-rater agreement and LLM-assisted classification

The task asks two classmates to classify D1–D10 independently and calculate percentage agreement. No actual independent ratings were supplied, so an observed agreement percentage cannot be claimed. The formula is exact matches ÷ total classifications × 100. For 10 defects, each matching label contributes 10 percentage points.

An LLM may propose classes quickly, but automatic labels should be reviewed, especially for safety and privacy categories. Acceptable use is triage assistance with human confirmation, not final risk acceptance. The recommended standard is the anomaly-attribute scheme with a required patient-safety/risk tag and a controlled technical primary class.

Evidence table

Agreement calculation	Formula	Example only
Exact-match agreement	matching labels ÷ 10 × 100	If 7 labels match, agreement = 70%
Disagreement review	List mismatched defects and reasons	D4 may be design or code; D7 may be requirement or integration
LLM role	Suggest labels; human confirms	Never allow unreviewed safety severity

7.7 Severity, priority and status are different dimensions

Defect classification becomes more useful when it separates severity, priority, type and workflow status. Severity describes the consequence of the failure; priority describes how soon the team should act; type identifies the nature of the defect; status describes where it is in the lifecycle. Combining these dimensions into one label creates confusion. A defect can be high severity but temporarily deferred only after an authorized risk decision, while a medium-severity defect can be urgent because it blocks a release demonstration.

Classification should be assigned using evidence. For instance, a report that loads slowly in a synthetic test may be medium severity until the team shows that the response-time target is breached under the specified workload. A defect that exposes another patient's report is high severity even if the interface appears visually correct. The classification rationale should reference the affected workflow and impact.

Consistent classification enables reporting and trend analysis. If teams use ‘critical’ differently, dashboards cannot compare releases. A short taxonomy with examples, a triage checklist and periodic review of disputed records helps maintain consistency.

Evidence table

Dimension	Meaning	Example	Should it change during lifecycle?
Severity	Impact if defect occurs	Duplicate successful charge	May be refined with impact evidence
Priority	Urgency of resolution	Fix before release	Can change with deadline and workaround
Type	Failure nature	Logic, integration, data, performance	Usually stable after triage
Status	Workflow stage	New, assigned, fixed, retest, closed	Changes as work progresses
Origin	Where introduced	Requirement, design, code, configuration	Updated after root-cause analysis
Escape stage	Where first detected	System test or production	Stable after discovery

Interpretation and control implication: The table is a planning aid. Each expected result must be checked against the approved requirement and verified in the intended build. Where a requirement remains ambiguous, record the decision and owner before treating the case as a release gate.

7.8 Defect lifecycle and closure criteria

A defect lifecycle should define the meaning of each status. ‘New’ means the issue is recorded but not yet triaged; ‘Assigned’ identifies an owner; ‘In Progress’ indicates active investigation or correction; ‘Fixed’ means a change is available for verification; ‘Retest’ indicates QA is checking the agreed condition; and ‘Closed’ means acceptance criteria have been met. ‘Reopened’ is appropriate when the failure persists or returns.

Closure must require more than a developer's statement that code was changed. The record should contain the fix version, retest result, regression evidence where appropriate and any linked change request. If a defect cannot be reproduced, close it only under an approved policy with supporting evidence and a reason; otherwise retain it in a state that accurately reflects uncertainty.

Rejected, duplicate and deferred defects also need discipline. A duplicate should link to the primary record; a rejected issue should explain why it does not violate the requirement; and a deferred issue should have an owner, target release and accepted risk. These records remain useful for future analysis.

Evidence table

Status	Entry condition	Required information	Exit condition
New	Failure first reported	Steps, expected/actual, build, evidence	Triage decision
Triaged	Impact and validity assessed	Severity, priority, component	Owner and next action
Assigned	Owner accepted responsibility	Target fix version	Work begins
In progress	Investigation/fix underway	Cause and change reference	Fix ready
Fixed	Candidate correction available	Commit/build and developer notes	QA retest starts
Retest	Verification performed	Pass/fail evidence	Close or reopen

Closed	Acceptance criteria met	Final evidence and linked tests	No further action
Deferred	Authorized delay decision	Reason, owner, review date	Reassess at review date

Interpretation and control implication: The table is a planning aid. Each expected result must be checked against the approved requirement and verified in the intended build. Where a requirement remains ambiguous, record the decision and owner before treating the case as a release gate.

7.9 Classification quality audit

A periodic classification audit samples defect records and checks whether severity, priority, component, origin and escape stage are supported by evidence. The audit can identify systematic under-classification, missing root causes or teams that close defects without linking regression tests. The aim is to improve the data, not to punish individual reporters.

The audit should examine both closed and open defects. Open critical defects need clear containment and ownership. Closed critical defects need convincing evidence that the failure is resolved and that the regression control exists. Records with insufficient evidence should be returned for completion rather than included uncritically in metrics.

Taxonomy changes should be versioned. If a new ‘privacy’ category is added, historical data may need remapping or should be clearly separated from the new reporting period. Otherwise a trend chart can falsely suggest improvement simply because the labels changed.

Evidence table

Audit question	Good evidence	Warning sign	Action
Is severity justified?	Impact and affected users described	Severity selected without rationale	Re-triage with impact criteria
Is priority separate?	Urgency and release context documented	Priority copied from severity automatically	Review deadline/workaround
Is origin known?	Requirement/design/code evidence	All defects labelled coding error	Perform root-cause review
Is closure valid?	Retest output and build ID	Closed after merge only	Reopen verification task
Is recurrence linked?	Related defect IDs	Repeated issue treated as unrelated	Create common-cause action
Is escape stage accurate?	First detection evidence	Stage guessed retrospectively	Mark unknown if not supported

Interpretation and control implication: The table is a planning aid. Each expected result must be checked against the approved requirement and verified in the intended build. Where a requirement remains ambiguous, record the decision and owner before treating the case as a release gate.

TASK 8 – DEFECT REPOSITORY: COMPARE DEFECT TRACKING TOOLS

Assignment weight and evidence expectations are taken from the supplied Software Testing Assignment 1 brief. The analysis below preserves the MediQueue scenario and its stated business rules. Any added experimental counts are identified as illustrative.

8.1 Repository requirements

Defects are currently communicated through a phone messaging group and can be forgotten. The hospital needs a single repository with durable IDs, reproducible steps, severity, priority, ownership, status, timestamps, links to tests and an auditable history. The three options are a basic issue tracker with labels, Bugzilla and JIRA with a defined workflow.

The report cannot include genuine screenshots from the live tools because no project instances or screenshots were supplied. The tables below define the fields and workflow to configure; screenshots should be captured from the actual chosen tracker during implementation.

Evidence table

Field	Required?	Purpose / quality rule
ID and summary	Yes	Unique ID; concise observable symptom
Module and class	Yes	Controlled values, not free-form synonyms
Severity and priority	Yes	Impact separate from urgency
Reporter, assignee, status	Yes	Ownership and lifecycle accountability
Environment/build	Yes	Reproduction context

Interpretation and control implication

A repository is only as useful as the quality of its records. Mandatory fields, ownership, timestamps and verification history enable defensible release decisions.

8.2 A – issue tracker with labels

A simple tracker is quick to establish and can support labels, milestones and comments. It is appropriate for a small team or pilot. However, inconsistent labels and weak workflow controls can reduce data quality. It may be difficult to enforce mandatory environment, severity or verification fields.

Use this option if the team needs an immediate low-overhead repository, but create a template and label dictionary before opening issues. The tool should not become another informal chat channel.

Evidence table

Field	Required?	Purpose / quality rule
ID and summary	Yes	Unique ID; concise observable symptom
Module and class	Yes	Controlled values, not free-form synonyms
Severity and priority	Yes	Impact separate from urgency
Reporter, assignee, status	Yes	Ownership and lifecycle accountability
Environment/build	Yes	Reproduction context
Steps/preconditions	Yes	Minimal, ordered, repeatable

Expected vs actual	Yes	Objective evidence of failure
Evidence and linked test	Strongly required	Logs/screenshots with sensitive data masked
Root cause/resolution	At resolution	Prevention and trend analysis

8.3 B – Bugzilla

Bugzilla provides structured fields, statuses, search, saved queries and reporting. It can support a clear lifecycle from New to Assigned, Resolved, Verified and Closed, with Reopened or Invalid paths as appropriate. It is a reasonable open-source option for a cost-sensitive team.

The limitation is configuration and usability: the team must ensure that fields reflect the hospital's severity model and that test-case links are maintained. A report that only counts open defects without severity and module filters is not enough for release governance.

Evidence table

Field	Required?	Purpose / quality rule
ID and summary	Yes	Unique ID; concise observable symptom
Module and class	Yes	Controlled values, not free-form synonyms
Severity and priority	Yes	Impact separate from urgency
Reporter, assignee, status	Yes	Ownership and lifecycle accountability
Environment/build	Yes	Reproduction context

8.4 C – JIRA with defined workflow

JIRA can provide configurable workflows, dashboards, saved filters, notifications and links among defects, requirements and test cases. It can generate defects-by-module-and-severity reports when fields are normalized and users follow the process.

The recommendation is JIRA if the hospital already has access and can maintain the workflow; otherwise Bugzilla is a defensible open-source choice. Tool selection matters less than consistent data, clear ownership, mandatory fields and an enforced retest process.

Evidence table

Field	Required?	Purpose / quality rule
ID and summary	Yes	Unique ID; concise observable symptom
Module and class	Yes	Controlled values, not free-form synonyms
Severity and priority	Yes	Impact separate from urgency
Reporter, assignee, status	Yes	Ownership and lifecycle accountability

Environment/build	Yes	Reproduction context
Steps/preconditions	Yes	Minimal, ordered, repeatable
Expected vs actual	Yes	Objective evidence of failure
Evidence and linked test	Strongly required	Logs/screenshots with sensitive data masked
Root cause/resolution	At resolution	Prevention and trend analysis

Interpretation and control implication

A repository is only as useful as the quality of its records. Mandatory fields, ownership, timestamps and verification history enable defensible release decisions.

8.5 Required issue template and lifecycle

Every defect should include ID, concise summary, module, primary class, risk category, severity, priority, status, reporter, assignee, environment/build, preconditions, reproducible steps, expected result, actual result, evidence, linked requirement/test, timestamps and resolution. Sensitive patient data must be masked or replaced with synthetic identifiers.

Lifecycle: New → Triage/Open → Assigned → Fixed/Resolved → Retest → Verified → Closed. Reopened returns to Assigned; Rejected/Not a defect requires a reason. A fix is not verified merely because a developer says it is complete; the original reproduction and relevant regression suite must pass.

Evidence table

Metric	Formula	Data needed
Mean time to fix/verify	Average(closed timestamp − opened timestamp)	Real timestamp history
Reopen rate	Reopened ÷ resolved × 100	State-change audit history
Defects/KLOC	Module defects ÷ module KLOC	Defect counts by module
Critical open defects	Count of severity-critical not closed	Current tracker query

8.6 Timing metrics and normalization

Mean time to fix (MTTF as used in the assignment) should be computed from opened timestamp to verified/closed timestamp, with the chosen start/end definitions documented. Reopen rate = reopened defects ÷ defects resolved in the period × 100. Since no real event timestamps are provided, actual MTTF and reopen rate cannot be computed from the brief.

Raw defect counts should be normalized by module size: defects/KLOC = module defect count ÷ module KLOC. Billing's 12 KLOC may have more total defects simply because it is the largest module; a conclusion that it is the worst module requires density, severity, exposure and escape data—not count alone.

Evidence table

Metric	Formula	Data needed
Mean time to fix/verify	Average(closed timestamp − opened timestamp)	Real timestamp history
Reopen rate	Reopened ÷ resolved × 100	State-change audit history
Defects/KLOC	Module defects ÷ module KLOC	Defect counts by module
Critical open defects	Count of severity-critical not closed	Current tracker query

8.7 Recommendation and governance

Adopt JIRA with a defined workflow if licensing and administrative capacity permit; otherwise adopt Bugzilla using the same field schema and lifecycle. Require the QA lead to review high-risk open defects daily during release week and require a clinical owner for Pharmacy sign-off.

Minimum release dashboard: open defects by severity and module, escaped defects by cause, mean time to verify, reopen rate, critical test pass rate, risk acceptance log and requirement-to-test traceability. Screenshots in the final academic submission must be real screenshots from the selected tool, not invented mockups.

Evidence table

Metric	Formula	Data needed
Mean time to fix/verify	Average(closed timestamp − opened timestamp)	Real timestamp history
Reopen rate	Reopened ÷ resolved × 100	State-change audit history
Defects/KLOC	Module defects ÷ module KLOC	Defect counts by module
Critical open defects	Count of severity-critical not closed	Current tracker query

Interpretation and control implication

A repository is only as useful as the quality of its records. Mandatory fields, ownership, timestamps and verification history enable defensible release decisions.

8.7 Repository configuration and workflow design

A defect repository is only as useful as its fields, workflow and adoption. The team should define required fields before creating large numbers of records. At minimum, each issue needs a unique identifier, concise title,

environment/build, reproduction steps, expected and actual results, severity, priority, component, owner, status, evidence attachments and links to related requirements or tests.

Workflow states should reflect actual decisions rather than mirror every team preference. Too many statuses make reports hard to interpret; too few hide important transitions such as retest and reopen. Permissions should ensure that users can report defects easily while high-impact classification changes, risk acceptance and closure remain auditable.

Repository configuration should be tested like any other system. Create sample issues, transition them through the workflow, check required fields, test notifications, validate search filters and confirm that exported reports preserve key information. A repository migration should preserve IDs and links wherever practical.

Evidence table

Field	Purpose	Required?	Validation rule
Summary	Quickly identifies failure	Yes	One clear observable symptom
Steps to reproduce	Makes failure repeatable	Yes	Numbered, deterministic steps
Expected/actual	Defines the mismatch	Yes	Both explicitly stated
Build/environment	Enables reproduction	Yes	Version and environment required
Severity/priority	Supports triage	Yes	Selected from controlled values
Component	Routes ownership	Yes	Use maintained component list
Evidence	Supports diagnosis	When applicable	No secrets or unnecessary patient data
Requirement/test link	Provides traceability	For covered scope	Link to stable ID
Fix version	Tracks delivery	When fixed	Must match candidate build
Root cause	Supports prevention	For significant defects	Use reviewed taxonomy

Interpretation and control implication: The table is a planning aid. Each expected result must be checked against the approved requirement and verified in the intended build. Where a requirement remains ambiguous, record the decision and owner before treating the case as a release gate.

8.8 Dashboards, filters and actionable reporting

Dashboards should answer management questions rather than merely display counts. A useful release dashboard shows open defects by severity, age of unresolved critical issues, defects awaiting retest, reopen rate, escaped defects, test execution status and outstanding risk acceptances. Each number should link to the underlying records so the team can inspect context.

Filters should support daily triage. Examples include all P0/P1 defects without an owner, all fixed defects awaiting retest, all defects linked to the appointment module, and all issues reopened in the current release. Saved filters reduce manual reporting and make the process repeatable across team members.

Metrics should be interpreted carefully. A sudden increase in defects can indicate worse quality, better detection or a larger test scope. A decline can mean improvement or under-reporting. Pair counts with release size, test effort, severity, detection stage and changes in reporting practices.

Evidence table

Dashboard tile	Question answered	Useful filter	Decision supported
Open critical defects	What can block release?	Severity P0/P1 and status not closed	Hold, fix or formally escalate
Awaiting retest	What fixes need verification?	Status = Fixed/Retest	Allocate QA capacity
Aging issues	What is stalled?	Open longer than agreed threshold	Escalate ownership
Escaped defects	What reached users?	Detection stage = production	Root-cause prevention
Reopened issues	Which fixes may be weak?	Status = Reopened	Review correction and regression
Module trend	Where are defects concentrated?	Component grouped by release	Target prevention effort
Unlinked issues	What lacks traceability?	No requirement/test link	Complete evidence

Interpretation and control implication: The table is a planning aid. Each expected result must be checked against the approved requirement and verified in the intended build. Where a requirement remains ambiguous, record the decision and owner before treating the case as a release gate.

8.9 Repository governance and data privacy

Hospital-related defect records can accidentally contain patient identifiers, screenshots with personal information, tokens or payment details. Repository policy should instruct reporters to use synthetic data, redact screenshots and remove secrets from logs. Access should be role-based, retention should follow organizational policy and exported reports should be checked before sharing.

Governance includes backups, access review, workflow ownership and a documented response to repository outages. The repository should not be the only copy of critical release decisions if the tool is unavailable during a release. Approved exports or change records may be retained in a controlled location, with a clear source of truth.

Integrations with code hosting, CI and chat can improve traceability but should be configured with least privilege. Automated comments and build links must not expose confidential information. Test the integration permissions and verify that issue closure cannot be triggered accidentally by an unrelated commit message.

Evidence table

Governance control	Implementation	Evidence	Review frequency
Least privilege	Role-based access by project responsibility	Access list	Periodic and on role change
Sensitive-data prevention	Reporter checklist and redaction guidance	Sample audit	Each release / incident
Backup and recovery	Scheduled backup and restore test	Restore result	Defined operations cadence
Workflow ownership	Named repository administrator	Ownership record	At team or process change
Integration security	Scoped tokens and secret storage	Configuration review	After integration changes
Retention	Approved retention and deletion policy	Policy record	According to organizational schedule

Interpretation and control implication: The table is a planning aid. Each expected result must be checked against the approved requirement and verified in the intended build. Where a requirement remains ambiguous, record the decision and owner before treating the case as a release gate.

TASK 9 – INTEGRATED TESTING PRACTICE: COMPARE COMPLETE CONFIGURATIONS

Assignment weight and evidence expectations are taken from the supplied Software Testing Assignment 1 brief. The analysis below preserves the MediQueue scenario and its stated business rules. Any added experimental counts are identified as illustrative.

9.1 Integration objective and constraints

Task 9 combines the decisions from Tasks 1–8 into three complete configurations within the 100-person-hour release budget. A useful comparison must consider not only detection count but also the risk addressed, the time spent, the repair stage, test repeatability and operational cost. The release is three weeks long, and the system has a specified 800-user peak.

The configurations are: A, manual end-stage testing with a basic issue tracker; B, structured reviews and risk-based manual testing with Bugzilla; C, continuous engineering with automated business-rule regression, risk-based system testing and a configured repository. All three must include minimum patient-safety checks.

Evidence table

Configuration	Illustrative effort	Strength	Limitation	Decision
A. Manual end-stage	100 h	Fast to start	Late discovery, weak repeatability	Not preferred
B. Structured risk-based	100 h	Reviews and traceability	Manual regression cost	Transition option
C. Continuous + automation	100 h	Repeatable regression and fast feedback	Setup/maintenance required	Recommended

Interpretation and control implication

The integrated configuration must fit the 100-hour budget and preserve critical gates. If automation is not yet mature, use a phased transition rather than pretending it exists.

9.2 Configuration A – minimum manual

Configuration A performs a short requirement walkthrough, manual scenario execution near release and defect recording in a basic tracker. It is low-cost to start but has limited regression repeatability and weak coverage of rare boundaries. It may detect obvious fee and report display errors but is less reliable for race conditions and repeated payment submission.

Within 100 hours, the team may spend 10 hours planning, 60 executing manual cases, 15 retesting fixes and 15 reporting/release checks. This is an illustrative allocation and should be adjusted to real task estimates.

Evidence table

Configuration	Illustrative effort	Strength	Limitation	Decision
A. Manual end-stage	100 h	Fast to start	Late discovery, weak repeatability	Not preferred
B. Structured risk-based	100 h	Reviews and traceability	Manual regression cost	Transition option
C. Continuous + automation	100 h	Repeatable regression and fast feedback	Setup/maintenance required	Recommended

9.3 Configuration B – structured risk-based

Configuration B adds formal requirement review, decision tables, independent test cases, risk prioritization, code review and a controlled Bugzilla lifecycle. It improves early detection and traceability without requiring a full continuous pipeline. It is a practical transition step if automation capability is limited.

An illustrative allocation is 18 hours for review and planning, 42 for test design/execution, 20 for integration and targeted performance checks, 12 for retest and 8 for reporting/release review. The main risk is that repeated manual regression consumes time and can be inconsistently executed.

Evidence table

Configuration	Illustrative effort	Strength	Limitation	Decision
A. Manual end-stage	100 h	Fast to start	Late discovery, weak repeatability	Not preferred
B. Structured risk-based	100 h	Reviews and traceability	Manual regression cost	Transition option
C. Continuous + automation	100 h	Repeatable regression and fast feedback	Setup/maintenance required	Recommended

9.4 Configuration C – continuous automated and risk-based

Configuration C automates stable business rules and regression, runs build and unit tests on every change, executes integration checks for database and transaction invariants, and reserves manual effort for exploratory, acceptance, concurrency and clinical workflows. A release gate blocks critical failures and records the accepted residual risks.

An illustrative 100-hour allocation is 15 hours for requirement/risk review and planning, 25 for test design and automation maintenance, 25 for integration/regression and defect diagnosis, 20 for targeted performance/concurrency/clinical acceptance, and 15 for retest, reporting and release review. These categories are planning assumptions, not time-sheet results.

Evidence table

Configuration	Illustrative effort	Strength	Limitation	Decision
A. Manual end-stage	100 h	Fast to start	Late discovery, weak repeatability	Not preferred
B. Structured risk-based	100 h	Reviews and traceability	Manual regression cost	Transition option
C. Continuous + automation	100 h	Repeatable regression and fast feedback	Setup/maintenance required	Recommended

Interpretation and control implication

The integrated configuration must fit the 100-hour budget and preserve critical gates. If automation is not yet mature, use a phased transition rather than pretending it exists.

9.5 Cross-configuration comparison

Compare configurations on critical fault detection, coverage of boundary rules, escaped-defect risk, repeatability, traceability, environment/tool cost and fit with the three-week cycle. Configuration A has the lowest initial setup cost but the highest dependence on individual tester memory. B improves evidence and review. C provides the best regression feedback when automation is reliable and maintained.

The preferred configuration is C for recurring releases, with B-style risk gates for high-consequence decisions. If the team cannot build stable automation immediately, use B as a controlled transition while prioritizing the most deterministic and frequently repeated tests for automation.

Evidence table

Release gate	Required evidence	Fail condition
Critical business rules	Age/cancellation/validity tests	Any critical rule test fails
Payment integrity	Refresh/retry produces one bill	Duplicate charge or invoice
Appointment integrity	Concurrent slot booking	Two confirmations for same slot
Performance	800 concurrent users; response ≤ 2 s target	Target not met without accepted mitigation
Defect status	Triage and severity report	Open critical defect without approved containment

9.6 Integrated workflow and decision table

The integrated workflow begins with approved requirements and a risk register; creates decision tables and traceable tests; runs unit tests, static checks and integration tests; performs risk-based system and acceptance testing; logs defects with reproducible evidence; retests fixes; and conducts a documented release decision. Payment, appointment concurrency and prescription validity require explicit non-waivable controls.

A release decision table should state the condition, evidence owner and outcome. A critical billing or pharmacy test failure blocks release. A noncritical cosmetic issue may be accepted with a documented owner and due date. Performance failure beyond the two-second target requires capacity investigation and explicit risk acceptance, not silent waiver.

Evidence table

Release gate	Required evidence	Fail condition
Critical business rules	Age/cancellation/validity tests	Any critical rule test fails
Payment integrity	Refresh/retry produces one bill	Duplicate charge or invoice
Appointment integrity	Concurrent slot booking	Two confirmations for same slot
Performance	800 concurrent users; response ≤ 2 s target	Target not met without accepted mitigation
Defect status	Triage and severity report	Open critical defect without approved containment
Clinical acceptance	Pharmacist/medical sign-off	No authorized sign-off

9.7 Risk transfer and limitations

Automation reduces repetitive effort but introduces maintenance risk: a brittle test can fail for irrelevant reasons or encode a wrong expected value. Test data must be isolated and reset; payment and appointment tests must be safe to rerun; concurrency tests need reproducible synchronization; and production data must not be copied into insecure test environments.

The recommendation is a risk-based continuous practice, not an assertion that automation finds every defect. The team should monitor escaped defects, root causes, test flakiness, coverage of critical requirements and time-to-feedback after each release.

Evidence table

Release gate	Required evidence	Fail condition
Critical business rules	Age/cancellation/validity tests	Any critical rule test fails
Payment integrity	Refresh/retry produces one bill	Duplicate charge or invoice
Appointment integrity	Concurrent slot booking	Two confirmations for same slot
Performance	800 concurrent users; response ≤ 2 s target	Target not met without accepted mitigation
Defect status	Triage and severity report	Open critical defect without approved containment

Interpretation and control implication

The integrated configuration must fit the 100-hour budget and preserve critical gates. If automation is not yet mature, use a phased transition rather than pretending it exists.

9.7 Integrated toolchain and evidence flow

An integrated testing practice is more than a collection of tools. It defines how a requirement becomes a test, how a code change triggers execution, how a failure becomes a defect, and how a verified fix contributes to a release decision. A traceable flow reduces manual transcription and makes it possible to answer which approved rules were tested in a given build.

For MediQueue, a requirement identifier such as BR-03 for the age concession should link to the decision table, automated unit tests, API tests, system scenarios and any defect raised against that rule. A commit should identify the change and build; CI results should identify the build; the defect record should link to the failure and fix; and the release record should reference the accepted evidence.

Automation should be introduced in a controlled order. First stabilize test data and expected results; then automate deterministic unit and API cases; next add integration and end-to-end smoke tests; finally schedule heavier concurrency and performance scenarios. Automating unstable scenarios without improving testability can create noise rather than confidence.

Evidence table

Stage	Tool or artefact	Input	Output / trace link
Requirement	Requirements register	Approved rule	Stable requirement ID
Test design	Test management or repository	Requirement and risk	Test case IDs
Code change	Version control	Work item and implementation	Commit / pull request
Build and checks	CI pipeline	Versioned commit	Build ID and test report
Defect	Issue tracker	Failed assertion and evidence	Defect ID linked to test
Fix verification	CI plus QA retest	Candidate correction	Retest evidence
Release gate	Release checklist	Quality and risk evidence	Go/no-go record
Monitoring	Logs and metrics	Production behaviour	Incident/defect feedback

Interpretation and control implication: The table is a planning aid. Each expected result must be checked against the approved requirement and verified in the intended build. Where a requirement remains ambiguous, record the decision and owner before treating the case as a release gate.

9.8 Automation pyramid and suite maintenance

A balanced suite uses many fast, focused tests and fewer expensive end-to-end tests. Unit tests are well suited to fee boundaries and date comparisons. API tests verify business rules and persistence contracts. Integration tests cover payment callbacks, appointment reservation and database constraints. End-to-end tests confirm that the patient journey works through the interface, but they should be reserved for high-value paths because they are slower and more sensitive to environment changes.

Automation creates maintenance obligations. Test code should have readable names, stable fixtures, independent assertions and clear failure messages. If a test fails intermittently, the team should investigate rather than repeatedly rerun until it passes. Flaky tests reduce trust and can train the team to ignore genuine failures.

Every automated test should have an owner and a purpose. Tests that no longer protect a requirement, duplicate other checks or depend on obsolete interfaces should be reviewed. Deleting a test should be a deliberate decision supported by traceability and risk analysis, not a quick way to make the pipeline green.

Evidence table

Layer	Example	Frequency	Main limitation
Unit	Fee function at ages 4, 5, 17, 18, 59, 60	Every relevant commit	Cannot prove integration wiring
API	Prescription validity and duplicate dispense	Every merge / build	May miss visual issues
Integration	Payment callback plus database state	Merge or nightly	Requires controlled dependencies
End-to-end	Register, book, pay and view confirmation	Release smoke and scheduled run	Slow and environment-sensitive
Concurrency	Two simultaneous requests for one slot	Nightly / release gate	Needs synchronized test harness
Performance	800-user representative workload	Performance gate / release candidate	Environment must be representative

Interpretation and control implication: The table is a planning aid. Each expected result must be checked against the approved requirement and verified in the intended build. Where a requirement remains ambiguous, record the decision and owner before treating the case as a release gate.

9.9 Operational readiness and observability

Testing should verify that the application can be operated safely after deployment. Observability includes useful logs, metrics and alerts that help distinguish a slow database from a payment gateway delay or a growing queue. The system should avoid logging secrets or unnecessary clinical information while still retaining transaction IDs and state transitions required for diagnosis.

Operational tests should cover deployment, migration, configuration, rollback and recovery. A release can pass functional tests but fail because a migration locks a table, a required secret is missing or a monitoring alert is misconfigured. A deployment rehearsal with synthetic data can expose these issues before the production window.

Rollback plans should state which changes are reversible and how data consistency will be protected. For example, rolling back application code after a schema change may not be safe unless the schema remains backward compatible. The release plan should include validation after deployment and a clear decision owner.

Evidence table

Operational scenario	Test action	Expected evidence	Risk controlled
Configuration missing	Start candidate without required setting	Clear startup failure and alert	Silent partial operation
Migration failure	Inject failure in staging migration	Safe stop or documented recovery	Corrupt schema
Slow database	Simulate elevated query latency	Latency metrics and alert	Undetected degradation
Payment gateway outage	Run controlled unavailable response	Safe user message and retry state	Duplicate charges

Rollback	Revert candidate in staging	Service restored and data consistent	Unsafe recovery
Log review	Inspect workflow correlation	Trace without secrets or excess patient data	Poor diagnosis/privacy exposure

Interpretation and control implication: The table is a planning aid. Each expected result must be checked against the approved requirement and verified in the intended build. Where a requirement remains ambiguous, record the decision and owner before treating the case as a release gate.

TASK 10 – RE-APPLY AND ADAPT THE TESTING PRACTICE

Assignment weight and evidence expectations are taken from the supplied Software Testing Assignment 1 brief. The analysis below preserves the MediQueue scenario and its stated business rules. Any added experimental counts are identified as illustrative.

10.1 Changed constraints and adaptation principle

The final task asks the QA team to adapt the selected practice when constraints change. A useful process should preserve safety invariants while adjusting the mix of manual work, automation, exploratory testing and reporting. If the available budget falls, the team must not treat every test as equally optional; it should rank by severity, likelihood, exposure and detectability.

The adaptation decision should be evidence-based: retain practices that demonstrably reduce risk, modify activities whose cost exceeds value, and redesign controls that repeatedly fail to prevent critical escapes. Decisions and residual risks must be recorded for audit.

Evidence table

Constraint/change	Adaptation	Must remain protected
Budget reduced	Prioritize P0/P1 risk; reuse automation	Patient safety and financial invariants
Rule changes	Update decision table and tests before code	Approved expected results
Critical escape repeats	Root-cause review and architecture control	No waiver without risk authority
Pipeline too slow	Split fast commit gates and scheduled heavy tests	Release-candidate full suite
Flaky tests	Stabilize data, synchronization and environment	Do not silently ignore failures

Interpretation and control implication

Adaptation should reduce low-value repetition before removing safety controls. Every deferred test or accepted defect needs a named owner and recorded rationale.

10.2 Strategy A – retain the current practice

Retaining the existing practice avoids transition effort and training. It may be appropriate when the current process meets targets and no meaningful change has occurred. In this case, the 29 escaped defects and nine critical escapes are evidence that the previous release process requires improvement, so unconditional retention is not recommended.

Retention should apply only to proven components, such as existing stable test harnesses or useful test cases, not to ineffective release habits. Keep what works and use defect evidence to change what does not.

Evidence table

Constraint/change	Adaptation	Must remain protected
Budget reduced	Prioritize P0/P1 risk; reuse automation	Patient safety and financial invariants
Rule changes	Update decision table and tests before code	Approved expected results
Critical escape repeats	Root-cause review and architecture control	No waiver without risk authority
Pipeline too slow	Split fast commit gates and scheduled heavy tests	Release-candidate full suite
Flaky tests	Stabilize data, synchronization and environment	Do not silently ignore failures

10.3 Strategy B – modify the practice

Modification adds the highest-value controls first: review age/cancellation/validity decision tables, automate fee boundaries, test idempotent payment retries, test concurrent slot reservation, include day-7/day-8 pharmacy cases and run a peak-load test. It can fit a constrained release if work is selected by risk and reused regression assets are available.

An example constrained plan reserves effort for requirement review, critical automated regression, targeted integration and concurrency tests, clinical acceptance, defect retest and release governance. Lower-risk cosmetic permutations can be deferred, but the deferral and residual risk must be documented.

Evidence table

Constraint/change	Adaptation	Must remain protected
Budget reduced	Prioritize P0/P1 risk; reuse automation	Patient safety and financial invariants
Rule changes	Update decision table and tests before code	Approved expected results
Critical escape repeats	Root-cause review and architecture control	No waiver without risk authority
Pipeline too slow	Split fast commit gates and scheduled heavy tests	Release-candidate full suite
Flaky tests	Stabilize data, synchronization and environment	Do not silently ignore failures

10.4 Strategy C – redesign the practice

Redesign is justified if the current architecture prevents reliable testability, the repository lacks traceability, or critical controls repeatedly fail despite local fixes. Examples include adding idempotency keys and database uniqueness constraints, restructuring slot reservation to be atomic, introducing synthetic data management and creating a CI pipeline with mandatory quality gates.

Redesign offers durable benefits but may exceed one release's capacity. A phased plan should separate immediate containment from structural remediation, with explicit owners and milestones.

Evidence table

Constraint/change	Adaptation	Must remain protected
Budget reduced	Prioritize P0/P1 risk; reuse automation	Patient safety and financial invariants
Rule changes	Update decision table and tests before code	Approved expected results
Critical escape repeats	Root-cause review and architecture control	No waiver without risk authority
Pipeline too slow	Split fast commit gates and scheduled heavy tests	Release-candidate full suite
Flaky tests	Stabilize data, synchronization and environment	Do not silently ignore failures

Interpretation and control implication

Adaptation should reduce low-value repetition before removing safety controls. Every deferred test or accepted defect needs a named owner and recorded rationale.

10.5 Evidence-based decision framework

Use a simple decision matrix with safety impact, repeatability, implementation effort, cycle-time impact and expected prevention value. The highest-weight criteria should be patient safety, financial integrity and risk of escaped critical defects. A lower-cost option is not automatically better if it leaves a known duplicate-billing path open.

Illustrative scoring should be labelled as judgement rather than measured fact. The team should review scores with QA, development, operations and clinical representatives and record why any high-risk control is deferred.

Evidence table

Metric	Baseline / method	Target direction

DRE	135 ÷ 164 = 82.32% historical	Improve with severity-aware interpretation
Escaped critical defects	Count by release and root cause	Reduce to zero where practicable
Boundary-rule coverage	Trace each business rule to tests	100% of critical boundaries covered
Peak response	Measure under 800 concurrent users	≤ 2 seconds requirement
Reopen rate	Reopened defects ÷ resolved defects	Decrease through better fixes and retest

10.6 Release-hour allocation under constraint

If the budget is reduced below 100 hours, protect requirement review, core business-rule regression, payment idempotency, appointment concurrency, prescription validity and performance smoke testing. Reduce duplicate manual execution before removing independent verification of a critical rule. Use automation to preserve repeated checks but retain a human review of expected results.

Suggested ordering: P0 patient-safety and financial invariants; P1 core workflow integration and peak performance; P2 usability and cross-browser breadth; P3 cosmetic details. This ordering is not permission to ignore a defect; it is a transparent method for sequencing work.

Evidence table

Metric	Baseline / method	Target direction
DRE	135 ÷ 164 = 82.32% historical	Improve with severity-aware interpretation
Escaped critical defects	Count by release and root cause	Reduce to zero where practicable
Boundary-rule coverage	Trace each business rule to tests	100% of critical boundaries covered
Peak response	Measure under 800 concurrent users	≤ 2 seconds requirement
Reopen rate	Reopened defects ÷ resolved defects	Decrease through better fixes and retest

10.7 Final decision and conditions for reversal

Recommended decision: MODIFY immediately, then progressively REDESIGN toward continuous risk-based testing. The immediate modification addresses known failures within the existing team and budget. Redesign is triggered if critical defects recur, the payment/booking architecture cannot enforce invariants, or the test repository cannot provide auditable release evidence.

Success should be measured by reduced escaped critical defects, improved DRE, fewer repeated root causes, increased critical-rule coverage, shorter feedback time and stable performance under 800 concurrent users. DRE alone must not be optimized by increasing low-value pre-release defect counts; severity and clinical consequence must remain visible.

Evidence table

Metric	Baseline / method	Target direction
DRE	135 ÷ 164 = 82.32% historical	Improve with severity-aware interpretation
Escaped critical defects	Count by release and root cause	Reduce to zero where practicable
Boundary-rule coverage	Trace each business rule to tests	100% of critical boundaries covered
Peak response	Measure under 800 concurrent users	≤ 2 seconds requirement
Reopen rate	Reopened defects ÷ resolved defects	Decrease through better fixes and retest

Interpretation and control implication

Adaptation should reduce low-value repetition before removing safety controls. Every deferred test or accepted defect needs a named owner and recorded rationale.

10.8 Adaptation when people or environments change

Testing practices must remain effective when team members, infrastructure or release cadence changes. Staff turnover can remove undocumented knowledge, so the test basis, setup instructions, ownership and known limitations should be recorded. New team members should be able to reproduce a test failure using documented data and environment steps rather than relying on an informal handover.

If the environment becomes less representative, the team should explicitly identify which conclusions remain valid. A fast local test can prove deterministic business logic but cannot establish that the production system meets an 800-user response target. Where a full environment is unavailable, use component-level evidence and a scheduled representative performance gate, and document the residual uncertainty.

When the release cadence increases, the team should move repeatable checks earlier and reduce unnecessary manual repetition. It should not compress clinical acceptance or critical risk review to fit the calendar. Faster delivery requires more dependable automation and clearer gates, not merely shorter test windows.

Evidence table

Changed condition	Adaptation	Control retained	Residual risk to record
QA team member leaves	Maintain runbooks and paired ownership	Critical test ownership	Knowledge transfer gap
Test environment unavailable	Run isolated tests and restore integration gate later	No critical release sign-off without required evidence	Integration uncertainty
Release cadence doubles	Automate stable regression and risk-based triggers	Critical invariants and approval gates	Coverage of lower-priority changes
Budget reduced	Cut duplicate low-value runs first	Safety, privacy and financial tests	Deferred breadth
New payment provider	Add contract and fault-injection tests	Idempotency and reconciliation	Provider-specific differences
Major architecture change	Rebuild traceability and test strategy	Requirement-level invariants	Migration and integration risk

Interpretation and control implication: The table is a planning aid. Each expected result must be checked against the approved requirement and verified in the intended build. Where a requirement remains ambiguous, record the decision and owner before treating the case as a release gate.

10.9 Continuous improvement loop

Continuous improvement should be a closed loop: measure a problem, identify its cause, implement a change, verify the change and decide whether to retain it. For example, if duplicate callbacks cause repeated processing, the team can introduce callback deduplication and a regression test. The next releases should then be checked for recurrence and for any new failure introduced by the fix.

Improvement actions need owners and dates. ‘Improve testing’ is too broad to verify. A better action is ‘add a synchronized two-request appointment test to the release pipeline, owned by the booking component lead, and demonstrate that only one request succeeds’. The action is complete only when evidence is linked and the result is reviewed.

The team should keep a small improvement backlog. Too many simultaneous process changes make it difficult to determine which change produced an effect. Select actions by risk reduction, repeat occurrence, effort and feasibility; complete them; then review the results before starting a new batch.

Evidence table

Problem signal	Improvement experiment	Success measure	Review decision
Repeated fee boundary defects	Add decision table and six boundary tests	No missed boundary in regression	Retain and extend if effective
Duplicate payment escapes	Add idempotency and callback replay tests	One transaction per business payment	Block release if invariant fails
Flaky end-to-end suite	Isolate fixtures and replace timing assumptions	Lower flake rate over repeated runs	Refactor remaining unstable cases
Long diagnosis time	Add correlation IDs and structured logs	Reduced time to identify component	Check privacy and observability
Unclear release decisions	Adopt mandatory go/no-go checklist	Every release has evidence and owner	Audit sample of release records
Recurring prescription issue	Add domain-approved validity cases	No invalid dispense in test suite	Clinical owner reviews rule changes

Interpretation and control implication: The table is a planning aid. Each expected result must be checked against the approved requirement and verified in the intended build. Where a requirement remains ambiguous, record the decision and owner before treating the case as a release gate.

10.10 Final implementation roadmap

The recommended implementation should be phased to achieve near-term protection without requiring a complete organizational redesign in one release. In the first phase, clarify ambiguous rules, agree severity definitions and protect the highest-risk invariants. In the second phase, automate deterministic business rules and establish stable test data. In the third phase, connect CI results to defect records and add targeted concurrency and performance gates.

The fourth phase improves observability, reporting and root-cause analysis. Once the team has reliable evidence, it can use trend data to refine priorities and remove redundant checks. Each phase should have a named owner, deliverables, acceptance criteria and a review date. This makes process change measurable and prevents an initiative from becoming a document-only exercise.

The final decision remains to modify the current practice immediately and progressively redesign toward continuous, risk-based engineering. The practice should be judged by reduced critical escapes, stable financial and appointment invariants, useful defect evidence and an honest release decision—not by the number of pages in the plan or the raw number of test cases.

Evidence table

Phase	Timeline concept	Deliverables	Completion gate
1. Stabilize rules	Immediate	Approved fee, cancellation, prescription and booking rules	Business/clinical decisions recorded
2. Protect invariants	Next iteration	Boundary, idempotency and concurrency regression tests	Critical tests pass
3. Integrate evidence	Following iteration	CI reports linked to test and defect IDs	Traceability demonstrated
4. Add operational tests	Release candidate	Performance, rollback and monitoring evidence	Operations and QA review
5. Improve by data	Each release	Trend review and prevention backlog	Actions assigned and reviewed
6. Reassess process	Periodic	Updated risk model and test allocation	Stakeholder sign-off

Interpretation and control implication: The table is a planning aid. Each expected result must be checked against the approved requirement and verified in the intended build. Where a requirement remains ambiguous, record the decision and owner before treating the case as a release gate.

ANNEXURE A – TEST CASE REGISTER

Test cases must be traceable to a requirement and carry concrete expected values. The examples below focus on rules that were implicated by the supplied defects. They are not a complete production test suite.

Test ID	Requirement	Input	Expected result	Type
TC-01	BR-02	Age 4, base ₹500	₹0	Boundary
TC-02	BR-03	Age 5, base ₹500	₹400	Boundary
TC-03	BR-03	Age 17, base ₹500	₹400	Boundary
TC-04	BR-04	Age 18, base ₹500	₹500	Boundary
TC-05	BR-04	Age 59, base ₹500	₹500	Boundary
TC-06	BR-05	Age 60, base ₹500	₹300	Boundary
TC-07	BR-06	Cancel 1 h 59 min before	₹100 fee	Boundary
TC-08	BR-06	Cancel exactly 2 h before	No fee	Boundary
TC-09	BR-06	Cancel 2 h 1 min before	No fee	Boundary
TC-10	BR-07	Third active booking	Rejected	Negative
TC-11	BR-08	Prescription day 7	Allowed only if policy defines inclusive boundary	Boundary/policy
TC-12	BR-08	Prescription day 8	Rejected	Boundary
TC-13	D2	Refresh confirmation page	Exactly one bill	Idempotency
TC-14	D4	Two simultaneous slot confirmations	One booking succeeds	Concurrency

TC-15	D5	Surname D'Souza	Registration succeeds safely	Input
TC-16	D6	800 concurrent users	Response time ≤ 2 s target	Performance
TC-17	D7	Reschedule after reminder scheduled	Old reminder invalidated; new time used	Integration
TC-18	D10	Blood-test report PDF	Report-issue date shown	Data mapping

ANNEXURE B – PYTHON REFERENCE IMPLEMENTATION

The reference implementation is deliberately simple and should be reviewed against the approved specification. It uses integer age values and the stated ₹500 base fee. The cancellation comparison uses a strict less-than operator because the supplied wording says "less than 2 hours".

Illustrative Python implementation:

```
def consultation_fee(age, base_fee=500):
    if age < 0:
        raise ValueError("age must be non-negative")
    if age < 5:
        return 0
    if age <= 17:
        return round_half_up(base_fee * 0.80)
    if age <= 59:
        return round_half_up(base_fee)
    return round_half_up(base_fee * 0.60)

def round_half_up(value):
    # For non-negative currency values; avoids Python's banker's rounding.
    return int(value + 0.5)

def cancellation_fee(hours_before_slot):
    return 100 if hours_before_slot < 2 else 0

def can_add_appointment(active_appointments):
    return active_appointments < 2

def prescription_valid(days_since_prescription):
    # Confirm with the product owner whether day 7 is inclusive.
```

return 0 <= days_since_prescription <= 7

Example	Input	Expected	Reason
Fee age 60	60, ₹500	₹300	40% concession
Fee age 12	12, ₹500	₹400	20% concession
Cancel exactly 2 h	2.0 hours	₹0	Strictly less than 2 hours
Third appointment	2 active bookings	Reject	Maximum 2 active
Prescription day 8	8 days old	Reject	Outside 7-day validity

ANNEXURE C – DEFECT LOG TEMPLATE AND SAMPLE RECORDS

The brief allows a faculty-provided 164-record log or a realistic log of at least 40 records. The ten supplied defects are factual scenario inputs; any additional rows created for practice are synthetic and must be labelled as such. Do not describe synthetic entries as hospital incidents.

ID	Module	Summary	Primary class	Severity	Status
D1	Billing	Age 60 charged full fee	Code logic	Medium–High	Open
D2	Billing	Refresh creates duplicate bill	Transaction/idempotency	High	Open
D3	Appointment	3-hour cancellation charged late fee	Boundary logic	Medium	Open
D4	Appointment	Concurrent users get same slot	Concurrency	High	Open
D5	Registration	Apostrophe causes DB error	Input/data handling	Medium	Open
D6	Lab Reports	35-second page at peak	Performance	High at peak	Open
D7	Reminders	Old time sent after reschedule	Requirement/integration	Medium–High	Open
D8	Billing	TC-52 expects wrong fee	Test oracle	High risk	Open
D9	Pharmacy	Day-9 prescription dispensed	Date logic/safety	High	Open
D10	Lab Reports	Wrong date field in PDF	Data mapping	Medium–High	Open

Additional synthetic rows should be marked SYN-001, SYN-002 and so on, and should never be blended with the ten supplied scenario defects without a source column identifying the record origin.

ANNEXURE D – CALCULATION SHEET

Calculations in this report use the provided baseline and cost assumptions. The 135 pre-release defects plus 29 post-release defects equal 164 total known defects. The DRE and escape fraction are complementary when the denominator is the same.

Metric	Calculation	Result
Baseline DRE	135 ÷ 164 × 100	82.32%
Escaped fraction	29 ÷ 164 × 100	17.68%
Critical among escaped defects	9 ÷ 29 × 100	31.03%
Billing size share	12 ÷ 47 × 100	25.53% of KLOC
Appointment size share	10 ÷ 47 × 100	21.28% of KLOC
Raw input space	250×20×3×3×2×2	180,000 combinations
Exhaustive time at 1/s	180,000 ÷ 3,600	50 hours

ANNEXURE E – RELEASE CHECKLIST AND RISK ACCEPTANCE

The release checklist is designed to prevent deadline pressure from silently bypassing patient-safety controls. A waiver must identify the defect, affected workflow, containment, owner, expiry date and approving authority.

Gate	Pass condition	Owner	Evidence
Requirements	All critical rules approved; ambiguities resolved	Product owner + QA	Signed decision table
Unit tests	All critical business-rule tests pass	Developers	CI report
Integration	Payment idempotency and slot uniqueness pass	QA + developers	Automated run IDs
Pharmacy	Validity and dispensing controls accepted	Pharmacist	Clinical acceptance record
Performance	800 concurrent users; response target assessed	QA/operations	Load report
Defect status	No open critical defect without formal containment	QA lead	Tracker dashboard
Release approval	Technical and clinical sign-off recorded	QA lead + clinical owner	Release record

ANNEXURE F – LIMITATIONS AND REFERENCES

This report is based on the supplied assignment brief and provided model-answer document as a formatting/detail reference. The provided answer document describes a different university portal scenario; its scenario-specific values and conclusions were not copied into this MediQueue report. No real tool screenshots, production logs or measured execution results were supplied.

Source and evidence limitations

- Primary source: supplied Software Testing Assignment 1 brief for Lakeside Hospital MediQueue, including the stated rules, operating data, and D1–D10 defect symptoms.
- Formatting/detail reference: supplied 93-page answer/front-page document. Its main answer content uses a different CampusConnect student-portal scenario, so its scenario-specific numbers were not treated as MediQueue evidence.
- General testing concepts used: boundary-value analysis, decision tables, regression testing, defect-removal efficiency, risk-based testing, fault-tree analysis, Pareto analysis, defect tracking and CI quality gates.
- All illustrative experiment results, staffing estimates and synthetic examples should be replaced with actual test output and tool evidence where the assignment requires observed results.

FINAL RECOMMENDATIONS

The hospital should adopt continuous engineering activity and automated DevOps gates for stable regression tests, while applying risk-driven reviews and clinical sign-off to payment, appointment and pharmacy workflows. This combines fast feedback with explicit control over patient-safety decisions.

Priority	Action	Owner	Verification
P0	Correct age 60 comparator and wrong concession oracle; add boundary tests	Billing developer + QA	Ages 4, 5, 17, 18, 59, 60 pass
P0	Make payment confirmation idempotent	Billing/API developer	Refresh/retry creates one bill
P0	Enforce atomic appointment-slot reservation	Appointment developer + DBA	Concurrent confirmation yields one booking
P0	Enforce prescription validity and duplicate-dispense constraints	Pharmacy developer + pharmacist	Day 8 rejected; duplicate dispense blocked
P1	Clarify reminder rescheduling behaviour	Product owner + QA	Approved state-transition requirement
P1	Run load test at 800 concurrent users	QA/operations	Response time assessed against ≤2 s
P1	Adopt structured defect repository and required fields	QA lead	All D1–D10 traceable and reproducible
P2	Review Pareto and per-KLOC metrics after full log is tagged	QA lead	Validated report using complete 164 records

FINAL NOTE ON ACADEMIC EVIDENCE

This document is a detailed report draft, not a claim that experiments were executed in a live hospital environment. Modelled figures are marked as illustrative, and the missing full defect log, real tracker screenshots, independent classmate ratings and actual test-run outputs are identified. Before submission, replace placeholders and illustrative results with genuine evidence if the faculty expects actual execution.

ANNEXURE G – PAYMENT IDEMPOTENCY TEST MATRIX

Payment retries are a high-priority risk because network timeouts and browser refreshes are normal operating conditions. The invariant is that one consultation must result in no more than one successful charge and one bill, even when the client repeats a request. Tests should use a payment sandbox and synthetic patient identifiers.

ID	Trigger	Expected behaviour	Evidence
PAY-01	Submit payment once	One transaction and one bill	Transaction ID + invoice ID
PAY-02	Refresh confirmation page	Existing result shown; no second charge	Before/after bill count
PAY-03	Double-click confirm	One request is processed idempotently	Request logs
PAY-04	Retry after timeout	Server returns stored transaction outcome	Idempotency key record
PAY-05	Same key, changed amount	Request rejected and audited	API response + audit event
PAY-06	Payment succeeds but UI times out	Reconciliation finds successful payment	Gateway and local status comparison
PAY-07	Two requests arrive concurrently	Only one bill persists	Database row count
PAY-08	Payment gateway returns failure	No successful bill; retry policy clear	Gateway response and bill status
PAY-09	Duplicate callback arrives	Callback handled once	Callback processing log
PAY-10	User navigates back and resubmits	No duplicate transaction	Invoice count and status

Evidence handling note: capture actual execution output and link it to the test ID. Any unexecuted case must be marked Not Run, not Pass. Where the expected result depends on an unresolved policy question, record the decision owner and keep the case blocked until the requirement is approved.

ANNEXURE H – APPOINTMENT CONCURRENCY AND STATE TESTS

Appointment correctness is a system invariant, not just a screen validation. The server must reserve a doctor's slot atomically and the database must reject conflicting reservations. A concurrency test should synchronize requests so that multiple clients attempt the same slot at nearly the same time, then assert the final persisted state.

ID	Setup / trigger	Expected result	Primary layer
APT-01	One free slot; one request	Booking succeeds	Integration
APT-02	One free slot; two simultaneous requests	Exactly one succeeds	Concurrency
APT-03	Patient has zero active bookings	First and second bookings allowed	Business rule
APT-04	Patient already has two active bookings	Third booking rejected	Business rule
APT-05	Cancel 2 h 1 min before slot	No late fee	Boundary
APT-06	Cancel exactly 2 h before slot	No late fee under strict wording	Boundary
APT-07	Cancel 1 h 59 min before slot	₹100 late fee	Boundary
APT-08	Cancel then rebook slot	Slot available only after valid cancellation commit	State transition
APT-09	Reschedule appointment	Old slot released and new slot reserved atomically	Integration
APT-10	Two reschedules collide	No duplicate slot allocation	Concurrency

Evidence handling note: capture actual execution output and link it to the test ID. Any unexecuted case must be marked Not Run, not Pass. Where the expected result depends on an unresolved policy question, record the decision owner and keep the case blocked until the requirement is approved.

ANNEXURE I – PHARMACY VALIDITY AND DISPENSING TESTS

Pharmacy test design must include clinical review. The supplied rule says a prescription may be dispensed only within seven days of its date. The exact inclusivity of day seven should be explicitly approved. Tests should also ensure that the same prescription cannot be dispensed twice and that expired batches are blocked regardless of a valid prescription.

ID	Input / condition	Expected result	Risk
PH-01	Prescription dated today	Valid if other checks pass	Routine
PH-02	Prescription age 6 days	Allowed if policy confirms validity	Boundary
PH-03	Prescription age exactly 7 days	Follow approved inclusive/exclusive policy	Policy boundary
PH-04	Prescription age 8 days	Rejected	High
PH-05	Prescription age 9 days	Rejected; reproduces D9 class	High
PH-06	Expired batch selected	Dispensing blocked	High
PH-07	Quantity exceeds prescribed quantity	Dispensing blocked or authorized override required	High
PH-08	Prescription already dispensed	Second dispense blocked	High
PH-09	Two pharmacists dispense simultaneously	Only one dispense transaction succeeds	High
PH-10	Unauthorized user opens dispensing flow	Access denied and audited	Security

Evidence handling note: capture actual execution output and link it to the test ID. Any unexecuted case must be marked Not Run, not Pass. Where the expected result depends on an unresolved policy question, record the decision owner and keep the case blocked until the requirement is approved.

ANNEXURE J – LAB REPORT AND PERFORMANCE TEST PLAN

The performance requirement is a response time of no more than two seconds with 800 concurrent users during the morning peak. A performance test must state the workload mix, ramp-up, steady-state duration, data volume, environment, monitoring and acceptance criteria. A single average can hide a poor tail, so report percentiles and error rates as well as mean response time.

Test	Workload / assertion	Measure	Pass interpretation
LR-01	800 concurrent users open report page	p50/p95/p99 response	Compare with ≤2 s target and agreed percentile
LR-02	Mixed browse/download workflow	Throughput and errors	No unexplained errors or timeouts
LR-03	Large report PDF	Download latency and memory	No crash; correct content

LR-04	Peak load plus database monitoring	CPU, memory, query latency	No saturated bottleneck without mitigation
LR-05	Sustained 30-minute load	Latency drift and error rate	No progressive degradation
LR-06	Report generated for blood test	Issue date field	Report-issue date, not sample date
LR-07	Concurrent report requests	Database pool and queue depth	No resource exhaustion
LR-08	Load test after index/query change	Regression comparison	Performance does not regress

Evidence handling note: capture actual execution output and link it to the test ID. Any unexecuted case must be marked Not Run, not Pass. Where the expected result depends on an unresolved policy question, record the decision owner and keep the case blocked until the requirement is approved.

ANNEXURE K – REQUIREMENT-TO-TEST TRACEABILITY

Traceability allows the QA lead to determine which requirements are tested, which defects relate to a requirement and what must be rerun after a change. A requirement change to concession rates, appointment cancellation or prescription validity should automatically identify affected test cases and downstream modules.

Requirement	Affected module(s)	Core tests	Change-triggered regression
BR-01 base fee ₹500	Billing	Fee calculation and invoice tests	Billing unit + payment integration
BR-02 age below 5 free	Billing/Registration	Age 4 and age 5	Fee decision-table suite
BR-03 ages 5–17 receive 20% off	Billing	Age 5, 12, 17	Fee oracle + invoice regression
BR-04 ages 18–59 full fee	Billing	Age 18, 59	Fee decision-table suite
BR-05 age 60+ receives 40% off	Billing	Age 59, 60, 61	Senior boundary regression
BR-06 cancellation <2 h fee	Appointment/Billing	1:59, 2:00, 2:01	Cancellation and refund regression
BR-07 max 2 active bookings	Appointment	Attempt third booking	Appointment state regression
BR-08 prescription ≤7 days	Pharmacy	Day 7 and day 8	Dispensing regression
NFR-01 response ≤2 s at 800 users	Lab Reports	Peak load scenario	Release-candidate performance run

Evidence handling note: capture actual execution output and link it to the test ID. Any unexecuted case must be marked Not Run, not Pass. Where the expected result depends on an unresolved policy question, record the decision owner and keep the case blocked until the requirement is approved.

ANNEXURE L – DEFECT TRIAGE AND ROOT-CAUSE WORKSHEET

A defect review should separate immediate containment from root-cause prevention. The team first confirms reproducibility, impact and affected versions; then identifies the origin artefact and detection gap; finally it assigns a corrective action and verifies that the regression test would fail before the fix and pass after it.

Field	Prompt	Example for D4
Observed failure	What did the user see?	Two patients received the same doctor slot.
Business impact	What invariant was violated?	A doctor slot was double-booked.
Origin artefact	Where was the fault introduced?	Likely reservation design/code; verify by inspection.

Detection gap	Why did earlier controls miss it?	No effective synchronized concurrency test.
Containment	What reduces immediate risk?	Temporarily prevent conflicting confirmations; reconcile bookings.
Root cause	What allowed it?	Non-atomic read-then-write and missing uniqueness constraint.
Corrective action	What changes permanently?	Atomic reservation plus database uniqueness.
Regression evidence	What test proves the fix?	Two simultaneous requests; exactly one succeeds.
Owner / due date	Who closes the action?	Appointment lead; date agreed in triage.

Evidence handling note: capture actual execution output and link it to the test ID. Any unexecuted case must be marked Not Run, not Pass. Where the expected result depends on an unresolved policy question, record the decision owner and keep the case blocked until the requirement is approved.

ANNEXURE M – RELEASE READINESS SCORECARD

The scorecard is intended for the release meeting. Each row must be supported by evidence and an accountable owner. A numerical score must not be used to average away a critical failure: if a mandatory safety or billing gate fails, the release is blocked unless a formally authorized, documented containment decision is made.

Control area	Required evidence	Status field	Release rule
Requirements	Approved decision tables and resolved ambiguities	Pass / Fail / N.A.	No unresolved critical ambiguity
Business-rule tests	Boundary and negative cases pass	Pass / Fail	All P0 tests pass
Payment integrity	Refresh/retry produces one bill	Pass / Fail	Any duplicate bill blocks release
Appointment integrity	Concurrency invariant passes	Pass / Fail	Any double booking blocks release
Pharmacy safety	Validity, expiry and duplicate dispense tests pass	Pass / Fail	Clinical owner sign-off required
Performance	Peak workload report at 800 users	Pass / Fail / Risk	≤2 s target assessed; deviations approved
Defect repository	Critical issues triaged and linked	Pass / Fail	No unowned critical defect
Regression	Build and regression results recorded	Pass / Fail	No unexplained failure
Final approval	QA + clinical/business sign-off	Approved / Rejected	Release record retained

Evidence handling note: capture actual execution output and link it to the test ID. Any unexecuted case must be marked Not Run, not Pass. Where the expected result depends on an unresolved policy question, record the decision owner and keep the case blocked until the requirement is approved.

ANNEXURE N – DETAILED TEST DATA DESIGN

Test data must represent valid, invalid, boundary and conflicting states without exposing real patient information. A data plan should define creation, ownership, reset behaviour, uniqueness, cleanup and the relationship between patient, appointment, payment and prescription records. Synthetic data is preferred because it is repeatable and can be shared under controlled access.

Evidence table

Data set	Purpose	Example setup	Expected control
AGE-BOUNDARY	Verify concession bands	Synthetic patients aged 4, 5, 17, 18, 59 and 60	Exact fee values independently reviewed
BOOKING-COLLISION	Verify unique slot reservation	Two synthetic patients request same doctor/time	Only one confirmed booking
PAYMENT-RETRY	Verify idempotency	One payment key with timeout and retry	One transaction and one bill
PRESCRIPTION-DATE	Verify validity boundary	Prescription dates around policy limit	Invalid age/date blocked
ROLE-MATRIX	Verify authorization	Patient, receptionist, pharmacist, admin accounts	Only permitted actions succeed
REPORT-ACCESS	Verify data isolation	Two patients with separate reports	Cross-patient access denied
LOAD-POOL	Performance workload	Synthetic mix of browse, book, pay and report	Target assessed under stated concurrency
RECOVERY	Verify rollback	Candidate deployment with reversible fixture data	Service and data state recover safely

Use and validation note: this annexure is a working control artefact. Populate environment-specific fields only after confirming the actual implementation. Illustrative examples are not measured results; all execution claims must be supported by reproducible evidence and linked to a build identifier.

ANNEXURE O – API AND DATABASE ASSERTION CATALOGUE

API response codes are not enough to establish correctness. Tests should check response payloads, persisted records, transaction state and audit events where appropriate. The database assertions below are examples of the kind of evidence required; exact table and column names must match the actual implementation and should not be invented in the final execution record.

Evidence table

Scenario	API-level assertion	Persistence assertion	Audit/secondary assertion
Fee calculation	Returned fee equals approved amount	Invoice stores same amount and currency	Fee rule/version recorded if supported
Booking success	One booking ID returned	One active reservation for slot	Confirmation event recorded
Booking collision	Conflict response returned	No second active reservation	Failure is diagnosable
Payment retry	Stored outcome returned for same key	Only one successful transaction	Retry is traceable by correlation ID
Cancellation boundary	Correct fee decision returned	Appointment state and ledger agree	Cancellation time recorded
Prescription expired	Dispense rejected	No dispense transaction created	Reason is captured without excess sensitive data
Report authorization	Allowed or denied response	Only authorized record accessed	Access attempt auditable
Migration	Service starts with new schema	Constraints/indexes exist	Migration version recorded

Use and validation note: this annexure is a working control artefact. Populate environment-specific fields only after confirming the actual implementation. Illustrative examples are not measured results; all execution claims must be supported by reproducible evidence and linked to a build identifier.

ANNEXURE P – EXPLORATORY TEST CHARTERS

Exploratory testing complements scripted regression by investigating behaviours that are difficult to enumerate fully. Each charter should have a mission, timebox, data conditions, risks to probe and a short debrief.

Exploratory sessions must still leave evidence: notes, screenshots with sensitive data redacted, defect IDs and unresolved questions.

Evidence table

Charter	Mission	Suggested probes	Debrief output
Payment interruption	Explore failure and recovery during payment	Refresh, back navigation, timeout, callback replay, browser close	Observed states and duplicate-charge evidence
Appointment state	Explore booking/cancel/reschedule transitions	Two tabs, simultaneous users, cancellation and rebooking	State transition gaps and collision evidence
Fee interpretation	Challenge age and date assumptions	Birthday on appointment day, timezone/date rollover, missing DOB	Ambiguities for business owner
Prescription workflow	Explore validity and dispensing restrictions	Boundary dates, duplicate dispense, expired stock, quantity mismatch	Clinical rule questions and defects
Report privacy	Explore authorization and navigation	Direct URL, changed identifier, expired session, role change	Access-control evidence
Performance degradation	Explore user experience under slow dependencies	Slow database, gateway timeout, repeated submission	Resilience observations and logs

Use and validation note: this annexure is a working control artefact. Populate environment-specific fields only after confirming the actual implementation. Illustrative examples are not measured results; all execution claims must be supported by reproducible evidence and linked to a build identifier.

ANNEXURE Q – DEFECT REPORT EXEMPLARS

A high-quality defect record describes an observable failure and enough context to reproduce it. The examples below are illustrative templates, not claims that the defects were executed in a real hospital system. Replace build identifiers, timestamps, screenshots and outcomes with genuine evidence when performing the tests.

Evidence table

Field	Example: fee boundary	Example: booking collision
Title	Age 60 receives full consultation fee	Concurrent requests confirm same doctor slot
Environment	Test build / synthetic patient	Test build / two synthetic users
Preconditions	Base fee ₹500; approved senior rule	One available doctor slot
Steps	Open billing; select age 60; calculate fee	Synchronize two booking requests for same slot
Expected	₹300 under 40% concession rule	Exactly one request succeeds
Actual	Record observed value only after execution	Record both responses and persisted state
Severity	Set based on financial impact	High if duplicate active bookings are confirmed
Evidence	Request/response and invoice record	Correlation IDs and booking rows
Regression link	Age decision-table test ID	Concurrency test ID

Use and validation note: this annexure is a working control artefact. Populate environment-specific fields only after confirming the actual implementation. Illustrative examples are not measured results; all execution claims must be supported by reproducible evidence and linked to a build identifier.

ANNEXURE R – PERFORMANCE TEST EXECUTION TEMPLATE

The performance target in the brief is a response time no greater than two seconds with 800 concurrent users during the morning peak. The test report must identify what 'response time' means, whether the threshold applies to mean or percentile, which endpoints are measured, how concurrency is generated and whether the test environment is representative. Do not label a result as passed until the acceptance interpretation is approved.

Evidence table

Field	Required entry
Build and environment	Build ID, infrastructure size, database version, network configuration
Workload mix	Percent of registration, booking, payment, report and browsing requests
Ramp-up	How users are increased to the target concurrency
Steady-state duration	Length of period at target load
Data volume	Synthetic patient, appointment and report count
Measurements	p50, p95, p99, throughput, error rate, CPU, memory, DB latency
Acceptance rule	Approved percentile/threshold and allowed error rate
Observed result	Actual measured values; do not use illustrative values as execution
Bottleneck analysis	Evidence-backed bottleneck and proposed remedy
Retest	Same workload after changes, with comparison to baseline

Use and validation note: this annexure is a working control artefact. Populate environment-specific fields only after confirming the actual implementation. Illustrative examples are not measured results; all execution claims must be supported by reproducible evidence and linked to a build identifier.

ANNEXURE S – SECURITY AND PRIVACY TEST MATRIX

The system handles patient and payment-related workflows, so security and privacy checks are part of release quality. The matrix is a planning checklist, not a claim of formal penetration testing. Security testing should be authorized, conducted in a controlled environment and coordinated with the system owner.

Evidence table

ID	Test idea	Expected result	Evidence
SEC-01	Patient attempts to open another patient's report URL	Access denied	Request/response and authorized test IDs
SEC-02	Unauthenticated request to protected API	Authentication required	API response
SEC-03	Receptionist attempts pharmacist-only action	Permission denied	Role and audit record
SEC-04	Expired session submits payment	Safe rejection or re-authentication	Session state and transaction count
SEC-05	Logs inspected for secrets	No credentials or payment secrets exposed	Redacted log review
SEC-06	Invalid input supplied to API	Validation error without unsafe processing	Response and server log
SEC-07	Repeated failed login attempts	Approved throttling/monitoring behaviour	Security event record
SEC-08	Exported defect evidence reviewed	Patient identifiers and secrets redacted	Evidence checklist

Use and validation note: this annexure is a working control artefact. Populate environment-specific fields only after confirming the actual implementation. Illustrative examples are not measured results; all execution claims must be supported by reproducible evidence and linked to a build identifier.

ANNEXURE T – TEST ENVIRONMENT AND REPRODUCIBILITY CHECKLIST

A test result is useful only when another person can understand which version, data and configuration produced it. The environment checklist standardizes build identification, database reset, service availability, clock/timezone settings, mock-provider behaviour and test account access. These details are especially important for date boundaries and asynchronous payment callbacks.

Evidence table

Control	Checklist item	Status/evidence field
Versioning	Record application commit/build and schema version	Build ID / migration ID
Configuration	Record non-secret feature flags and environment profile	Config version
Data	Use synthetic fixtures and documented reset steps	Fixture version
Time	Confirm timezone and clock assumptions	Timezone and timestamp source
Dependencies	Record payment gateway/sandbox and service versions	Dependency status
Access	Use least-privilege test accounts	Role/account alias
Logging	Enable correlation IDs and useful timestamps	Log query reference
Isolation	Ensure tests do not interfere with one another	Isolation check
Cleanup	Remove or reset generated test records	Cleanup result
Repeatability	Second tester can reproduce the case	Reproduction confirmation

Use and validation note: this annexure is a working control artefact. Populate environment-specific fields only after confirming the actual implementation. Illustrative examples are not measured results; all execution claims must be supported by reproducible evidence and linked to a build identifier.

ANNEXURE U – REQUIREMENT AMBIGUITY REGISTER

A requirement ambiguity register makes uncertain policy visible and assigns it to an authorized decision owner. It prevents testers from silently turning an assumption into a pass/fail oracle. The sample entries below identify areas that should be clarified from the approved MediQueue specification before production use.

Evidence table

ID	Question	Why it matters	Decision owner	Release treatment
AMB-01	Is age calculated on registration date or appointment date?	Changes fee band near birthday	Business owner	Hold affected oracle until approved

AMB-02	Is prescription day seven inclusive?	Changes dispensing eligibility	Clinical/pharmacy owner	Block validity rule sign-off
AMB-03	Does exactly two hours before appointment incur a fee?	Boundary interpretation affects billing	Business owner	Use strict wording only if confirmed
AMB-04	Which percentile must be within two seconds?	Mean can hide slow tail	Service owner	Approve measurable performance gate
AMB-05	Can payment be retried with changed amount?	Risk of inconsistent charge	Payment owner	Reject conflicting idempotency payload
AMB-06	What happens if gateway succeeds but local persistence fails?	Potential duplicate charge or unknown status	Technical/payment owner	Require recovery and reconciliation design

Use and validation note: this annexure is a working control artefact. Populate environment-specific fields only after confirming the actual implementation. Illustrative examples are not measured results; all execution claims must be supported by reproducible evidence and linked to a build identifier.

ANNEXURE V – SAMPLE TEST EXECUTION SUMMARY

The following table is a blank execution structure. It must be populated with real test outcomes after running the test suite. A planned test case is not evidence of a pass, and an expected result is not the same as an observed result. Use Pass, Fail, Blocked or Not Run with supporting evidence.

Evidence table

Test ID	Build	Result	Evidence reference	Defect ID	Retest result
FEE-AGE-04	Enter actual build	Not Run	Add report/log	N/A	N/A
FEE-AGE-05	Enter actual build	Not Run	Add report/log	N/A	N/A
PAY-RETRY-01	Enter actual build	Not Run	Add transaction evidence	N/A	N/A
APT-CONC-01	Enter actual build	Not Run	Add synchronized request evidence	N/A	N/A
PH-VALID-08	Enter actual build	Not Run	Add dispensing evidence	N/A	N/A
PERF-800-01	Enter actual build	Not Run	Add load report	N/A	N/A
SEC-REPORT-01	Enter actual build	Not Run	Add authorization evidence	N/A	N/A
REG-CRITICAL-01	Enter actual build	Not Run	Add CI report	N/A	N/A

Use and validation note: this annexure is a working control artefact. Populate environment-specific fields only after confirming the actual implementation. Illustrative examples are not measured results; all execution claims must be supported by reproducible evidence and linked to a build identifier.

ANNEXURE W – RELEASE MEETING MINUTES TEMPLATE

The release meeting should record the evidence reviewed, outstanding defects, known limitations, test coverage gaps and explicit decision. A meeting without a written decision can leave the team uncertain about who accepted a risk. The minutes should distinguish ‘testing complete’ from ‘testing incomplete but risk accepted’.

Evidence table

Agenda item	Required discussion	Record
Candidate identity	Build, commit and deployment package	Version and checksum/reference

Critical test results	P0/P1 business and safety invariants	Pass/fail/blocked with evidence
Open defects	Severity, workaround, owner and due date	Linked defect IDs
Performance	Workload and response-time interpretation	Report link and exceptions
Security/privacy	Authorization and sensitive-data controls	Evidence and open issues
Operations	Monitoring, rollback, backup and support owner	Readiness status
Risk acceptance	Risk, containment, approver and expiry	Formal decision reference
Decision	Go, conditional go or no-go	Named authority and timestamp

Use and validation note: this annexure is a working control artefact. Populate environment-specific fields only after confirming the actual implementation. Illustrative examples are not measured results; all execution claims must be supported by reproducible evidence and linked to a build identifier.

ANNEXURE X – GLOSSARY OF TESTING TERMS

Term	Definition
Acceptance criteria	Conditions that must be met for a requirement or release to be accepted.
Boundary-value analysis	A technique focused on values at and around decision limits.
Build identifier	A unique label linking a test result to the software version executed.
Concurrency	Overlapping operations that may interact through shared state.
Defect-removal efficiency	The proportion of known defects detected before release under a defined counting method.
Equivalence partition	A group of inputs expected to be treated similarly by the system.
Expected result	The independently derived correct outcome for a test.
Idempotency	A repeated request has no additional effect beyond the original business operation.
Metamorphic testing	Testing relationships between outputs when inputs are transformed.
Oracle	A source or method used to decide whether observed behaviour is correct.
Regression testing	Testing after change to detect unintended effects.
Risk-based testing	Allocating effort according to likelihood and consequence of failure.
Severity	The impact of a defect if it occurs.
Test traceability	Links between requirements, test cases, defects, code changes and release evidence.
Testability	How readily behaviour can be controlled, observed and verified.
Verification	Checking that a work product meets specified requirements.
Validation	Checking that the delivered behaviour meets user or business needs.

The glossary is included to keep terminology consistent across the report. Definitions are concise working definitions for this assignment and should be aligned with the terminology adopted by the course instructor where required.

ANNEXURE Y – WORKED SCENARIOS AND END-TO-END WALKTHROUGHS

This annexure turns the report's core principles into end-to-end scenarios that connect requirements, test data, actions, expected outcomes, persistence checks and release evidence. The examples are planned test designs, not results from a live hospital system. During execution, the tester should record the actual build, data identifiers, timestamps, observed responses, database evidence and any linked defect.

Y.1 Walkthrough: consultation fee and payment

The purpose is to prove that the approved age rule produces the correct consultation amount and that the amount remains consistent from the user interface through the API, invoice, payment request and stored transaction. A fee displayed correctly on one screen is not sufficient if a later component recomputes it differently. The test should use synthetic patient records and a controlled payment sandbox.

Preconditions include a base fee of ₹500, an approved rule table, a known build, a clean synthetic patient, a payment sandbox account and a way to inspect invoice and transaction records. The tester should independently calculate the expected amount before execution. At age 60, for example, the 40% concession means the patient pays ₹300. The expected value must not be generated by calling the same production function used to calculate the actual value.

Step	Action	Expected outcome	Evidence to capture
1	Create synthetic patient with age 60 under the approved age-calculation rule	Patient record is valid and age basis is known	Synthetic patient alias and DOB basis
2	Open consultation billing and request fee calculation	Displayed fee is ₹300	Screen evidence and API response
3	Continue to payment without changing patient or service	Payment amount remains ₹300	Payment request and correlation ID
4	Complete sandbox payment	One successful transaction is recorded	Gateway transaction ID and status
5	Refresh confirmation page	Existing outcome is displayed; no new charge occurs	Before/after transaction count
6	Inspect invoice and ledger	Invoice, payment and ledger agree on amount	Redacted database/query evidence
7	Repeat with ages 4, 5, 17, 18 and 59	Each boundary produces the approved fee	Boundary execution report
8	Review the evidence with a second tester	Expected and actual results are independently checked	Reviewer initials/date

Y.2 Walkthrough: appointment collision and cancellation

This scenario checks the appointment state model. A doctor slot must not be confirmed for two patients at the same time, and cancellation must update the booking state, slot availability, fee ledger and notifications consistently. A visual message that says ‘booking failed’ does not prove that no reservation was written to the database; both response and persisted state should be inspected.

For the collision test, prepare two synthetic patients and one available slot. Synchronize the requests so they arrive as close together as practical. The expected result is exactly one confirmed reservation and one controlled conflict response. Then cancel the successful booking at several time boundaries, including 2 hours and 1 minute before, exactly 2 hours before, and 1 hour and 59 minutes before the appointment. Under the strict wording in the assignment, the ₹100 fee applies only when the cancellation is less than two hours before the appointment; confirm that interpretation with the authorized owner.

Scenario ID	Setup/action	Expected result	Secondary checks
E2E-APT-01	Two users submit the same slot concurrently	Only one confirmed booking	Database uniqueness and response codes
E2E-APT-02	Retry the winning request after a timeout	Same booking result is returned or safely reconciled	No duplicate active reservation
E2E-APT-03	Cancel 2 h 1 min before appointment	No late fee	Appointment and ledger states agree
E2E-APT-04	Cancel exactly 2 h before appointment	No fee if approved strict rule is confirmed	Boundary decision documented
E2E-APT-05	Cancel 1 h 59 min before appointment	₹100 late fee	One fee entry only
E2E-APT-06	Cancel then rebook the released slot	Slot is available only after cancellation commits	No stale reservation
E2E-APT-07	Reschedule into a slot taken concurrently	One consistent final state	Old slot release and new slot reservation atomic

Y.3 Walkthrough: prescription validity and report privacy

The pharmacy scenario verifies that an invalid prescription is blocked before a dispense transaction is created. Because the exact inclusivity of the seventh day is a policy question, the test should not treat the boundary as settled until the pharmacy or clinical owner approves it. A test report should record the approved interpretation, prescription date, test date, result and whether any dispense record was created.

The report privacy scenario checks authorization at the server boundary. A patient who changes an identifier in a URL or API request must not obtain another patient's report. The test should use two synthetic patient accounts and verify both the denied response and the absence of unauthorized data in the response body. Direct navigation, expired sessions and role changes should be covered because hiding a link in the interface is not an authorization control.

Test	Action	Expected outcome	Evidence
RX-BOUND-01	Use a prescription clearly inside the approved validity window	Dispensing may proceed if all other rules pass	Approved policy and dispense record
RX-BOUND-02	Use a prescription clearly outside the window	Dispensing is rejected	Response and absence of dispense record
RX-BOUND-03	Use the exact boundary date	Outcome follows signed policy interpretation	Decision reference and result
RX-DUP-01	Submit dispense action twice	Only one dispense transaction succeeds	Transaction IDs and stock state
PRIV-01	Patient A requests Patient B's report	Access denied	Request/response with synthetic aliases
PRIV-02	Use an expired session to call report API	Re-authentication or denial	Session state and response

PRIV-03	Receptionist attempts pharmacist-only action	Action denied and recorded as required	Role matrix and audit event

ANNEXURE Z – SAMPLE REQUIREMENT-TO-RELEASE EVIDENCE PACK

A release evidence pack allows a reviewer to trace a business rule from its approved wording to test design, execution, defects and final release decision. It should be concise enough to review but detailed enough to reproduce the conclusion. Each referenced item must have a stable identifier and the pack must identify missing evidence rather than silently treating it as complete.

Z.1 Evidence map

Evidence item	Minimum content	Owner	Review question
Approved requirements	Versioned business rules and ambiguity decisions	Business owner	Are the rules clear and approved?
Risk register	Likelihood, impact, rationale and treatment	QA lead / service owner	Are high-impact failures represented?
Test design	IDs, preconditions, steps, expected results and links	QA	Are boundaries and negative paths included?
Build evidence	Commit/build identifier, schema and environment	Development/operations	Can this result be tied to a specific candidate?
Execution results	Pass/fail/blocked/not run and supporting output	Test executor	Are outcomes based on actual execution?
Defect records	Reproduction, severity, owner, status and evidence	Defect owner	Are unresolved critical defects visible?
Retest evidence	Corrected build and regression result	QA	Did the fix resolve the failure without regression?
Performance report	Workload, duration, percentiles, errors and environment	Performance tester	Does the result match the approved target?
Security/privacy evidence	Authorization checks and redaction review	Security/QA	Was access checked at the server boundary?
Release decision	Go/no-go, exceptions, approver and date	Release authority	Who accepted each residual risk?

Z.2 Evidence quality rules

Evidence must be specific, attributable, reproducible and proportionate to risk. ‘Tested successfully’ is weak evidence because it does not identify the build, input, expected result or observed outcome. A stronger record says which test ID ran against which build, what synthetic data was used, what result was observed, where the logs are stored and who reviewed the result. Sensitive information should be redacted and access to evidence should follow the same least-privilege principles as the application.

A screenshot can support evidence but rarely proves the full state of a transaction. For payment idempotency, capture the request identifiers and transaction count before and after retry, along with the gateway result where available. For appointment concurrency, preserve the two request outcomes and final reservation state. For

prescription validity, record the approved policy and confirm that the failed attempt did not create a dispense record.

The pack should clearly separate planned evidence from executed evidence. Test cases marked Not Run or Blocked must remain visible. If a performance test was not run in a representative environment, the report should state that the response-time requirement remains unverified. A conditional release decision, if authorized, must identify the risk owner, containment, monitoring, review date and conditions that would trigger rollback.

Z.3 Release decision worksheet

Gate	Pass condition	Status	Exception / evidence reference
Business-rule correctness	Approved fee and cancellation boundaries pass	Fill after execution	Link actual run report
Payment integrity	Retries and duplicate callbacks create no duplicate charge	Fill after execution	Link transaction evidence
Appointment integrity	Concurrent booking preserves slot uniqueness	Fill after execution	Link concurrency report
Pharmacy validity	Invalid prescriptions cannot be dispensed	Fill after policy approval and execution	Link clinical approval
Privacy and authorization	Unauthorized report access is denied	Fill after execution	Link security evidence
Performance	Approved response-time criterion met at 800 concurrent users	Fill after representative load test	Link load report
Regression	Required critical regression suite passes	Fill after execution	Link CI report
Operations	Monitoring, rollback and recovery are ready	Fill after rehearsal	Link readiness record
Final authorization	Authorized owner records go/no-go decision	Fill at release meeting	Link signed decision

The worksheet is intentionally not pre-filled with ‘Pass’. The correct status must come from actual evidence. This distinction is important in academic reports as well as production testing: a test design demonstrates how a requirement will be checked, while an execution report demonstrates what happened when it was checked.

SUPPLEMENTARY TECHNICAL NOTES – CALCULATION AND INTERPRETATION

Worked calculation: defect-removal efficiency

The supplied scenario states 135 defects were identified before release and 29 were found after release. Under the conventional calculation used in this report, defect-removal efficiency is pre-release defects divided by the total known defects, multiplied by 100. The arithmetic is $135 \div (135 + 29) \times 100 = 135 \div 164 \times 100 \approx 82.32\%$. The escaped-defect share is $29 \div 164 \times 100 \approx 17.68\%$. These percentages describe the provided counts and do not imply that each defect had equal impact.

The calculation depends on consistent counting. If duplicate reports are counted twice, if reopened issues are treated inconsistently, or if post-release defects have not yet been fully discovered, the result changes. A better

report therefore records the counting rules, time window, severity distribution and source of the defect data. The metric should be used alongside analysis of the 29 escaped defects, especially whether any involved patient safety, privacy or financial integrity.

Worked calculation: fee rule

Age band	Base fee	Concession	Calculation	Payable
Below 5	₹500	100%	₹500 × 0%	₹0
5–17	₹500	20%	₹500 × 80%	₹400
18–59	₹500	0%	₹500 × 100%	₹500
60 and above	₹500	40%	₹500 × 60%	₹300

These values assume the concession is applied to the stated ₹500 base consultation fee and that no additional taxes, service fees or insurance adjustments are part of the assignment rule. If the real implementation includes additional components, they should be tested separately and the business owner should confirm the order of calculation.

Worked calculation: test-budget balance

A 100 person-hour test budget is a capacity constraint, not a reason to omit high-impact checks. The proposed allocation in Task 4 totals 100 hours. During execution, the team should compare planned hours with actual effort and record the reasons for variance. If defect diagnosis consumes additional time, the team should reassess low-priority breadth and communicate the residual risk rather than silently dropping critical regression or release review.

Control	Question to ask before reducing effort	Decision principle
Requirement review	Can an unresolved rule change patient charges or dispensing eligibility?	Do not bypass critical clarification
Automation	Will this test be repeated after future changes?	Automate stable, high-value checks
Concurrency	Could simultaneous requests corrupt shared state?	Retain for booking/payment invariants
Performance	Could slow response trigger retries or disrupt peak use?	Use representative workload evidence
Exploratory breadth	Is there overlap with stronger deterministic tests?	Reduce duplicate repetition first
Reporting	Can the release decision be audited later?	Retain evidence and explicit risk ownership

ANNEXURE AA – TEST DESIGN REVIEW CHECKLIST

A structured test design review checks whether the test suite can detect the failures that matter, whether expected results are independently derived, and whether the evidence will be sufficient for a release decision. The review should be performed before execution for high-risk cases and repeated after major requirement changes. A checklist does not replace judgement; it makes omissions visible and helps reviewers ask consistent questions.

For MediQueue, reviewers should pay particular attention to decision boundaries, transaction state, concurrency, duplicate requests, authorization and recovery from partial failure. The review must also identify assumptions that need approval. If a prescription-validity boundary is unclear, the test should be recorded as awaiting a policy decision rather than assigning an arbitrary expected result.

Review area	Questions	Acceptable evidence	Action if missing
Requirement mapping	Does each critical rule have a test ID?	Traceability matrix	Add missing test or record approved exclusion
Expected results	Are expected values calculated independently?	Reviewed decision table and worked examples	Recalculate and obtain second-person review
Boundary coverage	Are values immediately below, at and above each threshold covered?	Boundary list for age and time rules	Add missing boundary cases
Negative behaviour	Are invalid inputs and denied actions tested?	Negative test cases and expected errors	Define safe failure behaviour
Persistence	Does the test inspect stored state where relevant?	Database/API evidence	Add persistence assertion
Concurrency	Can requests overlap against shared state?	Synchronized test design	Add controlled parallel execution
Retry behaviour	Are timeouts, refreshes and repeated callbacks included?	Fault-injection/replay cases	Add retry-safe tests
Privacy	Is test data synthetic and evidence redacted?	Data plan and evidence review	Replace data and redact records
Reproducibility	Can another tester repeat the test?	Build, setup and fixture instructions	Improve preconditions and steps
Release relevance	Is the test linked to a risk or requirement?	Risk and requirement IDs	Explain purpose or remove duplicate

ANNEXURE AB – DEFECT LEAKAGE REVIEW AND PREVENTION PLAN

Defect leakage review asks why a defect was not detected at an earlier stage and what change would make the failure less likely to escape again. It should not be limited to the final testing team. A defect may originate in an ambiguous requirement, an architectural decision, an implementation error, a test-data limitation or a release-process gap. The purpose is to strengthen the system of controls rather than assign blame.

The supplied case reports 29 post-release defects. That number should be analysed by severity, module, origin and detection gap if the detailed defect log is available. Without the complete log, the report can propose a review method but cannot truthfully claim a specific distribution. The sample categories below are a structure for analysis; actual counts must be calculated from verified records.

Review question	Evidence to inspect	Possible prevention action	Effectiveness measure
Was the requirement ambiguous?	Original wording, clarifications and acceptance criteria	Decision table and owner sign-off	Fewer ambiguity-related escapes
Was the code path untested?	Coverage and linked test cases	Add unit/API test for missing branch	Branch or requirement coverage improves
Was integration behaviour missed?	Interface contracts and end-to-end evidence	Add contract/integration test	Fewer interface-related escapes
Was a race condition missed?	Concurrency test plan and database constraints	Atomic operation and synchronized regression	No duplicate active state under collision
Was retry behaviour absent?	Timeout and callback scenarios	Idempotency and fault-injection tests	One transaction per business operation

Was test data unrealistic?	Fixture definitions and production incident data patterns	Expand synthetic representative fixtures	Fewer data-shape-related failures
Was release evidence incomplete?	Gate checklist and waiver record	Mandatory owner and evidence fields	Every exception has traceable approval
Did the fix fail to prevent recurrence?	Reopened and related defects	Root-cause action plus regression link	Reduced recurrence in later releases

A prevention action should be specific, assigned and testable. For example, ‘improve concurrency testing’ is too vague. A stronger action is ‘add a test that synchronizes two booking requests for the same doctor and slot, then asserts that one and only one active reservation exists’. The action is complete when the test is in the controlled suite, its result is linked to a build, and the team has reviewed whether the design control is sufficient.

ANNEXURE AC – FINAL SUBMISSION QUALITY CONTROL

Before submission, the report should be checked for content quality, formatting consistency and evidence integrity. Page count alone is not a quality measure. The purpose of the expansion is to provide deeper explanations, test matrices, worked scenarios, decision tables and reusable templates while keeping all illustrative data clearly labelled. The report should not state that a tool was executed, a hospital system was tested or a measured result was obtained unless that activity actually occurred.

Check	What to verify	Completion evidence
Content alignment	All ten tasks address the MediQueue assignment and its stated rules	Task-by-task review
Terminology	Severity, priority, status, oracle and traceability are used consistently	Editorial review
Calculations	Fee values, DRE and budget totals are independently recomputed	Calculation check
Illustrative values	Modelled outcomes and synthetic examples are labelled	Evidence audit
Tables	Headers are readable and rows are not split unnecessarily	Document inspection
Page layout	No unnecessary blank pages or isolated headings	Rendered PDF review
Font and spacing	Times New Roman and 1.5 body line spacing are consistent	Style inspection
Border	Single black page border is present	Rendered page inspection
References	Sources are described honestly; no unsupported citation is invented	Reference review
Execution claims	Not Run and Blocked cases are not labelled Pass	Execution record review
Student details	Name, registration number and course information are correct	Front-page check
Final file	DOCX opens and remains editable	Reopen in Word/LibreOffice

Final submission note: update any fields that require actual execution, attach genuine screenshots or test reports if requested by the instructor, and verify the final page count in the word processor used for submission. Pagination can vary slightly between Microsoft Word and LibreOffice because fonts, printer settings and layout engines differ.

ANNEXURE AD – TEST REVIEW SIGN-OFF RECORD

The test review sign-off records that the reviewer examined the test basis, expected results, traceability and limitations before the evidence was used to support a release decision. Sign-off does not mean that every planned test passed. It confirms that the status of each test is represented honestly, unresolved assumptions are visible, and the evidence is sufficient for the decision being requested. A reviewer should be able to distinguish a test that passed, a test that failed, a test blocked by the environment and a test that has not been run.

For this assignment, the sign-off should confirm that the consultation fee values follow the stated ₹500 base fee and concession bands; that the cancellation threshold is interpreted consistently; that the prescription-validity boundary has an approved interpretation; and that payment and appointment tests assert persisted state rather than relying only on screen messages. Performance claims must be tied to a representative workload and an agreed interpretation of the two-second target at 800 concurrent users. If these activities have only been designed and not executed, the sign-off must describe the document as a test plan and must not certify the software as passing.

The reviewer should record any limitation that could affect the conclusion, including missing source records, unavailable environments, incomplete defect history, unverified test data or differences between the assignment scenario and an actual implementation. Any residual risk must have an owner and an explicit decision. The sign-off should be revisited if a material requirement, build, database schema or payment integration changes after review.

Review item	Reviewer / role	Decision	Date / evidence reference
Requirement and ambiguity review	Business / clinical owner	Approved / pending	Enter actual reference
Test design and oracle review	QA reviewer	Approved / revise	Enter actual reference
Critical execution evidence	Test lead	Pass / fail / not run	Enter actual run ID
Open defect and risk review	Release authority	Accepted / blocked	Enter actual decision
Final report check	Student / reviewer	Complete / revise	Enter date and version

ANNEXURE AE – TESTING FLOW DIAGRAMS AND AUTOMATION EXAMPLES

This annexure supplements the report with practical visual aids and sample automation code. The diagrams explain a typical workflow and how JUnit assertions can work with Selenium browser actions. The code is illustrative and must be adapted to the actual MediQueue page structure, approved requirements, test environment and dependency versions. It has not been executed against a live hospital application; the sample output is a model of how a test report may look.

AE.1 End-to-end software testing flow

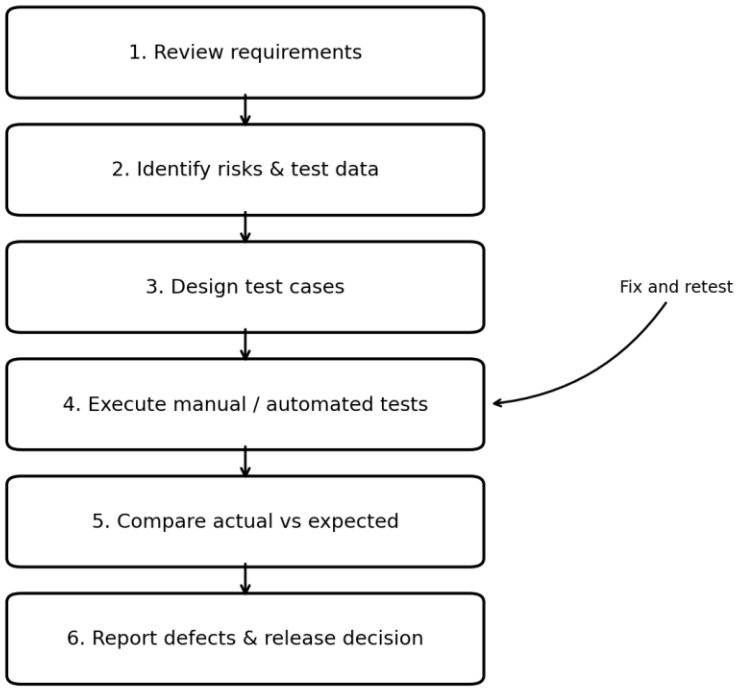


Figure AE.1. Typical testing flow for the MediQueue case. A failed comparison creates a defect; after a correction, the affected test and relevant regression tests are repeated. The release decision should consider unresolved high-severity defects and the quality of the evidence, not only the overall pass percentage.

AE.2 Example automation architecture

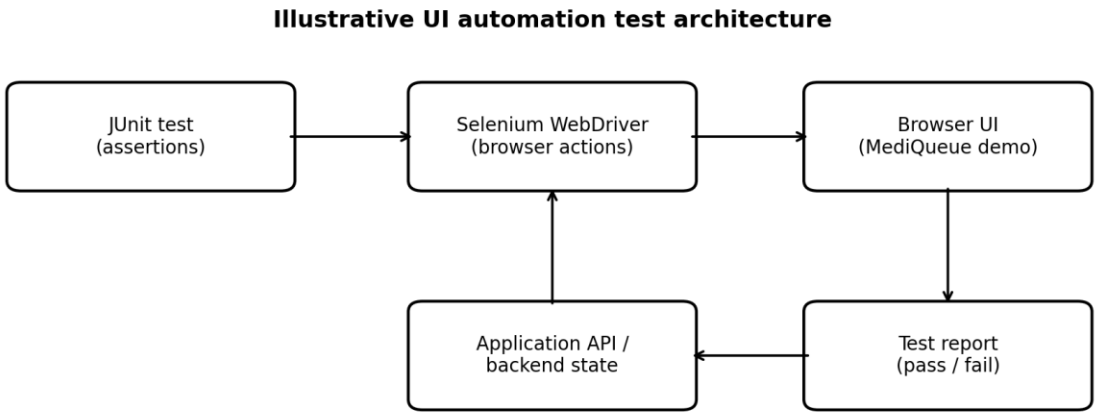


Figure AE.2. Conceptual relationship between JUnit, Selenium WebDriver, the browser interface and test evidence. Selenium performs browser interactions; JUnit organizes tests and evaluates assertions. Backend checks may be required to confirm that a user-interface message corresponds to a correct persisted transaction.

AE.3 Example JUnit test for consultation fee calculation

A unit test should isolate the fee rule from the browser and verify representative values around the age bands. The example assumes that a FeeCalculator class exists and that the expected policy is approved. The method and class names are examples, not a claim about the implementation in the supplied case.

```
import static org.junit.jupiter.api.Assertions.assertEquals;
import org.junit.jupiter.api.Test;

class FeeCalculatorTest {

    @Test
    void appliesApprovedAgeBandConcessions() {
        FeeCalculator calculator = new FeeCalculator(500);

        assertEquals(0,   calculator.feeForAge(4));
        assertEquals(400, calculator.feeForAge(5));
        assertEquals(400, calculator.feeForAge(17));
        assertEquals(500, calculator.feeForAge(18));
        assertEquals(500, calculator.feeForAge(59));
        assertEquals(300, calculator.feeForAge(60));
    }
}
```

Expected interpretation: if all six assertions succeed, the test framework marks this test as passed. If one assertion fails, the test reports the expected value and the observed value for the failing case. The age-boundary values must match the approved requirements. In particular, the assignment's rule wording should be confirmed before this code is treated as the final oracle.

AE.4 Example Selenium UI test with JUnit 5

The following sample demonstrates a browser-level check. It assumes a test environment exposes stable element IDs and a seeded synthetic patient. Replace the example URL and selectors with the actual test environment values. Never use real patient data in demonstration automation.

```
import static org.junit.jupiter.api.Assertions.assertEquals;
import org.junit.jupiter.api.AfterEach;
import org.junit.jupiter.api.Test;
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;

class ConsultationFeeUiTest {
    private WebDriver driver;

    @Test
    void displaysExpectedFeeForSyntheticPatient() {
        driver = new ChromeDriver();
        driver.get("http://localhost:8080/mediqueue-test");

        driver.findElement(By.id("patient-age")).sendKeys("60");
        driver.findElement(By.id("calculate-fee")).click();

        String displayedFee =
            driver.findElement(By.id("consultation-fee")).getText();

        assertEquals("₹300", displayedFee);
    }

    @AfterEach
    void closeBrowser() {
        if (driver != null) {
            driver.quit();
        }
    }
}
```

This browser test checks the visible fee only. A stronger end-to-end test would additionally inspect the payment request and stored invoice through approved test APIs or database fixtures. The expected fee must be independently derived from the specification, and the test should fail if the page shows a different amount or cannot locate the required elements.

AE.5 Example execution output

The following is an illustrative console-style output, included to show the type of evidence that may be attached to a report. It is not a screenshot of an actual test run and must not be described as a real execution result.

```
Test run: FeeCalculatorTest
  appliesApprovedAgeBandConcessions() ........ PASSED

Test run: ConsultationFeeUiTest
  displaysExpectedFeeForSyntheticPatient() ... PASSED

Summary (illustrative only)
  Tests found:       2
  Tests started:     2
  Tests successful:  2
```

```
Tests failed:      0
Execution status:  EXAMPLE - NOT EXECUTED
```

Evidence field	Example value	Why it matters
Build identifier	To be recorded during execution	Connects the result to the software version
Environment	Local test instance / browser version	Makes the test reproducible
Test data	Synthetic patient alias only	Avoids exposing patient information
Expected result	₹300 for the approved age-60 rule	Provides the independent oracle
Actual result	Record from real test execution	Distinguishes observed evidence from planned behaviour
Status	Pass / Fail / Blocked / Not Run	Prevents planned tests being misreported as passing

AE.6 Suggested automation coverage matrix

Requirement / risk	Suggested test level	Automation idea	Manual review still needed
Age-based consultation fee	Unit + UI	JUnit boundary tests; Selenium display check	Confirm policy wording and invoice consistency
Late cancellation fee	Unit + API + UI	Test times immediately before, at and after the threshold	Confirm inclusivity of the two-hour boundary
Duplicate payment request	API / integration	Replay same idempotency key and compare transaction count	Review gateway and ledger evidence
Concurrent appointment booking	Integration / concurrency	Submit synchronized requests for one slot	Inspect final database state and logs
Prescription validity	Unit + API	Test clearly valid, invalid and exact-boundary dates	Obtain clinical/pharmacy approval for boundary
Report authorization	API + security	Attempt cross-patient access using synthetic accounts	Review audit trail and data exposure
Response-time target	Performance	Load test at the specified concurrency	Confirm workload model and percentile criterion

Automation should prioritize stable, repeatable, high-value checks. It should not attempt to replace clinical or business approval of ambiguous rules, exploratory review of unexpected behaviour, or the release authority's assessment of residual risk.

ANNEXURE AF – 50 COMPACT AUTOMATION AND VALIDATION EXAMPLES

This annexure adds fifty short code examples and twenty-five matching process diagrams to make the report more practical and visually readable. Examples cover JUnit, Selenium WebDriver, REST API assertions, SQL diagnostics, Python helper checks and CI execution. They are educational templates and must be adapted to the actual application, approved business rules, schema and test environment. No sample output below should be interpreted as proof that the MediQueue application was executed or passed.

Formatting approach: each mini-section contains two related code examples and a compact flow diagram. This keeps the material connected to the report's testing topics while avoiding large empty areas. Code and diagrams are illustrative; execution evidence must be collected from the real test environment.

AF.1 Fee rule – code examples 1 and 2

Code 1: JUnit: fee for age 60

```
assertEquals(300, calculator.feeForAge(60));
```

Checks the approved 40% concession on a ₹500 base fee.

Code 2: JUnit: child fee

```
assertEquals(0, calculator.feeForAge(4));
```

Checks the under-five fee rule.

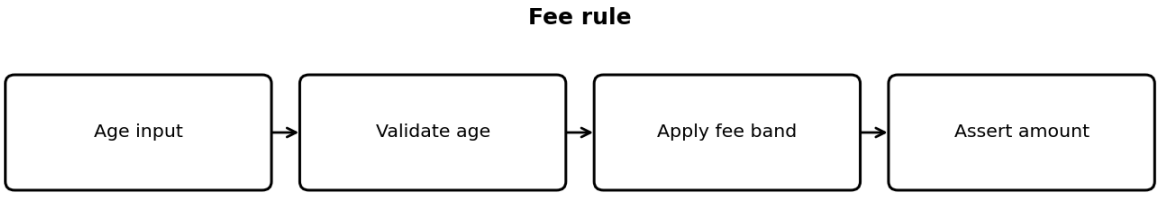


Figure AF.1. Fee rule workflow (illustrative).

Validation focus	Verify fee calculations, totals and rounding.
Pass condition	Correct amount is displayed for valid inputs.
Evidence to record	Input values, calculated total and assertion output.
Negative / boundary check	Try zero, boundary and invalid values.

Practical validation checklist

Review note: before marking this scenario as passed, record the software build, environment, synthetic test data, expected result, observed result and execution timestamp. Where the rule depends on clinical or business interpretation, obtain approval before finalizing the test oracle.

AF.2 Boundary testing – code examples 3 and 4

Code 3: JUnit: boundary age 5

```
assertEquals(400, calculator.feeForAge(5));
```

Checks the first day of the 5–17 age band.

Code 4: JUnit: adult fee

```
assertEquals(500, calculator.feeForAge(30));
```

Checks the standard consultation fee.

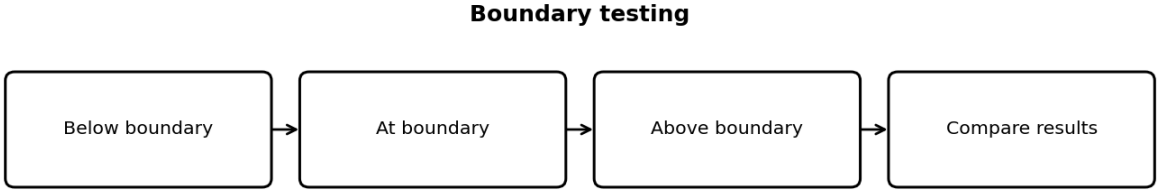


Figure AF.2. Boundary testing workflow (illustrative).

AF.3 Parameterized tests – code examples 5 and 6

Code 5: JUnit: boundary age 18

```
assertEquals(500, calculator.feeForAge(18));
```

Checks the start of the adult band.

Code 6: JUnit: boundary age 59

```
assertEquals(500, calculator.feeForAge(59));
```

Checks the last age before the senior band.

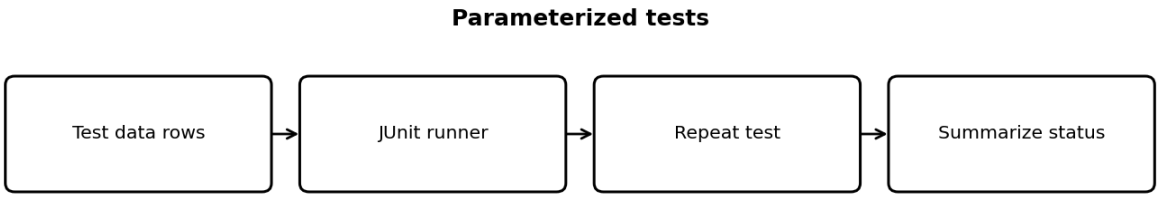


Figure AF.3. Parameterized tests workflow (illustrative).

AF.4 Input validation – code examples 7 and 8

Code 7: JUnit: parameterized test

```
@ParameterizedTest
@CsvSource({"4,0","5,400","60,300"})
void feeRule(int age, int expected) {
  assertEquals(expected, calculator.feeForAge(age));
}
```

Runs the same rule against multiple approved examples.

Code 8: JUnit: invalid age

```
assertThrows(IllegalArgumentException.class,
  () -> calculator.feeForAge(-1));
```

Verifies that an impossible age is rejected safely.

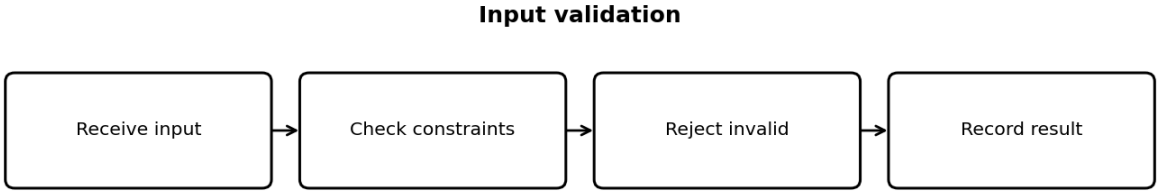


Figure AF.4. Input validation workflow (illustrative).

AF.5 Cancellation policy – code examples 9 and 10

Code 9: JUnit: null input

```
assertThrows(IllegalArgumentException.class,
  () -> calculator.calculate(null));
```

Illustrates defensive validation for missing input.

Code 10: JUnit: exception message

```
var ex = assertThrows(IllegalArgumentException.class,
  () -> calculator.feeForAge(-1));
assertTrue(ex.getMessage().contains("age"));
```

Checks that an invalid input gives a useful error.

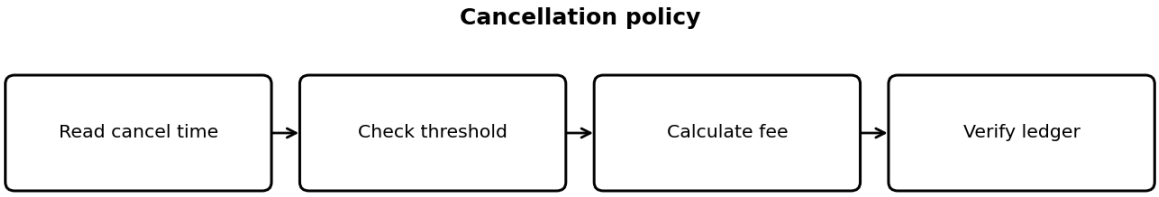


Figure AF.5. Cancellation policy workflow (illustrative).

Validation focus	Verify the stated behaviour against its requirement.
Pass condition	The observed result matches the expected outcome.
Evidence to record	Test inputs, build/environment and execution result.
Negative / boundary check	Repeat with boundary, invalid and unauthorized inputs.

Practical validation checklist

Validation focus	Verify the stated behaviour against its requirement.
Pass condition	The observed result matches the expected outcome.
Evidence to record	Test inputs, build/environment and execution result.
Negative / boundary check	Repeat with boundary, invalid and unauthorized inputs.

Practical validation checklist

Validation focus	Verify the test can be repeated consistently.
Pass condition	The same controlled input produces the expected result.
Evidence to record	Build version, environment, test data and result.
Negative / boundary check	Re-run after a code change and compare results.

Practical validation checklist

Validation focus	Verify the test can be repeated consistently.
Pass condition	The same controlled input produces the expected result.
Evidence to record	Build version, environment, test data and result.
Negative / boundary check	Re-run after a code change and compare results.

Practical validation checklist

Review note: before marking this scenario as passed, record the software build, environment, synthetic test data, expected result, observed result and execution timestamp. Where the rule depends on clinical or business interpretation, obtain approval before finalizing the test oracle.

AF.6 Prescription validity – code examples 11 and 12

Code 11: JUnit: cancellation threshold

```
assertEquals(0, policy.feeForMinutesBefore(120));
```

Illustrates the exact two-hour boundary; confirm policy inclusivity.

Code 12: JUnit: late cancellation

```
assertEquals(100, policy.feeForMinutesBefore(119));
```

Checks the late-cancellation rule just inside the threshold.

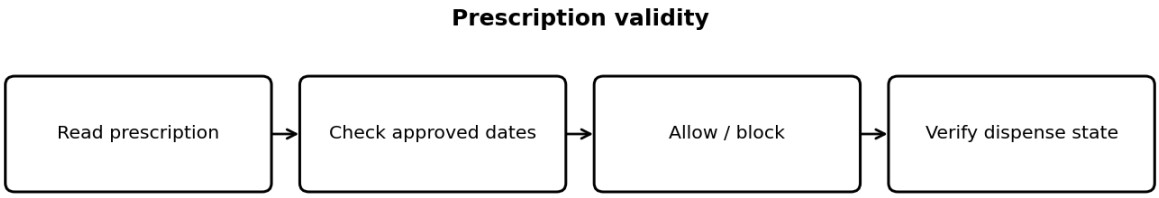


Figure AF.6. Prescription validity workflow (illustrative).

AF.7 Payment idempotency – code examples 13 and 14

Code 13: JUnit: valid prescription

```
assertTrue(policy.isValid(validPrescription, today));
```

Uses a synthetic prescription within the approved validity window.

Code 14: JUnit: expired prescription

```
assertFalse(policy.isValid(expiredPrescription, today));
```

Checks that an expired prescription is rejected.

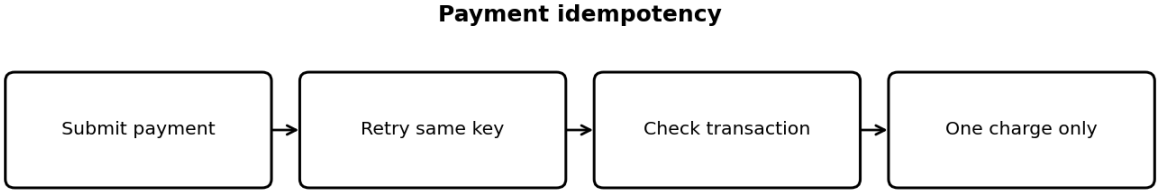


Figure AF.7. Payment idempotency workflow (illustrative).

AF.8 Slot uniqueness – code examples 15 and 16

Code 15: JUnit: duplicate payment

```
service.pay(request);
service.pay(request);
assertEquals(1, ledger.countByKey(request.key()));
```

Checks idempotency using a stable synthetic request key.

Code 16: JUnit: appointment slot

```
assertTrue(repository.reserve(slotId, patientA));
assertFalse(repository.reserve(slotId, patientB));
```

Checks that a slot cannot be reserved twice.

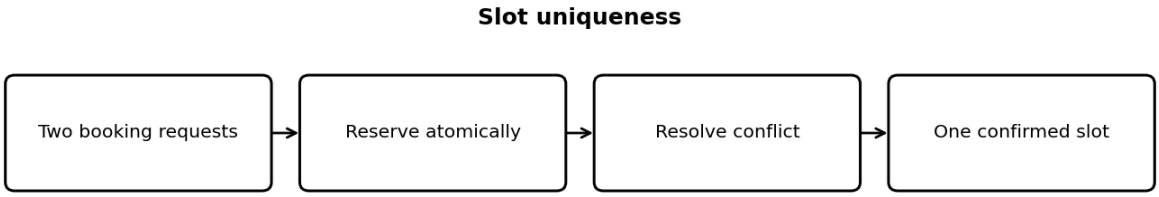


Figure AF.8. Slot uniqueness workflow (illustrative).

AF.9 Audit trail – code examples 17 and 18

Code 17: JUnit: status transition

```
assertEquals(BookingStatus.CANCELLED, booking.status());
```

Confirms the final state after a successful cancellation.

Code 18: JUnit: audit event

```
assertTrue(auditLog.containsEvent("REPORT_ACCESS_DENIED"));
```

Illustrates an audit assertion for a denied action.

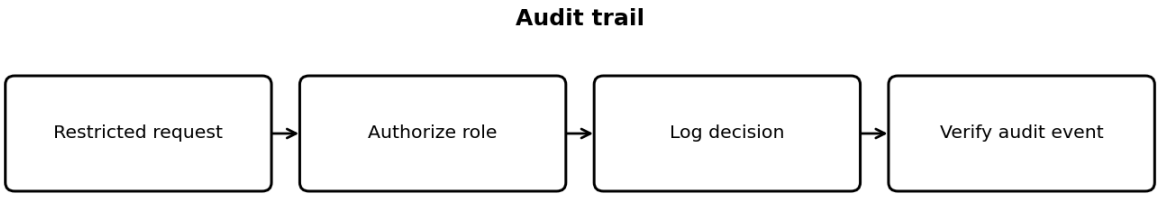


Figure AF.9. Audit trail workflow (illustrative).

AF.10 API assertion – code examples 19 and 20

Code 19: JUnit: API status

```
assertEquals(403, response.statusCode());
```

Checks a forbidden request in an API integration test.

Code 20: JUnit: JSON field

```
assertEquals("CANCELLED", body.get("status").asText());
```

Checks a response field rather than relying only on a page message.

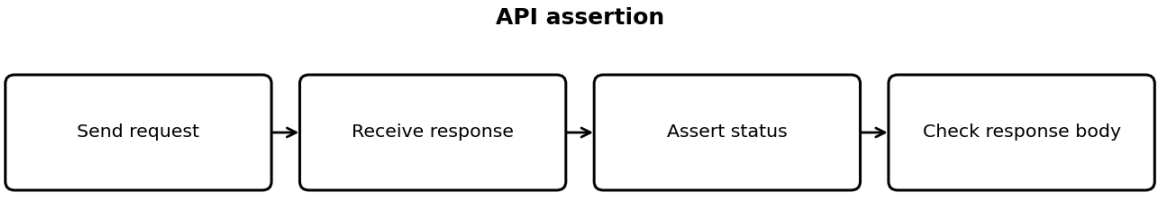


Figure AF.10. API assertion workflow (illustrative).

Validation focus	Verify endpoint contract, status and payload.
Pass condition	Response matches the documented schema and rule.
Evidence to record	Request, response, status code and timestamp.
Negative / boundary check	Try missing fields, malformed input and unauthorized access.

Practical validation checklist

Validation focus	Verify the stated behaviour against its requirement.
Pass condition	The observed result matches the expected outcome.
Evidence to record	Test inputs, build/environment and execution result.
Negative / boundary check	Repeat with boundary, invalid and unauthorized inputs.

Practical validation checklist

Validation focus	Verify the stated behaviour against its requirement.
Pass condition	The observed result matches the expected outcome.
Evidence to record	Test inputs, build/environment and execution result.
Negative / boundary check	Repeat with boundary, invalid and unauthorized inputs.

Practical validation checklist

Validation focus	Verify the stated behaviour against its requirement.
Pass condition	The observed result matches the expected outcome.
Evidence to record	Test inputs, build/environment and execution result.
Negative / boundary check	Repeat with boundary, invalid and unauthorized inputs.

Practical validation checklist

Validation focus	Verify the stated behaviour against its requirement.
Pass condition	The observed result matches the expected outcome.
Evidence to record	Test inputs, build/environment and execution result.
Negative / boundary check	Repeat with boundary, invalid and unauthorized inputs.

Practical validation checklist

Review note: before marking this scenario as passed, record the software build, environment, synthetic test data, expected result, observed result and execution timestamp. Where the rule depends on clinical or business interpretation, obtain approval before finalizing the test oracle.

AF.11 Browser setup – code examples 21 and 22

Code 21: Selenium: open test app

```
driver.get("http://localhost:8080/mediqueue-test");
```

Opens a local test environment, not a production system.

Code 22: Selenium: enter age

```
driver.findElement(By.id("patient-age")).sendKeys("60");
```

Uses a stable element identifier to enter synthetic data.

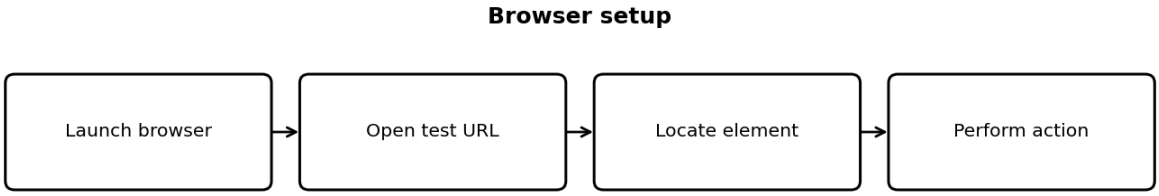


Figure AF.11. Browser setup workflow (illustrative).

AF.12 UI fee check – code examples 23 and 24

Code 23: Selenium: click calculate

```
driver.findElement(By.id("calculate-fee")).click();
```

Triggers the fee calculation from the user interface.

Code 24: Selenium: read fee

```
String fee = driver.findElement(By.id("consultation-fee")).getText();
```

Reads the displayed value for assertion.

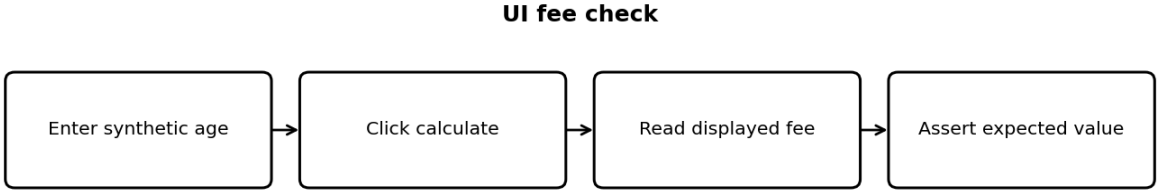


Figure AF.12. UI fee check workflow (illustrative).

AF.13 Explicit wait – code examples 25 and 26

Code 25: Selenium: assert fee

```
assertEquals("₹300", fee);
```

Compares actual interface text with the independently derived result.

Code 26: Selenium: wait for result

```
new WebDriverWait(driver, Duration.ofSeconds(5))
  .until(ExpectedConditions.visibilityOfElementLocated(
    By.id("consultation-fee")));
```

Waits for a visible result instead of using a fixed sleep.

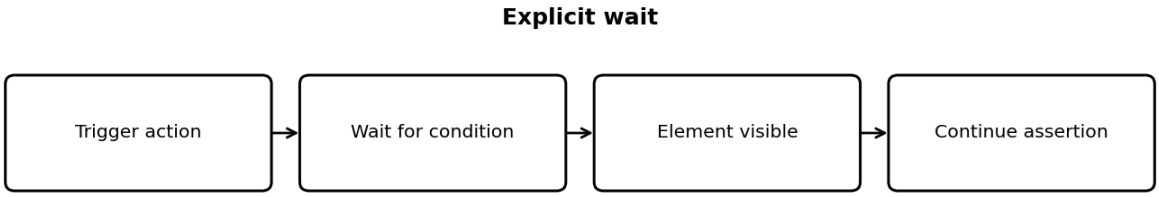


Figure AF.13. Explicit wait workflow (illustrative).

AF.14 Booking flow – code examples 27 and 28

Code 27: Selenium: verify error

```
assertTrue(driver.findElement(By.id("form-error")).isDisplayed());
```

Checks that invalid input feedback is visible.

Code 28: Selenium: select patient

```
driver.findElement(By.id("patient-search")).sendKeys("SYN-1002");
```

Uses a synthetic patient alias in test data.

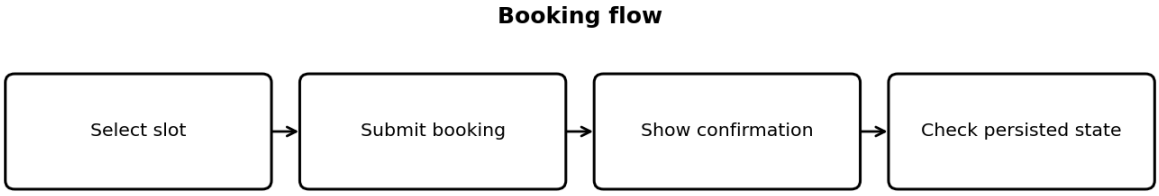


Figure AF.14. Booking flow workflow (illustrative).

AF.15 Cancellation flow – code examples 29 and 30

Code 29: Selenium: submit booking

```
driver.findElement(By.id("confirm-booking")).click();
```

Exercises the booking confirmation interaction.

Code 30: Selenium: booking result

```
assertTrue(driver.findElement(By.id("booking-confirmation")).isDisplayed());
```

Checks confirmation while backend state should also be verified.

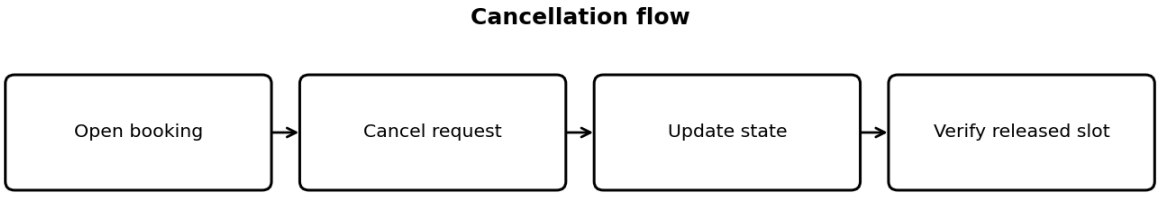


Figure AF.15. Cancellation flow workflow (illustrative).

Validation focus	Verify the stated behaviour against its requirement.
Pass condition	The observed result matches the expected outcome.
Evidence to record	Test inputs, build/environment and execution result.
Negative / boundary check	Repeat with boundary, invalid and unauthorized inputs.

Practical validation checklist

Validation focus	Verify the stated behaviour against its requirement.
Pass condition	The observed result matches the expected outcome.
Evidence to record	Test inputs, build/environment and execution result.
Negative / boundary check	Repeat with boundary, invalid and unauthorized inputs.

Practical validation checklist

Validation focus	Verify automated checks run in the intended pipeline.
Pass condition	Pipeline reports pass/fail accurately and preserves evidence.
Evidence to record	Commit/build ID, logs and report artifact.
Negative / boundary check	Confirm a failing test causes a visible failed build.

Practical validation checklist

Validation focus	Verify fee calculations, totals and rounding.
Pass condition	Correct amount is displayed for valid inputs.
Evidence to record	Input values, calculated total and assertion output.
Negative / boundary check	Try zero, boundary and invalid values.

Practical validation checklist

Validation focus	Verify the stated behaviour against its requirement.
Pass condition	The observed result matches the expected outcome.
Evidence to record	Test inputs, build/environment and execution result.
Negative / boundary check	Repeat with boundary, invalid and unauthorized inputs.

Practical validation checklist

Review note: before marking this scenario as passed, record the software build, environment, synthetic test data, expected result, observed result and execution timestamp. Where the rule depends on clinical or business interpretation, obtain approval before finalizing the test oracle.

AF.16 Role authorization – code examples 31 and 32

Code 31: Selenium: cancel booking

```
driver.findElement(By.id("cancel-booking")).click();
```

Exercises the cancellation path.

Code 32: Selenium: confirm cancellation

```
assertEquals("Cancelled", driver.findElement(
  By.id("booking-status")).getText());
```

Checks the visible cancellation status.

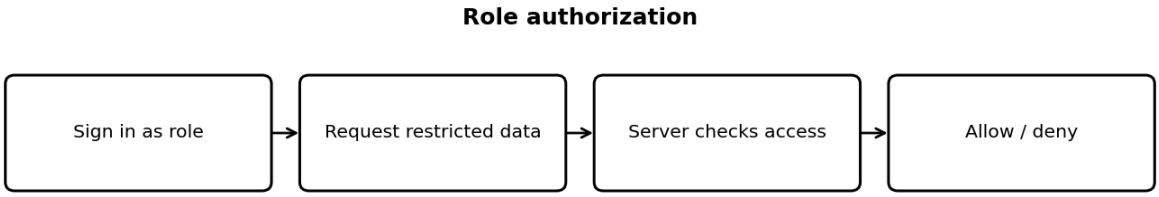


Figure AF.16. Role authorization workflow (illustrative).

AF.17 Screenshot evidence – code examples 33 and 34

Code 33: Selenium: role-based control

```
assertFalse(driver.findElements(By.id("dispense-medicine")).size() > 0);
```

Checks that an unauthorized role does not see a restricted action; server authorization is still required.

Code 34: Selenium: screenshot evidence

```
File shot = ((TakesScreenshot) driver).getScreenshotAs(OutputType.FILE);
```

Illustrates capturing UI evidence during a real test run.

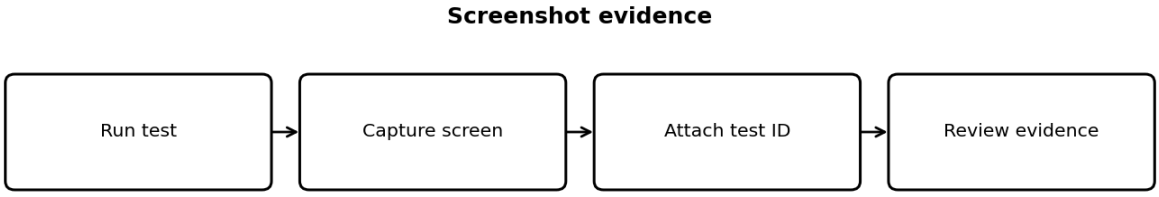


Figure AF.17. Screenshot evidence workflow (illustrative).

AF.18 API performance – code examples 35 and 36

Code 35: Selenium: close browser

```
@AfterEach void close() { if (driver != null) driver.quit(); }
```

Releases browser resources after each test.

Code 36: API: validate status code

```
response.then().statusCode(400);
```

Example REST-assured assertion for invalid input.

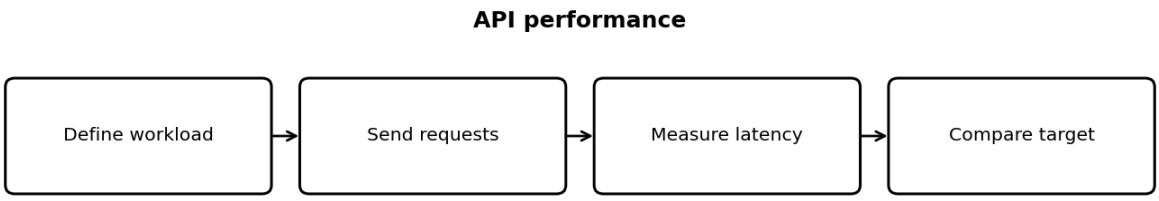


Figure AF.18. API performance workflow (illustrative).

AF.19 SQL reconciliation – code examples 37 and 38

Code 37: API: response time

```
response.then().time(lessThan(2000L));
```

Illustrative threshold check; use a representative load test for concurrency targets.

Code 38: API: deny cross-patient access

```
given().auth().oauth2(patientAToken)
.when().get(reportBUrl)
.then().statusCode(403);
```

Uses synthetic accounts to test object-level authorization.

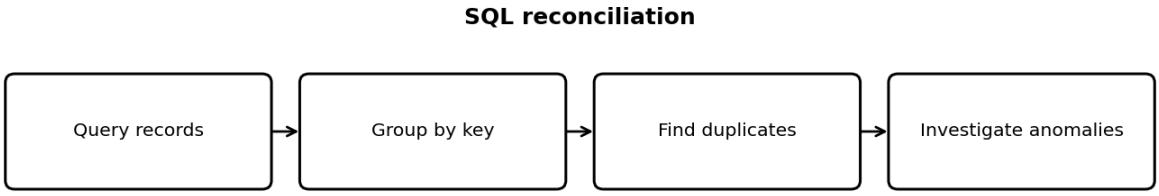


Figure AF.19. SQL reconciliation workflow (illustrative).

AF.20 Synthetic data – code examples 39 and 40

Code 39: API: required field

```
response.then().body("patientId", not(emptyOrNullString()));
```

Checks required response data without exposing real patient details.

Code 40: API: duplicate request key

```
given().header("Idempotency-Key", "TEST-KEY-001")
.when().post("/payments")
.then().statusCode(anyOf(is(200), is(201)));
```

Illustrates a repeatable idempotency-key test; verify transaction count too.

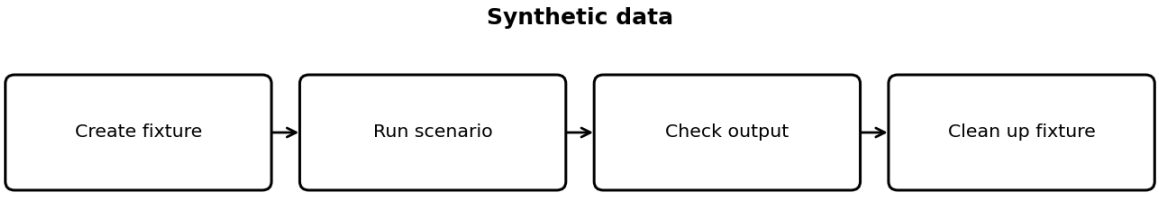


Figure AF.20. Synthetic data workflow (illustrative).

Validation focus	Verify test records are synthetic and fit the scenario.
Pass condition	No real patient identifiers or unintended personal data are used.
Evidence to record	Dataset version, generation rule and validation output.
Negative / boundary check	Scan for accidental real identifiers or duplicates.

Practical validation checklist

Validation focus	Verify automated checks run in the intended pipeline.
Pass condition	Pipeline reports pass/fail accurately and preserves evidence.
Evidence to record	Commit/build ID, logs and report artifact.
Negative / boundary check	Confirm a failing test causes a visible failed build.

Practical validation checklist

Validation focus	Verify endpoint contract, status and payload.
Pass condition	Response matches the documented schema and rule.
Evidence to record	Request, response, status code and timestamp.
Negative / boundary check	Try missing fields, malformed input and unauthorized access.

Practical validation checklist

Validation focus	Verify test evidence is complete and traceable.
Pass condition	Each result links to a requirement, build and execution.
Evidence to record	Requirement ID, test ID, result and timestamp.
Negative / boundary check	Check that failures include reproducible details.

Practical validation checklist

Validation focus	Verify the stated behaviour against its requirement.
Pass condition	The observed result matches the expected outcome.
Evidence to record	Test inputs, build/environment and execution result.

Negative / boundary check	Repeat with boundary, invalid and unauthorized inputs.

Practical validation checklist

Review note: before marking this scenario as passed, record the software build, environment, synthetic test data, expected result, observed result and execution timestamp. Where the rule depends on clinical or business interpretation, obtain approval before finalizing the test oracle.

AF.21 CI pipeline – code examples 41 and 42

Code 41: API: malformed date

```
given().queryParam("date", "not-a-date")
.when().get("/appointments")
.then().statusCode(400);
```

Confirms malformed dates are rejected predictably.

Code 42: API: pagination

```
given().queryParam("page", 0).queryParam("size", 20)
.when().get("/reports").then().statusCode(200);
```

Checks a basic paginated API request.

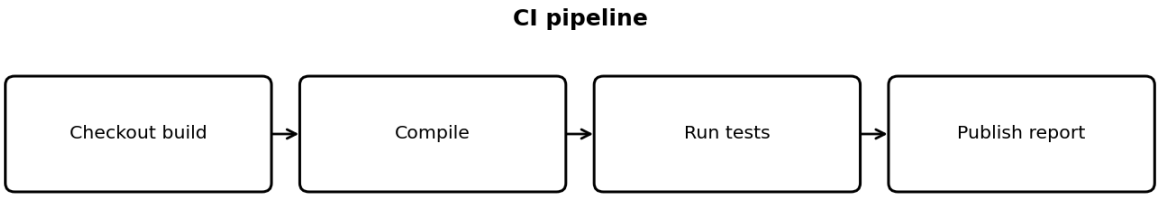


Figure AF.21. CI pipeline workflow (illustrative).

AF.22 Defect lifecycle – code examples 43 and 44

Code 43: SQL: duplicate active slots

```
SELECT slot_id, COUNT(*)
FROM appointments WHERE status='CONFIRMED'
GROUP BY slot_id HAVING COUNT(*) > 1;
```

A diagnostic query for duplicate active bookings; adapt to the schema.

Code 44: SQL: payment reconciliation

```
SELECT invoice_id, COUNT(*)
FROM payments WHERE status='SUCCESS'
GROUP BY invoice_id HAVING COUNT(*) > 1;
```

Helps identify possible duplicate successful payment records.

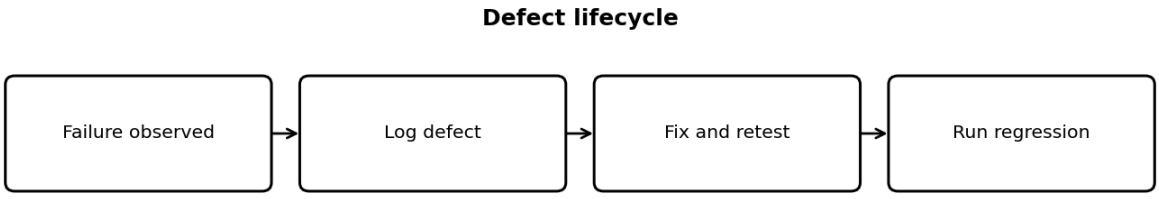


Figure AF.22. Defect lifecycle workflow (illustrative).

AF.23 Release gate – code examples 45 and 46

Code 45: SQL: missing audit events

```
SELECT report_id FROM report_access
WHERE result='DENIED' AND audit_event_id IS NULL;
```

Illustrative audit completeness check; adapt field names.

Code 46: Python: boundary list

```
ages = [4, 5, 17, 18, 59, 60]
for age in ages:
    print(age, expected_fee(age))
```

Generates a concise set of age-boundary checks.

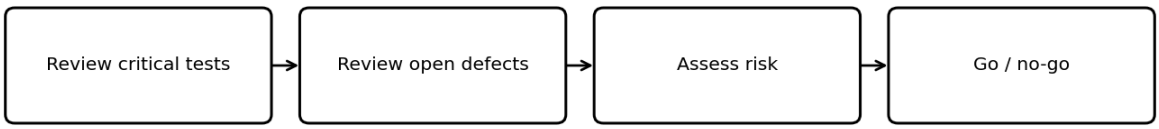


Figure AF.23. Release gate workflow (illustrative).

AF.24 Test evidence – code examples 47 and 48

Code 47: Python: response-time summary

```
times_ms = [180, 210, 195, 240, 205]
print('max_ms =', max(times_ms))
```

Shows a simple descriptive check; percentile analysis needs a larger sample.

Code 48: Python: check test status

```
assert failed_tests == 0, 'Review failed test cases'
```

A small quality gate that should be used only after test execution.

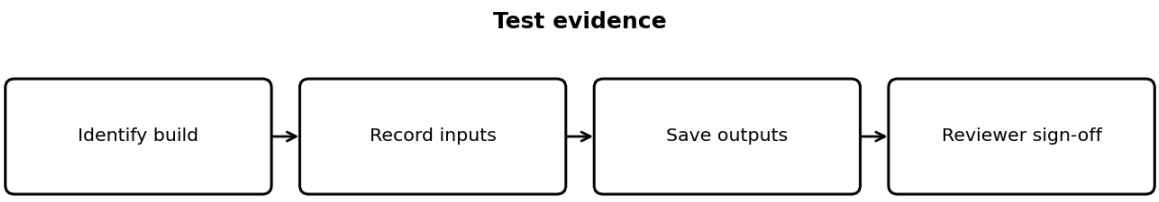


Figure AF.24. Test evidence workflow (illustrative).

AF.25 Privacy test – code examples 49 and 50

Code 49: CI: run unit tests

```
./mvnw test
```

Runs Maven tests in a controlled development or CI environment.

Code 50: CI: package test report

```
mvn surefire-report:report
```

Generates a report from the actual test-run results.

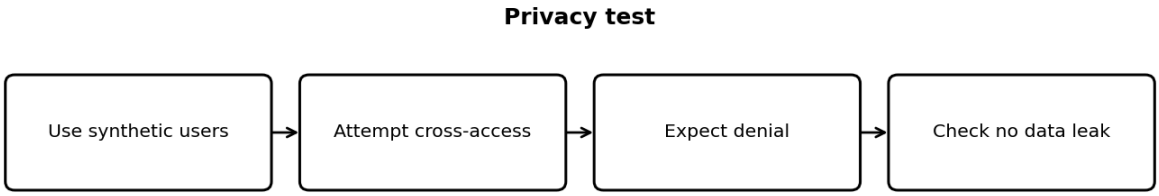


Figure AF.25. Privacy test workflow (illustrative).

Validation focus	Verify that one user cannot access another user's data.
Pass condition	Cross-user access is denied without exposing records.
Evidence to record	Synthetic user IDs, response code and audit log.
Negative / boundary check	Repeat with guessed IDs and altered ownership values.

Practical validation checklist

Validation focus	Verify the test can be repeated consistently.
Pass condition	The same controlled input produces the expected result.
Evidence to record	Build version, environment, test data and result.
Negative / boundary check	Re-run after a code change and compare results.

Practical validation checklist

Validation focus	Verify release criteria before approval.

Pass condition	Release proceeds only when mandatory checks pass.
Evidence to record	Gate checklist, unresolved defects and approval record.
Negative / boundary check	Confirm critical failures block the release.

Practical validation checklist

Validation focus	Verify defect reproduction and closure evidence.
Pass condition	The fix removes the defect without breaking related behaviour.
Evidence to record	Issue ID, reproduction steps, before/after result.
Negative / boundary check	Run regression tests around affected functionality.

Practical validation checklist

Validation focus	Verify automated checks run in the intended pipeline.
Pass condition	Pipeline reports pass/fail accurately and preserves evidence.
Evidence to record	Commit/build ID, logs and report artifact.
Negative / boundary check	Confirm a failing test causes a visible failed build.

Practical validation checklist

Review note: before marking this scenario as passed, record the software build, environment, synthetic test data, expected result, observed result and execution timestamp. Where the rule depends on clinical or business interpretation, obtain approval before finalizing the test oracle.